

파이썬을 이용한 알기쉬운 수치해석

김시조 지음



도서출판 **흥릉**

김시조 (공학박사/교수)

현재 국립안동대학교 공과대학 기계로봇공학과 교수

학력 연세대학교 기계공학과 학사(1984)

POSTECH 기계공학과 석사(1991)

POSTECH 기계공학과 박사(1995)

경력 미국 에크론 대학 교환교수(1997)

미국 미네소타 대학 교환교수(2007)

미국 미시시피 주립 대학 교환교수(2014)

국방과학연구소 연구원(1984~1989)

파이썬을 이용한 알기쉬운 수치해석

인 쇄: 2024년 12월 23일 초판 1쇄

발 행: 2024년 12월 30일 초판 1쇄

저 자: 김시조

발행인: 송 준

발행처: 도서출판 홍릉

주 소: 01093 서울시 강북구 인수봉로 50길 10

등 록: 1976년 10월 21일 제5-66호

전 화: 02-999-2274~5, 903-7037

팩 스: 02-905-6729

e-mail: hongpub@hongpub.co.kr

http://www.hongpub.co.kr

ISBN: 979-11-5600-256-7(93000)

정 가: 18,000원

낙장 및 파본은 구입처나 본사에서 교환하여 드립니다.

판권 소유에 위배되는 사항(인쇄, 복제, 제본)은 법에 저촉됩니다.

저자와의 협의하에 인지는 생략됩니다.

머릿말

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상 시키는데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되어, 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있으며, 또한, 알파고에 이은 알파폴드와 같은 AI 분야 노벨 화학상을 수여 받았다. 이는 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지 보여준다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결 고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히, 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어, Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

끝으로, 이 책이 수치해석 발전에 밑거름이 되기를 바라며, 이로부터 수치해석 분야에서의 AI, 자율주행 관련 연구에 도움이 되기를 희망한다.

김시조

차례

1 장 파이썬의 이해	1
(1.1) 파이썬의 특징	4
(1.2) 변수선언	6
(1.3) 배열(Array)과 행렬(Matrix)	9
(1.4) 튜플(tuple)	13
(1.5) 사전 (Dictionary)	16
(1.6) 수학 연산자 및 NumPy 비교	18
(1.7) 반복 계산	21
(1.8) 조건 계산	25
(1.9) 입출력	29
(1.10) 함수(Function)	32
(1.11) Numpy를 이용한 벡터, 행렬	35
(1.12) 객체지향 프로그래밍(OOP)	40
(1.13) matplotlib를 이용한 그래프 시각화	45
(1.14) 연습문제	50
2 장 비선형 방정식 근 구하기	53
(2.1) 비선형 방정식 근 구하기 개요	54
(2.2) 점진탐색법(Incremental Search Method)	56
(2.3) 이분법(Bisection Method)	60
(2.4) 뉴턴-랩슨법(Newton-Raphson Method)	64
(2.5) 연습문제	69
3 장 선형연립방정식 수치해법	70
(3.1) 선형연립방정식 수치해법 개요	71
(3.2) 가우스 소거법	79
(3.3) LU Decomposition Method	96

(3.4) Jacobi, Gauss-Seidal, SOR 반복법	120
(3.5) 연습문제	130
4 장 수치미분(Numerical Differentiation)	132
(4.1) 수치미분의 개요	133
(4.2) 전진, 후진, 중심차분, Richardson 외삽법	136
(4.3) 연습문제	145
5 장 수치적분(Numerical Integration)	146
(5.1) 리만 (Riemann) 적분	149
(5.2) 사다리꼴 적분(Trapezoidal rule)	155
(5.3) Richardson 외삽법(Extrapolation)	161
(5.4) Romberg Integration	164
(5.5) Simpson's 1/3rd Rule	168
(5.6) Newton-Cotes Quadrature	174
(5.7) Gauss Quadrature Rule(가우스 구적법)	175
(5.8) 연습문제	186
6 장 수치 보간(interpolation)	188
(6.1) 수치 보간 개요	189
(6.2) 뉴턴 분할 차분법	191
(6.3) Lagrange Interpolation 수치 보간	196
(6.4) 스플라인 보간	201
(6.5) 수치보간법들과의 차이점	206
(6.6) 연습문제	212
7 장 수치 회귀(Numerical Regression)	213
(7.1) 최소자승법에 의한 1차 다항식 근사	214
(7.2) 최소자승법에 의한 m 차 다항식 근사	220
(7.3) 연습문제	225
8 장 비선형연립방정식 수치해법	227

(8.1) Newton-Rhaphson Method	228
(8.2) 비선형연립방정식 수치해법	231
(8.3) 연습문제	238
9 장 상미분 방정식 (초기치문제)	239
(9.1) 초기치문제 상미분 방정식 수치해석 개요	241
(9.2) Euler Method	242
(9.3) 2차 Runge-Kutta Method(RK2)	244
(9.4) 4차 Runge-Kutta Method(RK4)	245
(9.5) 고계 상미분방정식을 1계 연립 ODE 변환 방법	247
(9.6) 미분-적분 방정식 수치기법	257
(9.7) 연습문제	262
10 장 Shooting Method [경계치 문제]	264
(10.1) 선형 Shooting Method	265
(10.2) 비선형 Shooting Method	272
(10.3) 연습문제	276
11 장 유한차분법	277
(11.1) 편미분 방정식 개요	278
(11.2) 유한차분법의 수치해석	280
(11.3) 연습문제	290
12 장 유한요소법 (Finite Element Method)	291
(12.1) 유한요소법 개요	293
(12.2) 형상함수(Shape Function)	295
(12.3) 실제좌표계에서의 유한요소법(FEM)	300
(12.4) 기준좌표계에서의 유한요소법(FEM)	305
(12.5) 연습문제	316
찾아보기	318

1 장 파이썬의 이해

(1.1) 파이썬의 특징

(1.1.1) 플랫폼 독립성

파이썬 코드를 Windows, macOS, Linux 등 다양한 운영 체제에서 동일한 결과를 얻을 수 있다.

```
with open('example.txt', 'r') as file:
    print(file.read())
```

(1.1.2) 인터프리터 언어

파이썬 코드를 작성하고, 바로 실행하여 결과를 확인할 수 있다. 파이썬 셸에서 간단한 계산을 바로 수행해 볼 수 있다.

```
print(2 + 3)
```

(1.1.3) 블록과 줄바꿈

① 예제: 조건문과 반복문의 블록

코드	출력 결과
<pre>n = 5 if n > 0: # 조건문 블록 시작 print("n is positive") for i in range(n): # 반복문 블록 시작 print(f"Iteration {i}") print("Loop finished") print("Outside all blocks")</pre>	<pre>n is positive Iteration 0 Iteration 1 Iteration 2 Iteration 3 Iteration 4 Loop finished</pre>

Outside all blocks

(1.1.4) 객체지향 프로그래밍 지원

파이썬에서 클래스를 정의하고 객체를 생성하여 사용하는 방법은 객체지향 프로그래밍의 핵심 개념을 적용한다.

코드	출력 결과
<pre>class Animal: def __init__(self, name): self.name = name def speak(self): return "makes a sound" class Dog(Animal): def speak(self): return "barks" pet = Dog("Buddy") print(pet.name, pet.speak())</pre>	Buddy barks

(1.1.5) 동적 타이핑

변수에 대한 타입 선언 없이도 다양한 타입의 값을 할당하고 변경할 수 있다. 이는 코드 작성을 더 유연하고 간단하게 해준다.

코드	출력 결과
<pre>number = 10 # 처음에는 정수를 할당 print(number) number = "ten" # 나중에는 문자열을 할당 print(number)</pre>	<pre>10 ten</pre>

(1.1.6) NumPy 라이브러리

코드	출력 결과
<pre>import numpy as np a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) print(a + b) # 배열 간 덧셈</pre>	[5 7 9]

(1.2) 변수선언

(1.2.1) 자료 형, 변수 선언과 변환

- ① 자료형과 변수
- ② 파이썬 메모리 관리
- ③ 다양한 변수 선언에 대한 파이썬 코드와 실행 결과

코드	출력 결과	설명
x = 10 print(x)	10	정수 값 10을 변수 x에 할당하였다. 파이썬은 값에 따라 자료형을 자동으로 결정하며, 여기서 x는 int 자료형으로 저장된다.
y = 3.14 print(y)	3.14	실수 값 3.14를 변수 y에 할당하였다. 파이썬은 이를 float 자료형으로 인식한다.
name = "Alice" print(name)	Alice	문자열 "Alice"를 변수 name에 할당하였다. 파이썬은 문자열을 str 자료형으로 처리한다.
is_valid = True print(is_valid)	True	Boolean 값 True를 변수 is_valid에 할당하였다. True는 논리적으로 참을 나타내며, 연산 시에는 숫자 1로 간주된다.
a = 5 b = a + 2 print(a, b)	5 7	변수 a에 정수 값 5를 저장하고, 변수 b에 a + 2의 결과를 저장하였다. 변수 간 연산이 정상적으로 수행되었음을 보여준다.
c = 10 c = "Hello" print(c)	Hello	변수 c에 처음에는 정수 10을 저장하고, 이후 문자열 "Hello"를 다시 저장하였다. 파이썬의 동적 타이핑 특성으로 같은 변수에 다른 자료형을 저장할 수 있다.
try: print(z) except NameError as e: print(e)	name 'z' is not defined	선언되지 않은 변수 z에 접근하려고 시도하면 NameError가 발생한다. 이는 파이썬이 초기화되지 않은 변수를 사용할 수 없도록 제한함을 보여준다.

④ 자료형 변환(type casting)의 의미 및 예제

코드	출력 결과	설명
c = '76.3'	없음	c는 문자열(str) 자료형으로 인식된다.
d = float(c)	없음	d는 실수형 자료형이 된다.
print(c)	76.3	문자열 c를 출력한다.
print(d)	76.3	실수형으로 변환된 d를 출력한다.

<code>print(c + d)</code>	오류 발생: <code>TypeError</code>	문자열과 숫자는 서로 연산할 수 없다.
---------------------------	-------------------------------	-----------------------

아래는 자료형 변환의 예제 코드, 실행 결과, 설명을 표로 정리한 내용이다:

코드	출력 결과	설명
<code>x = int(3.9)</code> <code>print(x)</code>	3	float 값을 int로 변환하여 소수점을 버리고 정수 부분만 저장한다.
<code>y = float(5)</code> <code>print(y)</code>	5.0	int 값을 float로 변환하여 소수점 아래에 .0을 추가한 실수형으로 저장한다.
<code>z = str(42)</code> <code>print(z)</code>	42	결과는 문자열로, 숫자가 아닌 텍스트 데이터로 처리된다.
<code>a = "3.14"</code> <code>b = float(a)</code> <code>print(b)</code>	3.14	문자열로 저장된 숫자 값을 float로 변환한다. 이 경우, 문자열이 숫자 형식이어야 변환이 가능하다.
<code>c = "10"</code> <code>d = int(c)</code> <code>print(d)</code>	오류 발생: <code>TypeError</code>	문자열 "10"을 정수형으로 변환하여 숫자 연산이 가능하게 만든다.
<code>e = 1 + True</code> <code>print(e)</code>	2	True는 숫자 1로 간주되어 정수 연산이 수행된다. 파이썬에서 True는 1, False는 0.

(1.2.2) 파이썬 예약어 (Keywords)

(1.3) 배열(Array)과 행렬(Matrix)

(1.3.1) 리스트(List)를 사용한 배열

① 리스트(list) 형식 및 특징

코드	출력 결과	설명
<code>array = [1, 2, 3, 4]</code> <code>print(array)</code>	[1, 2, 3, 4]	리스트를 사용하여 1차원 배열처럼 데이터를 저장한다.
<code>array_2d = [[1, 2], [3, 4]]</code> <code>print(array_2d)</code>	[[1, 2], [3, 4]]	2차원 데이터를 생성하였다. 수학적 연산에는 적합하지 않다.
<code>sum = array[0] + array[1]</code> <code>print(sum)</code>	3	리스트 요소를 인덱싱하여 접근하고, 간단한 덧셈 연산을 수행하였다.

다양한 리스트 선언 예제와 활용이다.

코드	출력 결과	설명
----	-------	----

colors = ['red', 'green'] print(colors)	['red', 'green']	비어 있는 리스트를 선언하였다.
empty_list = [] print(empty_list)	[]	비어 있는 공간.
mixed_list = ['Hello', 1, 2] print(mixed_list)	['Hello', 1, 2]	문자열과 숫자가 혼합된 리스트를 선언하였다.
nest = [['Hello', 'World'], 1, 2] print(nest)	[[' H e l l o ' , 'World'], 1, 2]	리스트 안에 또 다른 리스트를 포함한 중첩 리스트를 선언하였다.

리스트의 데이터 접근과 연산 예이다.

코드	출력 결과	설명
a = [10, 20, 30, 40] print(a[0])	10	리스트의 첫 번째 요소를 가져왔다. 인덱스는 0부터 시작한다.
print(a[-1])	40	음수 인덱스는 뒤에서부터 요소에 접근할 수 있도록 한다.
sum = a[0] + a[1] print(sum)	30	리스트 요소 간의 연산을 수행한다.

중첩 리스트(Nested List)에서 데이터 접근: 이 코드 예제는 리스트를 중첩하여 사용하는 방법과, 중첩된 리스트의 요소를 호출하는 방법을 잘 보여준다.

코드	출력 결과	설명
a = [1, 2, ['a', 'b', ['Life', 'is']]] print(a[2][2][0])	Life	인덱스를 중첩하여 원하는 데이터를 추출할 수 있다.

② 중첩 리스트의 행렬화:

$$A = \begin{bmatrix} v1 \\ v2 \\ v3 \end{bmatrix} = \begin{bmatrix} [1, 2, 3, 4, 5, 6, 7] \\ [1, 2, 3, 4, 5, 6, 7] \\ [1, 2, 3, 4, 5, 6, 7] \end{bmatrix}$$

코드	출력 결과	설명
v1 = [1, 2, 3, 4, 5, 6, 7] v2 = [1, 2, 3, 4, 5, 6, 7] v3 = [1, 2, 3, 4, 5, 6, 7] A = [v1, v2, v3] print(A[0])	[1, 2, 3, 4, 5, 6, 7]	리스트 v1, v2, v3를 생성하고, 이를 포함하는 2차원 리스트 A를 생성하였다. A[0]은 첫 번째 리스트 v1을 반환한다.
print(A[1])	[1, 2, 3, 4, 5, 6, 7]	두 번째 요소는 리스트 v2이다.
print(A[2])	[1, 2, 3, 4, 5, 6, 7]	세 번째 요소는 리스트 v3이다.

(1.3.2) 리스트(List) 슬라이싱(Slicing)

① 슬라이싱의 필요성

② 코드 기초 예제

코드	출력 결과	설명
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] print(a[2:5])	[2, 3, 4]	인덱스 2부터 4까지의 요소를 추출하였다. 끝 인덱스는 포함되지 않는다.
print(a[:4])	[0, 1, 2, 3]	처음부터 인덱스 3까지의 요소를 추출.
print(a[5:])	[5, 6, 7, 8, 9]	인덱스 5부터 리스트 끝까지 추출.
print(a[::2])	[0, 2, 4, 6, 8]	리스트의 처음부터 끝까지, 2씩 건너뛰며 요소를 추출하였다. step 값을 사용한 슬라이싱 예제.
print(a[-3:])	[7, 8, 9]	리스트의 끝에서 3개의 요소를 추출하였다.
print(a[::-1])	[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]	리스트를 거꾸로 뒤집었다. step 값을 -1로 설정하여 역순으로 요소를 추출하였다.

③ 데이터 분할:

슬라이싱을 사용하여 데이터셋을 학습용(train)과 테스트용(test)으로 나눌 수 있다.

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
train = data[:7] # 처음 7개
test = data[7:] # 나머지
print(train) # [1, 2, 3, 4, 5, 6, 7]
print(test) # [8, 9, 10]
```

④ 데이터 샘플링: 간격을 두고 데이터를 추출할 때 유용하다.

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sampled = a[::3] # 3칸 간격으로 추출

print(sampled) # [0, 3, 6, 9]
```

⑤ 리스트 복사: 슬라이싱을 사용하면 리스트의 일부 또는 전체를 복사할 수 있다.

```
original = [1, 2, 3, 4, 5]
copied = original[:] # 리스트 전체 복사
print(copied) # [1, 2, 3, 4, 5]
```

(1.3.3) NumPy와 배열과의 관계

코드	출력 결과	설명
import numpy as np array_np = np.array([1, 2, 3, 4]) print(array_np)	[1, 2, 3, 4]	NumPy 배열을 생성. 리스트와 달리 동일한 자료형만 저장 가능하며, 수학적 연산에 적합.
array_2d = np.array([[1, 2], [3, 4]]) print(array_2d)	[[1, 2], [3, 4]]	2차원 데이터를 NumPy 배열로 표현.
result = array_np + 10 print(result)	3	벡터화 연산을 지원하여 각 요소에 10을 더하는 연산.

(1.3.4) 행렬과 배열과의 관계

코드	출력 결과	설명
A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) C = A @ B print(C)	[[19 22] [43 50]]	두 행렬 간의 행렬 곱을 수행하였다. @ 연산자는 NumPy에서 행렬 곱을 나타낸다.
transpose = A.T print(transpose)	[[1 3] [2 4]]	행렬의 전치를 수행하였다.
D = np.dot(A, B) print(D)	[[19 22] [43 50]]	np.dot() 함수는 행렬 곱 연산을 수행. @ 연산자와 동일한 결과.

(1.4) 튜플(tuple)

(1.4.1) 튜플이란?

(1.4.2) 튜플(tuple) 형식

코드	출력 결과
T = ((10, 20), (30, 40), (50, 60)) print('T=', T) print('T[0] [0]=', T[0] [0]) T4 = (1, 2, ('a', 'b', ('Life', 4))) print(T4[2][2][0])	T= ((10, 20), (30, 40), (50, 60)) T[0] [0]= 10 Life

(1.4.3) 튜플(tuple) 특징

튜플의 요소 값은 변경하거나 삭제할 수 없다.

코드	출력 결과
<pre>T = ((10, 20), (30, 40)) T[0] [0]=55</pre>	<pre>TypeError Traceback (most recent call last) Cell In[3], line 2 1 T = ((10, 20), (30, 40)) ----> 2 T[0] [0]=55 TypeError: 'tuple' object does not support item assignment</pre>

값이 하나만 있는 튜플은 콤마 하나 꼭 붙여야한다.

코드	출력 결과
<pre>T=(55,) print(T[0])</pre>	<pre>55</pre>
코드	출력 결과
<pre>T=(55) print(T[0])</pre>	<pre>TypeError Traceback (most recent call last) Cell In[5], line 2 1 T=(55) ----> 2 print(T[0]) TypeError: 'int' object is not subscriptable</pre>

(1.4.4) 튜플(tuple) 예제

코드	출력 결과
<pre>my_list = [1, 2, 3, 4, 5] # 리스트 생성 print("my_list",my_list) my_tuple = (1, 2, 3, 4, 5) # 튜플 생성 print("my_tuple",my_list) my_list[0] = 6 # [6, 2, 3, 4, 5] # 리스트 값 변경 print("my_list[0]",my_list) my_list.append(6) # [6, 2, 3, 4, 5, 6] # 리스트 값 추가 print("my_list.append(6)",my_list) my_tuple[0] = 6 # 튜플 값 변경 - 오류 발생</pre>	<pre>my_list [1, 2, 3, 4, 5] my_tuple [1, 2, 3, 4, 5] my_list[0] [6, 2, 3, 4, 5] my_list.append(6) [6, 2, 3, 4, 5, 6] TypeError Traceback (most recent call last) Cell In[6], line 18 1 5 print("my_list.append(6)",my_list) 17 # 튜플 값 변경 - 오류 발생</pre>

<pre>print("my_tuple[0]",my_list) my_tuple.append(6) # 튜플 값 추가 - 오류 발생 print("my_tuple.append(6)",my_list)</pre>	<pre>---> 18 my_tuple[0] = 6 19 print("my_tuple[0]",my_list) 21 # 튜플 값 추가 - 오류 발생 TypeError: 'tuple' object does not support item assignment</pre>
--	--

(1.4.5) 튜플(tuple) slicing : ":" 사용

코드	출력 결과
<pre>T = (1, 2, 3, 4, 5, 6, 7) T1 = T[0:3] ;T2 = T[0:-3] T3 = T[:]; T4 = T[5:]; T5 = T[:5] print(f'T : {T}') print(f'T[0:3] : {T1}') print(f'T[0:-3] : {T2}') print(f'T[:] : {T3}') print(f'T[5:] : {T4}') print(f'T[:5] : {T5}')</pre>	<pre>T : (1, 2, 3, 4, 5, 6, 7) T[0:3] : (1, 2, 3) T[0:-3] : (1, 2, 3, 4) T[:] : (1, 2, 3, 4, 5, 6, 7) T[5:] : (6, 7) T[:5] : (1, 2, 3, 4, 5)</pre>

(1.4.6) 튜플(tuple) Extended slicing : "::" 사용

코드	출력 결과
<pre>A = [0, 1, 2, 3, 4, 5, 6] A_copy = A[:] # A[:] - 배열 전체를 복사 # A[1:2:3]: 인덱스 1부터 2까지 3의 간격으로 추출 A_slice_1 = A[1:2:3] A_reverse = A[::-1] # A[::-1] - 배열을 역순으로 추출 print(f'A_copy: {A_copy}') print(f'A_slice_1: {A_slice_1}') print(f'A_reverse: {A_reverse}')</pre>	<pre>A_copy: [0, 1, 2, 3, 4, 5, 6] A_slice_1: [1] A_reverse: [6, 5, 4, 3, 2, 1, 0]</pre>

(1.5) 사전 (Dictionary)

(1.5.1) 딕셔너리(dictionary)

(1.5.2) 딕셔너리(dictionary) 예제

코드	출력 결과
<pre># 딕셔너리 생성 student = { "name": "Alice", "age": 25, "major": "Engineering" } # 값 접근 print(student["name"]) # 출력: Alice # 값 추가 및 수정 student["age"] = 26 student["grade"] = "A" # 키 삭제 del student["major"] # 딕셔너리 출력 print(student)</pre>	<pre>Alice {'name': 'Alice', 'age': 26, 'grade': 'A'}</pre>

(1.6) 수학 연산자 및 NumPy 비교

(1.6.1) 파이썬의 기본 수학 연산자와 NumPy 연산 비교

(1.6.2) Python 과 NumPy의 기본 연산

Python의 기본 연산자는 스칼라 값(단일 숫자)을 처리하는 데 매우 직관적이다.

코드	출력 결과
<pre>a = 5 ; b = 3 # Python 기본 연산 print(a + b) # 출력: 8</pre>	8

NumPy에서도 단일 값 연산이 가능하지만, 주로 배열과 함께 사용된다.

코드	출력 결과
<pre>import numpy as np # NumPy 기본 연산 a = np.array([5]); b = np.array([3])</pre>	[8]

<code>print(a + b) # 출력: [8]</code>

(1.6.3) Python 과 NumPy의 배열 연산

Python의 기본 연산자는 리스트를 사용할 때 요소별 연산을 자동으로 수행하지 않는다. 대신, 반복문이나 리스트 컴프리헨션을 사용해야 한다.

코드	출력 결과
<pre># Python 기본 연산 (리스트) a = [1, 2, 3]; b = [4, 5, 6] c = [a[i] + b[i] for i in range(len(a))] print(c) # 출력: [5, 7, 9]</pre>	[5, 7, 9]

NumPy는 벡터화 연산을 지원하여 반복문 없이도 배열의 요소별 연산을 간단히 수행할 수 있다.

코드	출력 결과
<pre># NumPy 배열 연산 import numpy as np a = np.array([1, 2, 3]); b = np.array([4, 5, 6]) c = a + b ; print(c) # 출력: [5 7 9]</pre>	[5, 7, 9]

Python 과 NumPy의 배열 계산 성능 비교: NumPy는 배열 연산에 대해 Python 기본 연산보다 훨씬 빠르다.

코드	출력 결과
<pre># 성능 비교 import numpy as np ; import time # Python 리스트 연산 a = list(range(1_000_000)) b = list(range(1_000_000)) start = time.time() c = [a[i] + b[i] for i in range(len(a))] print("리스트 연산:\n", time.time() - start) # NumPy 배열 연산 a_np = np.arange(1_000_000) b_np = np.arange(1_000_000) start = time.time() ; c_np = a_np + b_np print("NumPy 연산:\n", time.time() - start)</pre>	리스트 연산: 0.11910867691040039 NumPy 연산: 0.0010001659393310547

Python 리스트의 연산에서는 더하기(+)나 반복하기 (*) 같은 연산자가 적용된다.

코드	출력 결과
<pre>v1 = [1, 2, 3] v1 * 3</pre>	[1, 2, 3, 1, 2, 3, 1, 2, 3]

numpy를 사용한 벡터(vector) +, - 연산:

코드	출력 결과
<pre>import numpy as np v1 = np.array ([1,2,3]) v2= np.array ([1,2,3]) print (v1 + v2)</pre>	[2 4 6]

numpy를 사용한 벡터(vector) 스칼라 곱하기 연산:

코드	출력 결과
<pre>import numpy as np v1 = np.array ([1,2,3]) v2= np.array ([1,2,3]) print (3*v1 + 2*v2)</pre>	[5 10 15]

코드	출력 결과
<pre>import numpy as np v1 = np.array ([1,2,3]) v2= np.array ([1,2,3]) print (sum(v1*v2)) print (np.dot(v1,v2))</pre>	14 14

(1.7) 반복 계산

(1.7.1) 반복계산이란?

(1.7.2) for 문을 이용한 반복 계산

① 특징:

② for 구문:

```
for variable in iterable:
    구문
```

③ 범위(range)를 사용:

주석으로 설명한 소스 코드	출력 결과
<pre># 첫 번째 for 루프: range() 함수를 사용해 1부터 3까지 반복 # range(1, 4)는 1, 2, 3을 생성 (4는 포함되지 않음) for val in range(1, 4): print(val) # 현재 반복 값(val)을 출력 # 두 번째 for 루프: 리스트 [1, 2, 3, 4]를 순회 # 리스트의 각 요소를 순서대로 변수 i에 할당 for i in [1, 2, 3, 4]: print(i) # 현재 리스트 요소(i)를 출력</pre>	<pre>1 2 3 1 2 3 4</pre>

④ 반복문을 이용한 합 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 1부터 10까지의 합 계산 total = 0 for i in range(1, 11): total += i print("1부터 10까지의 합:", total)</pre>	<pre>1부터 10까지의 합: 55</pre>

⑤ 리스트를 이용한 반복 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 리스트 요소의 제곱 계산 numbers = [1, 2, 3, 4, 5] squared = [] for num in numbers: squared.append(num ** 2) print("리스트의 제곱:", squared)</pre>	<pre>리스트의 제곱: [1, 4, 9, 16, 25]</pre>

⑥ 예제: 0부터 n까지의 홀수 합

주석으로 설명한 소스 코드	출력 결과
<pre> print("0부터 n까지의 홀수 합을 계산.") print("n을 입력하세요.") # 입력받은 값을 정수로 변환하여 변수 n에 저장 n = int(input()) # 합계를 저장할 변수 초기화 val = 0 # 0부터 n까지 반복 # i는 0부터 n까지 순서대로 값을 가짐 # i가 홀수?(2로 나눈 나머지가 1이면 홀수) for i in range(0, n + 1): if i % 2 == 1: val = val + i #홀수라면 val에 i를 더함 print("n까지의 홀수 합 : ", val) </pre>	0부터 n까지의 홀수 합을 계산. n을 입력하세요. 100 n까지의 홀수 합 : 2500

⑦ 예제: 0부터 n까지의 홀수 합

0에서 n까지 짝수 합 계산하는 함수를 만들고 n=100인 경우 그 결과는 어떻게 되는가?

주석으로 설명한 소스 코드	출력 결과
<pre> # 사용자에게 알림 메시지 출력 print("0부터 n까지의 짝수 합을 계산.") # 사용자로부터 n 값을 입력받음 print("n을 입력하세요.") # 입력받은 값을 정수로 변환하여 변수 n에 저장 n = int(input()) # 합계를 저장할 변수 초기화 val = 0 # 0부터 n까지 반복 # i는 0부터 n까지 순서대로 값을 가짐 # i가 홀수?(2로 나눈 나머지가 1이면 홀수) </pre>	0부터 n까지의 짝수 합을 계산. n을 입력하세요. 100 n까지의 짝수 합 : 2550

```

for i in range(0, n + 1):
    if i % 2 == 0:
        val = val + i # 짝수라면 val에 i를 더함

# 계산된 짝수 합을 출력
print("n까지의 짝수 합 : ", val)

```

⑧ 예제: 함수값 이용

주석으로 설명한 소스 코드	출력 결과
<pre> # 함수 정의: 입력값 x를 제공하여 반환하는 함수 f(x) def f(x): return x**2 # x를 제공한 값을 반환 # 반복문: range(0, 4)로 0부터 3까지 반복 for x in range(0, 4): # x는 0, 1, 2, 3의 값을 순서대로 가짐 print(f(x)) # 함수 f(x)의 결과(제곱값)를 출력 </pre>	<pre> 0 1 4 9 </pre>

(1.7.3) while 문을 이용한 반복 계산

주석으로 설명한 소스 코드	출력 결과
<pre> # 1부터 n까지의 합을 계산 # 프로그램 시작 메시지 출력 print("1부터 n까지의 합을 계산.") # 사용자로부터 n 값을 입력받음 # 입력값을 정수로 변환하여 n에 저장 n = int(input("n 값을 입력: ")) # 반복에 필요한 초기 변수 설정 i = 1 # 반복 변수, 1부터 시작 total = 0 # 합계를 저장할 변수 </pre>	<pre> 1부터 n까지의 합을 계산. n 값을 입력: 100 1부터 100까지의 합은 5050. </pre>

<pre># while 문을 이용한 반복 # i가 n 이하일 때 루프를 실행 # total 변수에 i 값을 더함 (누적합 계산) # 반복 변수 i를 1 증가시킴 (루프 탈출 조건 형성) while i <= n: total += i i += 1 # 최종 결과 출력 print(f"1부터 {n}까지의 합은 {total}.")</pre>	
--	--

(1.8) 조건 계산

(1.8.1) if ~ else 구문

(1.8.2) 다중 조건문 (if ~ else ~ else 구문)

(1.8.3) if 구조 예문

① 예제 코드: 숫자가 짝수인지 홀수인지 판별하기

주석으로 설명한 소스 코드	출력 결과
<pre># 사용자로부터 숫자를 입력받음 # 입력값을 정수로 변환 num = int(input("숫자를 입력하세요: ")) # if ~ else 구문을 이용해 짝수와 홀수 판별 # 숫자를 2로 나눈 나머지가 0이면 짝수 if num % 2 == 0: print(f"{num}은 짝수입니다.") else: # 나머지가 0이 아니면 홀수 print(f"{num}은 홀수입니다.")</pre>	숫자를 입력하세요: 5 5은 홀수입니다.

② 리스트와 조건문을 이용한 다중 조건 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 학생 점수와 등급 계산 scores = [95, 82, 67, 58, 89] grades = [] for score in scores: if score >= 90: grades.append('A') elif score >= 80: grades.append('B') elif score >= 70: grades.append('C') else: grades.append('D') print("점수:", scores) print("등급:", grades)</pre>	점수: [95, 82, 67, 58, 89] 등급: ['A', 'B', 'D', 'D', 'B']

③ 조건 표현식을 이용한 간단한 조건 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 숫자가 양수, 음수 또는 0인지 판별 number = -5 result = "양수" if number > 0 else "음수" if number < 0 else "0" print(f"숫자: {number}, 결과: {result}")</pre>	숫자: -5, 결과: 음수

(1.8.4) 비교연산자

① 예제 코드: 비교 연산자 사용

주석으로 설명한 소스 코드	출력 결과
<pre># 두 숫자를 입력받아 비교 a = int(input("첫 번째 숫자를 입력?: ")) b = int(input("두 번째 숫자를 입력?: ")) # a와 b가 같은지 비교 print(f"{a} == {b}: {a == b}")</pre>	첫 번째 숫자를 입력?: 1 두 번째 숫자를 입력?: 5 1 == 5: False 1 != 5: True 1 > 5: False 1 >= 5: False

<pre># a와 b가 다른지 비교 print(f"{a} != {b}: {a != b}") # a가 b보다 큰지 비교 print(f"{a} > {b}: {a > b}") # a가 b보다 크거나 같은지 비교 print(f"{a} >= {b}: {a >= b}") # a가 b보다 작은지 비교 print(f"{a} < {b}: {a < b}") # a가 b보다 작거나 같은지 비교 print(f"{a} <= {b}: {a <= b}")</pre>	<pre>1 < 5: True 1 <= 5: True</pre>
---	---

(1.9) 입출력

(1.9.1) 키보드로 입력 및 화면으로 출력

주석으로 설명한 소스 코드	출력 결과
<pre># 키보드로 입력받아 계산 후 출력 name = input("이름을 입력하세요: ") age = int(input("나이를 입력하세요: ")) print(f"안녕하세요, {name}님. \n당신의 나이는 {age}살입니다.")</pre>	<pre>이름을 입력하세요: 김시조 나이를 입력하세요: 22 안녕하세요, 김시조님. 당신의 나이는 22살입니다.</pre>

(1.9.2) 파일로 입력 및 파일로 출력

① 파일 쓰기

주석으로 설명한 소스 코드	출력 결과
<pre># 파일에 쓰기 with open("output.txt", "w") as file: file.write("안녕하세요.\n") file.write("Python 파일 입출력 예제입니다.\n")</pre>	<pre>안녕하세요. Python 파일 입출력 예제입니다.</pre>

② 파일 읽기

주석으로 설명한 소스 코드	출력 결과
<pre># 파일 읽기 with open("output.txt", "r") as file: content = file.read() print("파일 내용:\n", content)</pre>	파일 내용: 안녕하세요. Python 파일 입출력 예제입니다.

③ 하나로 합친 완전한 Python 소스 코드

주석으로 설명한 소스 코드	출력 결과
<pre># 파일로 입력 및 파일로 출력 # 파일에 데이터를 쓰고, 저장된 데이터를 읽어 출력 # 하는 코드 # 파일 쓰기 with open("output.txt", "w") as file: file.write("안녕하세요.\n") file.write("Python 파일 입출력 예제입니다.\n") print("데이터가 파일에 저장되었습니다: output.txt") # 파일 읽기 with open("output.txt", "r") as file: content = file.read() print("\n파일에서 읽어온 내용:") print(content)</pre>	output.txt 파일 내용: 안녕하세요. Python 파일 입출력 예제입니다. 터미널 출력: 데이터가 파일에 저장되었습니다: output.txt 파일에서 읽어온 내용: 안녕하세요. Python 파일 입출력 예제입니다.

(1.9.3) $m \times n$ 행렬 [A] 입출력 하기

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # m x n 행렬 생성 A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) print("생성된 행렬 A:") print(A) # 행렬을 파일에 저장</pre>	화면 출력:: 생성된 행렬 A: [[1 2 3] [4 5 6] [7 8 9]] 행렬 A가 'matrix.txt' 파일에 저장.

<pre> np.savetxt("matrix.txt", A, fmt="%d", delimiter=",") print("\n행렬 A가 'matrix.txt' 파일에 저장.") # 저장된 파일에서 행렬 불러오기 loaded_A = np.loadtxt("matrix.txt", delimiter=",", dtype=int) print("\n파일에서 불러온 행렬 A:") print(loaded_A) </pre>	파일에서 불러온 행렬 A: <pre> [[1 2 3] [4 5 6] [7 8 9]] </pre> matrix.txt 파일 내용:: <pre> 1,2,3 4,5,6 7,8,9 </pre>
---	---

(1.10) 함수(Function)

(1.10.1) 내장함수와 사용자 정의 함수

(1.10.2) 파이썬과 NumPy의 내장함수 종류

① Python List 내장함수

② append, remove, insert 내장 함수 예제

주석으로 설명한 소스 코드	출력 결과
<pre> v=[0 ,1 ,2 ,3] #변수v에 리스트를 저장 v.append(111) #v의 리스트에 111을 추가 print (v) #v의 리스트를 출력. v.extend([22 ,33 ,'aaa']) #새로운 리스트를 추가. print (v)#v의 리스트를 출력. v.remove(22) #v의 리스트에 22을 제거. v.remove(111) #v의 리스트에 111을 제거. v.remove(33) #v의 리스트에 33을 제거. v.remove('aaa') #v의 리스트에 33을 제거. v.remove(1) #v의 리스트에 1을 제거. print (v) #v의 리스트를 출력. v.insert(1 ,'bbb') #1번에 bbb를 추가. print (v) #v의 리스트를 출력. </pre>	<pre> [0, 1, 2, 3, 111] [0, 1, 2, 3, 111, 22, 33, 'aaa'] [0, 2, 3] [0, 'bbb', 2, 3] </pre>

(1.10.3) 사용자 정의 함수 선언과 특징

(1.10.4) 사용자 정의 함수 예제

① 함수 예제: 수학적 함수와 도함수

주석으로 설명한 소스 코드	출력 결과
<pre># f(x) 함수 정의 def f(x): return 9 - x * (x - 10) # f'(x) 도함수 정의 def fprime(x): return -2 * x + 10 # 함수 출력 예제 print("f(3) =", f(3)) # f(x)의 결과 print("f'(3) =", fprime(3)) # f'(x)의 결과</pre>	<pre>f(3) = 0 f'(3) = 4</pre>

② 함수 예제: 팩토리얼 계산

주석으로 설명한 소스 코드	출력 결과
<pre># 사용자 정의 함수: 팩토리얼 계산 def factorial(n): if n == 0 or n == 1: return 1 else: return n * factorial(n - 1) # 사용자 정의 함수 호출 print("5! =", factorial(5)) print("7! =", factorial(7))</pre>	<pre>5! = 120 7! = 5040</pre>

③ 함수 예제: 함수 F(x)를 정의하고 계산

주석으로 설명한 소스 코드	출력 결과
<pre>def f(x): return 9-x*(x-10)</pre>	<pre>-9182</pre>

<pre>x = 101 print(f(x))</pre>	
--------------------------------	--

④ 함수 예제: 사용자 정의 함수와 조건문 결합

even_or_odd() 함수는 조건문을 사용하여 입력된 숫자가 짝수인지 홀수인지 판별한다.

주석으로 설명한 소스 코드	출력 결과
<pre># 숫자가 짝수인지 홀수인지 판별하는 함수 def even_or_odd(number): if number % 2 == 0: return f"{number}는 짝수입니다." else: return f"{number}는 홀수입니다." # 함수 호출 print(even_or_odd(10)) print(even_or_odd(7))</pre>	10는 짝수입니다. 7는 홀수입니다.

(1.11) Numpy를 이용한 벡터, 행렬

(1.11.1) 내적 (Dot Product)과 외적 (Cross Product)

① 벡터의 내적

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 내적 계산 dot_product = np.dot(a, b) print("벡터 a와 b의 내적:", dot_product)</pre>	벡터 a와 b의 내적: 32

② 벡터의 외적

주석으로 설명한 소스 코드	출력 결과
----------------	-------

<pre># 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 외적 계산 cross_product = np.cross(a, b) print("벡터 a와 b의 외적:", cross_product)</pre>	벡터 a와 b의 외적: [-3 6 -3]
---	-------------------------------

(1.11.2) 두 벡터간의 거리 구하기

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 두 벡터 정의 a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) # 유클리드 거리 계산 distance = np.linalg.norm(a - b) print("벡터 a:", a) print("벡터 b:", b) print("두 벡터 간의 거리:", distance)</pre>	벡터 a: [1 2 3] 벡터 b: [4 5 6] 두 벡터 간의 거리: 5.196152422706632

(1.11.3) 두 행렬의 곱(matrix multiplication)

① NumPy를 이용한 행렬 곱

㉠ '*' 연산자로 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre># numpy 라이브러리 임포트 import numpy as np # 행렬 선언 A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]])</pre>	<pre>C = [[19 22] [43 50]] A1 = [[1 2] [3 4]] B1 = [[5 6] [7 8]] C1 = [[19 22]</pre>

<pre># 행렬 곱 수행 (np.matmul 사용) C = np.matmul(A, B) # NumPy의 행렬 곱 함수 print('C =', C) # 행렬을 NumPy의 matrix 객체로 변환 A1 = np.asmatrix(A) # A를 matrix 형식으로 변환 print('A1 =', A1) B1 = np.asmatrix(B) # B를 matrix 형식으로 변환 print('B1 =', B1) # matrix 객체에서 '*' 연산자로 행렬 곱 수행 # matrix 객체에서는 '*' 연산자가 행렬 곱으로 동작 C1 = A1 * B1 print('C1 =', C1)</pre>	[43 50]
---	---------

㉠ np.dot(A, B) 또는 @ 연산자로 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre>import numpy as np # 행렬 선언 A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) # 행렬 곱 계산 C = A @ B # 또는 np.dot(A, B) print("행렬 곱 결과:\n", C)</pre>	행렬 곱 결과: <pre>[[19 22] [43 50]]</pre>

㉡ numpy 를 사용하지 않은 행렬 곱 (AB)

주석으로 설명한 소스 코드	출력 결과
<pre># 2차원 배열을 생성하는 함수 정의 def A_0(rows, cols): A = [] # 빈 리스트 초기화 while len(A) < rows: # 행 개수가 rows보다 작으면 반복 A.append([]) # 새로운 행 추가 # 열 개수가 cols보다 작으면 반복</pre>	<pre>C = [[19, 22], [43, 50]]</pre>

```

        while len(A[-1]) < cols:
            A[-1].append(0.0) # 행에 0.0 추가
        return A # 생성된 2차원 배열 반환

# 두 행렬의 곱을 계산하는 함수 정의
def AmultiB(A, B):
    rowsA = len(A) # A 행렬의 행 개수
    colsA = len(A[0]) # A 행렬의 열 개수
    rowsB = len(B) # B 행렬의 행 개수
    colsB = len(B[0]) # B 행렬의 열 개수

    # 행렬 곱이 가능한지 확인 (A의 열 개수 == B의 행 개수)
    if colsA != rowsB:
        # 조건이 맞지 않으면 예외 발생
        raise ArithmeticError('colsA != rowsB')

    # 결과 행렬 초기화

    # 결과 행렬의 크기는 rowsA x colsB
    AmultiB = A_0(rowsA, colsB)

    # 행렬 곱 계산
    for i in range(rowsA): # A의 행 반복
        for j in range(colsB): # B의 열 반복
            sum = 0 # 곱셈과 덧셈의 결과를 저장할 변수

            # A의 열과 B의 행 반복
            for ii in range(colsA):
                # A의 i행과 B의 j열의 곱을 누적
                sum = sum + A[i][ii] * B[ii][j]

            # 결과 행렬의 (i, j) 위치에 저장
            AmultiB[i][j] = sum
    return AmultiB # 결과 행렬 반환

# 행렬 선언
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# A와 B의 곱 계산

```

<pre>C = AmultiB(A, B) # 사용자 정의 함수 AmultiB 사용 print('C =', C)</pre>	
---	--

(1.12) 객체지향 프로그래밍(OOP)

(1.12.1) 클래스 개요

(1.12.2) 클래스 선언

(1.12.3) 속성(변수)

(1.12.4) 클래스 메서드(함수)

① 클래스 메서드의 주요 특징

② 예제 코드

```
class Example:
    class_variable = 0 # 클래스 변수

    def __init__(self, value):
        self.instance_variable = value # 인스턴스 변수

    @classmethod
    def increment_class_variable(cls, amount):
        cls.class_variable += amount
        print(f"Class variable updated to {cls.class_variable}")

    @classmethod
    def create_instance(cls, value):
        return cls(value)

# 클래스 메서드 호출
Example.increment_class_variable(10) # Class variable updated to 10

# 새로운 객체 생성
```

```
instance = Example.create_instance(20)
print(instance.instance_variable) # 20
```

③ 예제 코드 실행

```
Class variable updated to 10
20
```

(1.12.5) 객체지향 프로그래밍(OOP) 예제 1

주석으로 설명한 소스 코드	출력 결과
<pre># 클래스 정의 class Animal: def __init__(self, name): # 생성자 self.name = name def speak(self): # 메서드 print(f"{self.name} is making a sound.") # 클래스 상속 class Dog(Animal): def speak(self): # 메서드 오버라이딩 (다형성) print(f"{self.name} says Woof!") # 객체 생성 및 사용 dog = Dog("Buddy") dog.speak() # 출력: Buddy says Woof!</pre>	Buddy says Woof!

(1.12.6) 객체지향 프로그래밍(OOP) 예제 2

```
# 재설정된 환경에서 코드를 다시 작성 및 실행
import numpy as np
import matplotlib.pyplot as plt
```

```

# 함수와 미분을 OOP로 구현
class Function:
    def __init__(self, func, d_func):
        self.func = func          # 함수
        self.d_func = d_func      # 함수의 미분

    def evaluate(self, x):
        """함수 값을 계산"""
        return self.func(x)

    def derivative(self, x):
        """미분 값을 계산"""
        return self.d_func(x)

    def visualize(self, x_range):
        """함수와 미분의 그래프를 가시화"""
        x = np.linspace(x_range[0], x_range[1], 500)
        y = self.evaluate(x)
        y_prime = self.derivative(x)

        plt.figure(figsize=(10, 6))
        plt.plot(x, y, label='f(x)', linewidth=2)
        plt.plot(x, y_prime, label="f'(x)", linestyle='--', linewidth=2)
        plt.axhline(0, color='black', linestyle='-', linewidth=0.8)
        plt.title('Function and its Derivative')
        plt.xlabel('x')
        plt.ylabel('y')
        plt.legend()
        plt.grid(True)
        plt.show()

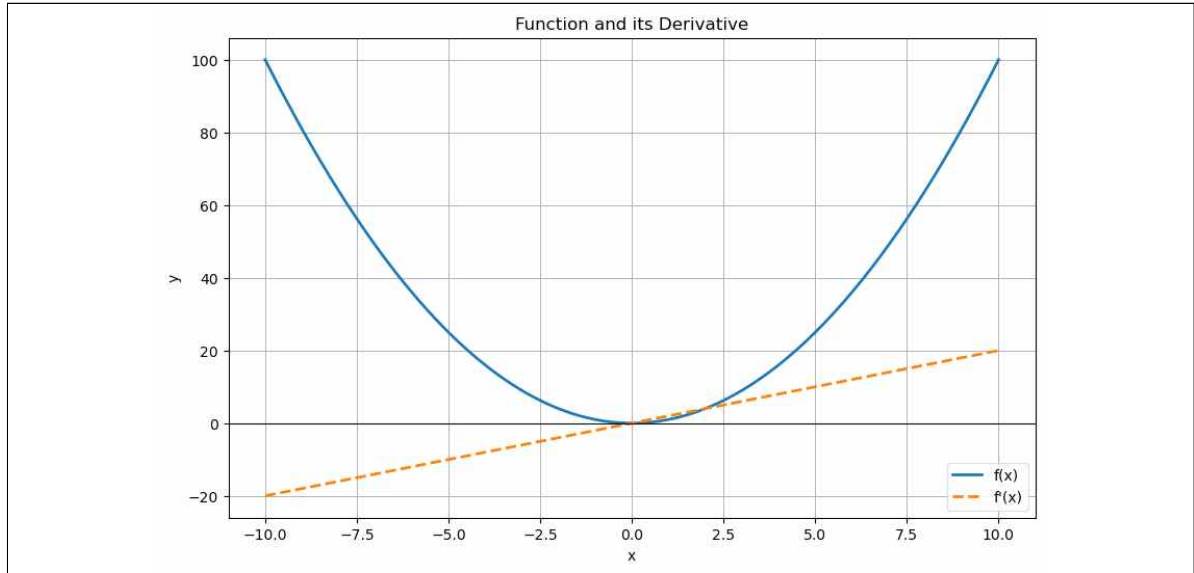
# 함수와 미분 정의
f = lambda x: x**2 # 함수 f(x) = x^2
df = lambda x: 2*x # 미분 f'(x) = 2x

# 객체 생성
quadratic_function = Function(f, df)

# 그래프 가시화
quadratic_function.visualize((-10, 10))

```

실행경과는 아래와 같다.



(1.13) matplotlib를 이용한 그래프 시각화

(1.13.1) 그래프 그리기 위한 주요 내장 함수

(1.13.2) 2차원 그래프

① 곡선 그래프 그리는 코드

```
import numpy as np
import matplotlib.pyplot as plt

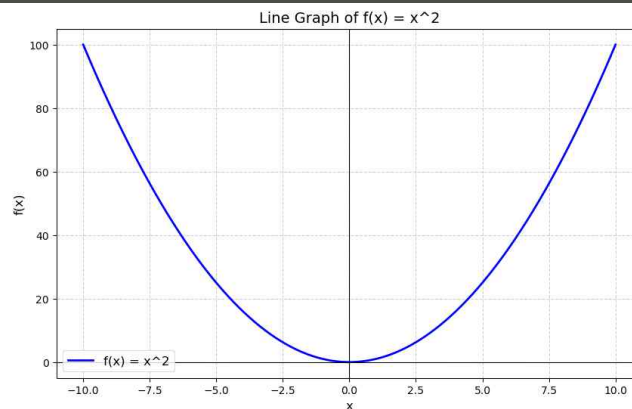
# 데이터 생성
x = np.linspace(-10, 10, 500) # x 범위를 -10에서 10까지 500개의 점 생성
y = x**2 # y 값은 f(x) = x^2로 계산

# 그래프 그리기
plt.figure(figsize=(10, 6)) # 그래프 크기 설정
plt.plot(x, y, label='f(x) = x^2', color='blue', linewidth=2) # 함수 그래프
```

```
# 주석 추가
plt.title('Line Graph of f(x) = x^2', fontsize=14) # 그래프 제목
plt.xlabel('x', fontsize=12) # x축 라벨
plt.ylabel('f(x)', fontsize=12) # y축 라벨
plt.axhline(0, color='black', linestyle='-', linewidth=0.8) # x축 표시
plt.axvline(0, color='black', linestyle='-', linewidth=0.8) # y축 표시
plt.grid(True, linestyle='--', alpha=0.7) # 격자선 추가
plt.legend(fontsize=12) # 범례 추가

# 그래프 표시
plt.show()
```

② 곡선 그래프 그리는 코드 실행



(1.13.3) 2차원: 산점도

① 2차원: 산점도 그리는 코드

```
import numpy as np
import matplotlib.pyplot as plt

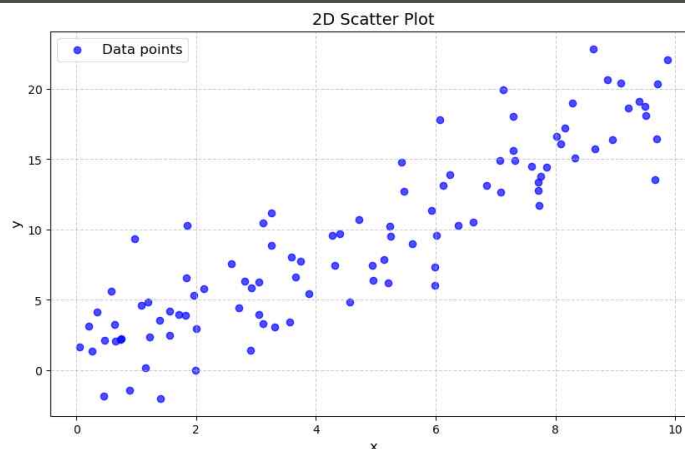
# 데이터 생성
np.random.seed(42) # 랜덤 시드 고정
x = np.random.rand(100) * 10 # 0~10 사이의 100개의 난수 생성
y = 2 * x + np.random.normal(0, 3, 100) # y = 2x + 랜덤 노이즈 추가
```

```
# 산점도 그래프 그리기
plt.figure(figsize=(10, 6)) # 그래프 크기 설정
plt.scatter(x, y, color='blue', alpha=0.7, label='Data points') # 산점도 그리기

# 주석 추가
plt.title('2D Scatter Plot', fontsize=14) # 그래프 제목
plt.xlabel('x', fontsize=12) # x축 라벨
plt.ylabel('y', fontsize=12) # y축 라벨
plt.grid(True, linestyle='--', alpha=0.7) # 격자선 추가
plt.legend(fontsize=12) # 범례 추가

# 그래프 표시
plt.show()
```

② 2차원: 산점도 그리는 코드 실행



(1.13.4) 3차원 그래프

① 3차원 곡면 그래프 코드

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

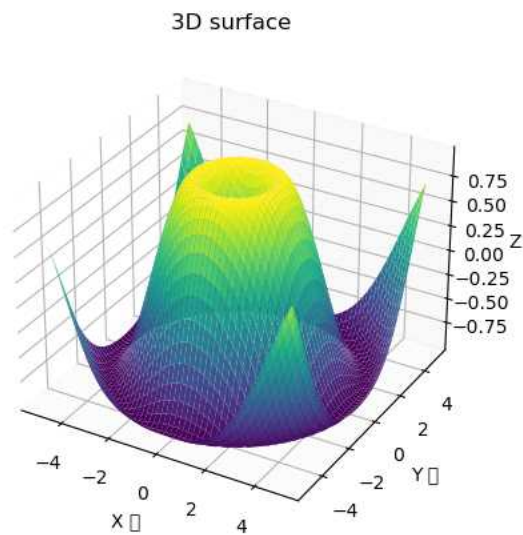
# 데이터 준비
x = np.linspace(-5, 5, 50)
```

```
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# 3D 그래프 그리기
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="viridis")

ax.set_title("3D surface")
ax.set_xlabel("X 축")
ax.set_ylabel("Y 축")
ax.set_zlabel("Z 축")
plt.show()
```

② 3차원 곡면 그래프 코드 실행



2 장 비선형 방정식 근 구하기

(2.1) 비선형 방정식 근 구하기 개요

(2.2) 점진탐색법(Incremental Search Method)

(2.2.1) 알고리즘의 원리

(2.2.2) 알고리즘

(2.2.3) 파이썬 소스 코드 함수 구현

```
def incremental_search(f, x_start, x_end, step_size):
    """
    점진탐색법 구현 함수

    Parameters:
        f: 탐색할 함수
        x_start: 탐색 시작점
        x_end: 탐색 끝점
        step_size: 구간 분할 간격

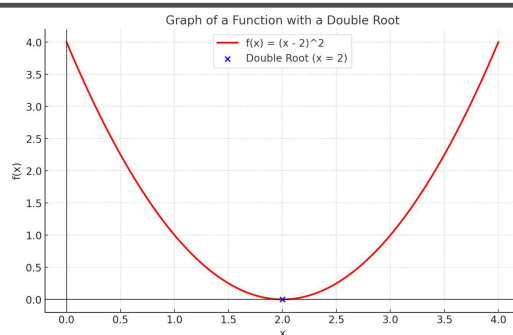
    Returns:
        근이 포함된 구간 또는 None
    """
    x = x_start
    while x < x_end:
        if f(x) * f(x + step_size) < 0: # 부호 변화 점검
            return (x, x + step_size)
        x += step_size
    return None # 근이 발견되지 않음
```

(2.2.4) 예제

주어진 함수의 해 $f(x) = x^3 - 6x^2 + 11x - 6 = 0$ 의 해를
[0,4][0,4] 구간에서 찾는다.

주석으로 설명한 소스 코드	출력 결과
<pre># 함수 정의 f = lambda x: x**3 - 6*x**2 + 11*x - 6 # f(x) = (x-1)(x-2)(x-3) # 점진탐색법 실행 x_range = incremental_search(f, 0, 4, 0.1) # 결과 출력 if x_range: print("근이 포함된 구간:", x_range) else: print("근이 존재하지 않습니다.")</pre>	근이 포함된 구간: (2.9, 3.0)

(2.2.5) 중근을 갖는 경우



① 해결 방법

② Python 코드 구현 (기울기 조건 추가):

다음 코드는 함수 값과 미분 값(기울기)을 기반으로 중근을 탐지하는 방법을 보여준다.

주석으로 설명한 소스 코드	출력 결과
import numpy as np	중근에 근접


```
def incremental_search_with_derivative(f, df, x_start,
x_end, step_size, tol=1e-6):
    """
    점진탐색법에 기울기 조건 추가

    Parameters:
        f: 함수
        df: 함수의 도함수
        x_start: 시작점
        x_end: 끝점
        step_size: 탐색 간격
        tol: 기울기 허용 오차

    Returns:
        중근이 존재할 수 있는 구간 또는 None
    """
    x = x_start
    while x < x_end:
        if abs(f(x)) < tol or abs(df(x)) < tol: # 함수 값 또는
            기울기가 0에 가까움
                return x
        x += step_size
    return None

# 함수와 도함수 정의
f = lambda x: (x - 1)**2 # 중근을 가지는 함수
df = lambda x: 2 * (x - 1) # 도함수

# 탐색 실행
root = incremental_search_with_derivative(f, df, 0, 2, 0.1)
print("중근에 근접한 점:", root)
```

한 점: 1.0

(2.3) 이분법(Bisection Method)

(2.3.1) 알고리즘의 원리

(2.3.2) 이분법 알고리즘

① Step 1: 초기 조건 설정

- ㉞ 범위 $[a, b]$ 에서 $f(a)f(b) < 0$ 를 만족하는지 확인한다.
- ㉟ 만족하지 않으면 종료한다.
- ② Step 2: 중간점 계산
 - ㉞ 중간점 c 를 계산한다: $c = (a + b) / 2$
- ③ Step 3: 근의 위치 판별
 - ㉞ $f(c)$ 를 계산 후 $f(a)f(c) < 0$ 이면, 구간 $[a, c]$ 에 근이 존재한다. 이경우 $b=c$ 로 설정한다.
 - ㉟ $f(c)$ 를 계산 후 $f(c)f(b) < 0$ 이면, 구간 $[c, b]$ 에 근이 존재한다. 이경우 $a=c$ 로 설정한다.
- ④ Step 4: 종료 조건 검사
 - ㉞ 구간 길이 $|b - a|$ 가 허용 오차 ϵ 보다 작아지면 알고리즘을 종료한다.
 - ㉟ 이때 c 가 근의 근사값으로 간주된다.
- ⑤ Step 5: 반복
 - ㉞ Step 2부터 Step 4까지 과정을 반복한다.
 - ㉟ 원하는 정확도 내에서 근을 계산한다.
- ⑥ 파이썬 소스 코드 함수 구현

```
def bisection(f, a, b, tol=1e-6, max_iter=100):
    """
```

이분법(Bisection Method) 구현 함수 (반복 과정 출력 포함)

Parameters:

f: 근을 구할 함수
 a: 구간의 시작점
 b: 구간의 끝점
 tol: 허용 오차
 max_iter: 최대 반복 횟수

Returns:

근의 근사값

```
"""
```

```
if f(a) * f(b) >= 0:
```

```
    raise ValueError("구간 [a, b]에 근이 포함되지 않았습니다.")
```

```
print(f"초기 구간: [a, b] = [{a}, {b}])")
```

```
iter_count = 0
```

```
while abs(b - a) > tol and iter_count < max_iter:
```

```
    c = (a + b) / 2 # 중간값 계산
```

```

    print(f"반복 {iter_count + 1}: [a, b] = [{a:.6f}, {b:.6f}], c = {c:.6f}, f(c) = {f(c):.6e}")

    # 근을 찾은 경우 또는 허용 오차 내 도달
    if f(c) == 0 or abs(b - a) < tol:
        print(f"최종 구간: [a, b] = [{a:.6f}, {b:.6f}]")
        return c

    # 구간 축소
    if f(a) * f(c) < 0:
        b = c
    else:
        a = c

    iter_count += 1

print(f"최종 구간: [a, b] = [{a:.6f}, {b:.6f}]")
return (a + b) / 2

# 2차 다항식 예제: f(x) = x^2 - 4
def quadratic_function(x):
    return x**2 - 4

# 초기 구간 설정
a = 1 # 구간 시작점
b = 5 # 구간 끝점
tol = 1e-6 # 허용 오차

# 이분법 실행
root = bisection(quadratic_function, a, b, tol)

# 결과 출력
print(f"f(x) = x^2 - 4의 근: x = {root:.6f}")
print(f"f({root:.6f}) = {quadratic_function(root):.6e}") # 함수값 확인

```

⑦ 출력

주어진 함수의 해 $f(x) = x^2 - 4 = 0$ 의 해를 [1,5] 구간에서 찾는다.

```

초기 구간: [a, b] = [1, 5]
반복 1: [a, b] = [1.000000, 5.000000], c = 3.000000, f(c) = 5.000000e+00

```

반복 2: [a, b] = [1.000000, 3.000000], c = 2.000000, f(c) = 0.000000e+00
 최종 구간: [a, b] = [1.000000, 3.000000]
 $f(x) = x^2 - 4$ 의 근: $x = 2.000000$
 $f(2.000000) = 0.000000e+00$

(2.3.3) Python SciPy 라이브러리를 활용한 이분법

Python의 SciPy 라이브러리를 사용하면 optimize.bisect() 함수를 통해 이분법을 쉽게 구현할 수 있다.

주석으로 설명한 소스 코드	출력 결과
<pre>from scipy.optimize import bisect # 함수 정의 f = lambda x: x**2 - 4 # SciPy 이분법 root = bisect(f, 1, 3) print("SciPy 이분법으로 구한 근:", root)</pre>	SciPy 이분법 으로 구한 근: 2.0

(2.4) 뉴턴-랩슨법(Newton-Raphson Method)

(2.4.1) 뉴턴-랩슨법의 설명 (테일러 급수 기반)

(2.4.2) 뉴턴-랩슨법의 설명 (그래프 기반)

(2.4.3) 파이썬 소스 코드 함수 구현

```
def newton_raphson(f, df, x0, tol=1e-6, max_iter=100):
```

```
    """
```

뉴턴-랩슨법 구현 함수 (반복 과정 상세 출력)

Parameters:

f: 근을 구할 함수

df: 함수의 도함수

x0: 초기 추정치

tol: 허용 오차

max_iter: 최대 반복 횟수

Returns:

근의 근사값

"""

x = x0

print(f"초기 추정값: x0 = {x0}")

for i in range(max_iter):

fx = f(x)

dfx = df(x)

if abs(fx) < tol:

print(f"반복 {i + 1}: x = {x:.6f}, f(x) = {fx:.6e}, 수렴")

return x

if dfx == 0:

raise ValueError("미분값이 0이 되어 수렴하지 않습니다.")

print(f"반복 {i + 1}: x = {x:.6f}, f(x) = {fx:.6e}, f'(x) = {dfx:.6e}")

x -= fx / dfx

raise ValueError("최대 반복 횟수를 초과하여 수렴하지 않았습니다.")

함수와 도함수 정의

f = lambda x: x**3 - 6*x**2 + 11*x - 6

df = lambda x: 3*x**2 - 12*x + 11

뉴턴-랩슨법 실행 (다른 초기값으로 실행)

x0 = 2.1 # 초기 추정값

root = newton_raphson(f, df, x0)

print("뉴턴-랩슨법으로 구한 근:", root)

(2.4.4) 실습 예제

주어진 함수의 해 $f(x) = x^3 - 6x^2 + 11x - 6 = 0$ 의 해를초기값 $x_n = 2.1$ 로 설정하여 뉴턴-랩슨법을 사용해 구한다.

초기 추정값: x0 = 2.1

반복 1: x = 2.100000, f(x) = -9.900000e-02, f'(x) = -9.700000e-01

반복 2: x = 1.997938, f(x) = 2.061847e-03, f'(x) = -9.999872e-01

반복 3: x = 2.000000, f(x) = -1.753115e-08, 수렴

뉴턴-랩슨법으로 구한 근: 2.0000000175311468

(2.4.5) Python SciPy 이용한 뉴턴-랩슨법

SciPy의 `optimize.newton` 함수를 사용하면 뉴턴-랩슨법을 간단히 구현할 수 있다.

주석으로 설명한 소스 코드	출력 결과
<pre>from scipy.optimize import newton # SciPy 뉴턴-랩슨법 실행 root = newton(f, x0=2.5, fprime=df) print("SciPy 뉴턴-랩슨법으로 구한 근:", root)</pre>	SciPy 뉴턴-랩슨법으로 구한 근: 2.0

3 장 선형연립방정식 수치해법

(3.1) 선형연립방정식 수치해법 개요

(3.1.1) 스칼라(Scalar), 벡터(Vector), 행렬(Matrix) 의 관련성 및 중요성

(3.1.2) 선형연립방정식들의 해 존재

(3.1.3) 행렬의 선형변환

① 행렬의 선형변환은?

벡터 공간에서 축의 스케일링, 회전, 반사, 전단 등을 포함한 변환을 수학적으로 설명하며, 이는 다양한 물리적, 공학적 시스템을 모델링하는 데 필수적이다.

② 선형변환을 이해하면?

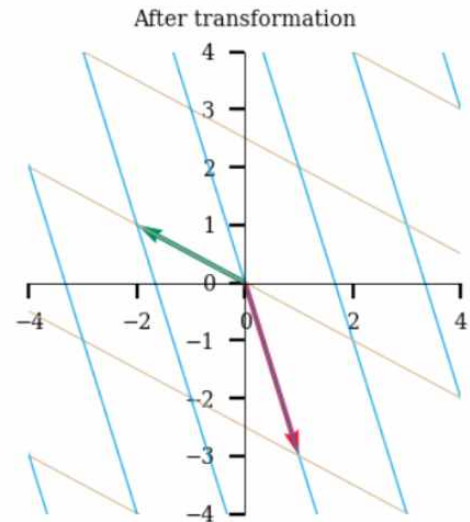
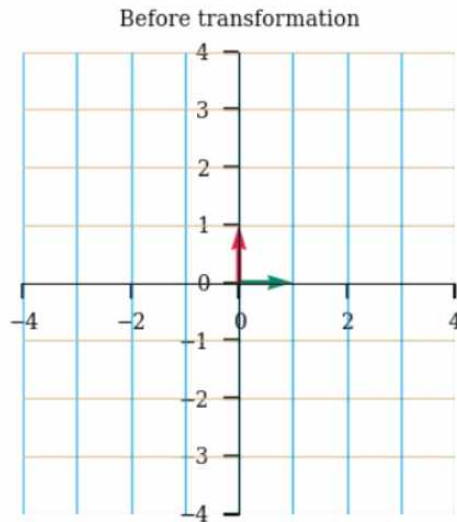
복잡한 시스템의 구조를 단순화하거나 최적화할 수 있으며, 특히 고유값과 고유벡터는 변환의 주축을 파악하는 데 중요한 도구다.

③ 예제:

빨간색 벡터와 녹색 벡터는 행렬 A 에 의해 변환된 기본 벡터를 나타낸다: 왼쪽 그래프는 원래의 기준 좌표계를 나타낸다. 이 벡터들은 변환 전에 단위 벡터로 시작하며, 원점에서 시작해 각 방향을 나타낸다. 오른쪽 그래프는 행렬 A 에 의해 변환된 후의 좌표계를 나타낸다. 변환 후 좌표계는 기울어진 형태로 나타나며, 원래의 직교 좌표계를 변경한다. 이러한 변환은 선형변환의 주요 특징인 방향 및 크기 변화를 시각적으로 보여준다.

```
import numpy as np
A = [[-2,1], [1,-3]]
A = np.array(A)
print(A)
```

$$\begin{bmatrix} -2 & 1 \\ 1 & -3 \end{bmatrix}$$



- $\begin{bmatrix} 0 \\ 1 \end{bmatrix} \rightarrow A \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$ (빨간색)
- $\begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow A \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$ (녹색)

㉞ 파이썬 코드로 계산:

코드	출력 결과
<pre>import numpy as np # 행렬 A 정의 A = np.array([[-2, 1], [1, -3]]) # 기본 벡터 e1 = np.array([1, 0]) # x축 단위 벡터 e2 = np.array([0, 1]) # y축 단위 벡터 # 변환 결과 계산 transformed_e1 = A @ e1 transformed_e2 = A @ e2 print("A:\n", A)</pre>	<pre>A: [[-2 1] [1 -3]] Transformed e1 (green vector): [-2 1] Transformed e2 (red vector): [1 -3]</pre>

```
print("Transformed e1 (green vector):",
transformed_e1)
print("Transformed e2 (red vector):",
transformed_e2)
```

(3.1.4) 해 존재를 위한 행렬의 행공간(row space)과 영공간(null space)

① 특이행렬 예제 코드

`np.linalg.solve`를 사용하여 $[A]x = b$ 의 특정 해를 계산하려 하지만, $[A]$ 특이행렬(역행렬이 없음)로, 특정 해를 구할 수 없다는 결과가 출력된다. 영공간 벡터는 $[A]x = 0$ 을 만족하는 벡터로, 전체 해집합은 특정 해 + 영공간 벡터의 조합으로 표현될 수 있다.

```
import numpy as np
from scipy.linalg import null_space

# 행렬과 벡터 정의
A = np.array([[1, 2], [2, 4]]) # 계수 행렬
b = np.array([8, 4]) # 상수 벡터

# 선형 연립방정식의 해 찾기
try:
    x_particular = np.linalg.solve(A, b) # 특정 해 계산
except np.linalg.LinAlgError:
    x_particular = "해를 구할 수 없음 (행렬이 특이행렬)"

# 영공간(null space) 계산
null_space_vector = null_space(A)

# 결과 출력
print("계수 행렬 A:")
print(A)
print("\n상수 벡터 b:")
print(b)
print("\n특정 해 (x_particular):")
print(x_particular)
print("\n영공간 벡터 (null_space):")
print(null_space_vector)
```

② 특이행렬 예제 코드 실행

계수 행렬 A:

```
[[1 2]  
 [2 4]]
```

상수 벡터 b:

```
[8 4]
```

특정 해 (x_particular):

해를 구할 수 없음 (행렬이 특이행렬)

영공간 벡터 (null_space):

```
[-0.89442719]  
 [ 0.4472136 ]]
```

(3.1.5) 선형연립방정식의 수치해법 개요

(3.2) 가우스 소거법

(3.2.1) 가우스 소거법 개요

(3.2.2) 전진 소거(Forward Elimination

(3.2.3) 전진소거 알고리즘

① 전진소거 코드:

```
for row in range(0, n-1): # 첫 번째 행부터 마지막 전 행까지 반복  
    for i in range(row+1, n): # 현재 행(row) 아래의 모든 행에 대해 반복  
        # 피벗(pivot)을 기준으로 소거 계수(factor) 계산  
        factor = A[i, row] / A[row, row]  
  
        # 행렬 A의 해당 행 업데이트 (열 방향 반복)
```

```
for j in range(row, n):
    A[i, j] = A[i, j] - factor * A[row, j]

# 벡터 b 업데이트
b[i] = b[i] - factor * b[row]
```

② 소스코드에서의 구현

```
import numpy as np

# 전진 소거를 수행하는 함수
def forward_elimination(A, b, n):
    """
    가우스 소거법의 전진 소거 단계:
    주어진 행렬 A와 벡터 b에 대해, A를 상삼각 행렬로 변환하고
    b를 변환된 A에 맞게 조정한다.

    Args:
        A: (numpy.ndarray) 계수 행렬 (n x n 크기)
        b: (numpy.ndarray) 상수 벡터
        n: (int) 행렬의 크기 (A는 n x n 행렬)

    Returns:
        A: (numpy.ndarray) 상삼각 형태로 변환된 행렬
        b: (numpy.ndarray) 변환된 벡터
    """
    for row in range(0, n-1): # 첫 번째 행부터 마지막 전 행까지 반복
        for i in range(row+1, n): # 현재 행(row) 아래의 모든 행에 대해 반복
            # 피벗(pivot)을 기준으로 소거 계수(factor) 계산
            factor = A[i, row] / A[row, row]

            # 행렬 A의 해당 행 업데이트 (열 방향 반복)
            for j in range(row, n):
                A[i, j] = A[i, j] - factor * A[row, j]

            # 벡터 b 업데이트
            b[i] = b[i] - factor * b[row]
```

```
# 결과 출력
print('A = \n%s and b = %s' % (A, b))
return A, b

# 초기 행렬 A와 벡터 b 정의
A = np.array([[ -1, 1, 2], # A는 3x3 크기의 행렬
               [ 3, -2, 1],
               [ 2, -1, 2]])

print("Original A:\n", A)

b = np.array([2, 6, 4]) # b는 3x1 크기의 벡터

# 전진 소거 수행
A, b = forward_elimination(A, b, 3)

# 결과 출력
print("Upper Triangular Matrix A:\n", A)
print("Modified Vector b:\n", b)
```

③ 결과 출력

```
Original A:
[[-1  1  2]
 [ 3 -2  1]
 [ 2 -1  2]]
A =
[[-1  1  2]
 [ 0  1  7]
 [ 0  0 -1]] and b = [ 2 12 -4]
Upper Triangular Matrix A:
[[-1  1  2]
 [ 0  1  7]
 [ 0  0 -1]]
Modified Vector b:
[ 2 12 -4]
```

④ 소스코드 설명

(3.2.4) 후진 대입(Backward Substitution)

① 후진 대입 최종 공식

② 후진 대입 코드:

```

for row in range(0, n-1): # 첫 번째 행부터 마지막 전 행까지 반복
    for i in range(row+1, n): # 현재 행(row) 아래의 모든 행에 대해 반복
        # 피벗(pivot)을 기준으로 소거 계수(factor) 계산
        factor = A[i, row] / A[row, row]

        # 행렬 A의 해당 행 업데이트 (열 방향 반복)
        for j in range(row, n):
            A[i, j] = A[i, j] - factor * A[row, j]

        # 벡터 b 업데이트
        b[i] = b[i] - factor * b[row]

```

③ 소스코드에서의 구현

```

import numpy as np

# 후진 대입(Back Substitution)을 수행하는 함수
def back_substitution(a, b, n):
    """
    후진 대입(Back Substitution) 과정:
    상삼각 행렬 형태의 계수 행렬 A와 벡터 b를 사용해 미지수 x를 계산.

    Args:
        a: (numpy.ndarray) 상삼각 행렬 형태의 계수 행렬 (n x n 크기)
        b: (numpy.ndarray) 상수 벡터
        n: (int) 행렬의 크기 (a는 n x n 행렬)

    Returns:

```

```

x: (numpy.ndarray) 계산된 미지수 벡터
"""

# 미지수 x를 저장할 벡터 (초기값: 모두 0)
x = np.zeros((n, 1))

# 첫 번째로 마지막 행(맨 아래)의 값을 계산
x[n-1] = b[n-1] / a[n-1, n-1] # x_n = b_n / a_nn

# n-2번째 행부터 첫 번째 행까지 반복
for row in range(n-2, -1, -1): # row는 n-2, n-3, ..., 0 순으로 감소
    sums = b[row] # 현재 행의 상수항 b[row]

    # 현재 행에서 이미 계산된 x[j] 값을 사용해 합을 계산
    for j in range(row+1, n): # j는 row+1부터 n-1까지
        sums = sums - a[row, j] * x[j] # b[row]에서 a[row, j] * x[j]를 차감

    # 현재 미지수 x[row] 계산
    x[row] = sums / a[row, row] # x[row] = (b[row] - 나머지 합) / 피벗 값

# 결과 출력
print('A = \n%s and b = %s' % (a, b))
return x

# 상삼각 행렬 A와 상수 벡터 b 정의
A = np.array([[1, 1, 2], # A는 3x3 상삼각 행렬
              [0, -2, 1],
              [0, 0, 2]])

b = np.array([2, 6, 4]) # b는 3x1 상수 벡터

# 후진 대입 수행
x = back_substitution(A, b, 3)

# 결과 출력
print('x = %s' % (x)) # 계산된 x 출력

```

④ 결과 출력

A =

```
[[ 1  1  2]
 [ 0 -2  1]
 [ 0  0  2]] and b = [2 6 4]
x = [[ 0.]
      [-2.]
      [ 2.]]
```

(3.2.5) 가우스 소거법 전체 코드 함수 구현

① 소스코드

```
import numpy as np

# 전진 소거 함수
def forward_elimination(A, b, n):
    """
    전진 소거(Forward Elimination) 과정:
    주어진 행렬 A와 벡터 b를 상삼각 형태로 변환.
    """
    for row in range(0, n-1): # 첫 번째 행부터 마지막 전 행까지 반복
        for i in range(row+1, n): # 현재 행(row) 아래의 모든 행에 대해 반복
            factor = A[i, row] / A[row, row] # 피벗(row, row) 기준 소거 계수 계
            산
            for j in range(row, n): # 현재 행의 계수 갱신
                A[i, j] = A[i, j] - factor * A[row, j]
            b[i] = b[i] - factor * b[row] # 상수 벡터 b 갱신
    return A, b

# 후진 대입 함수
def back_substitution(a, b, n):
    """
    후진 대입(Back Substitution) 과정:
    상삼각 행렬 형태의 계수 행렬 A와 벡터 b를 사용해 미지수 x를 계산.
    """
    x = np.zeros((n, 1)) # 해를 저장할 벡터 x (초기값: 0)
    x[n-1] = b[n-1] / a[n-1, n-1] # 마지막 변수부터 계산 (x_n)
    for row in range(n-2, -1, -1): # n-2번째 행부터 첫 번째 행까지 반복
        sums = b[row] # 현재 행의 상수항 b[row]
```

```

        for j in range(row+1, n): # 이미 계산된 x[j]를 사용
            sums = sums - a[row, j] * x[j]
        x[row] = sums / a[row, row] # x[row] 계산
    return x

# 문제 정의 (첨부된 행렬과 상수 벡터)
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float) # 계수 행렬
b = np.array([8, -11, -3], dtype=float) # 상수 벡터
n = 3 # 행렬의 크기

# 주어진 문제 출력
print("주어진 연립 방정식:")
for i in range(n):
    equation = " + ".join([f"{A[i, j]}*x{j+1}" for j in range(n)]) + f" = {b[i]}"
    print(equation)
print()

# 1. 전진 소거 수행
A, b = forward_elimination(A, b, n)

# 2. 후진 대입 수행
x = back_substitution(A, b, n)

# 결과 출력
print("\n계수 행렬 (상삼각 형태):")
print(A)
print("\n변환된 상수 벡터:")
print(b)
print("\n미지수 벡터 (해):")
print(x)

```

② 결과 출력

$$\begin{aligned}
 2x + y - z &= 8 \\
 -3x - y + 2z &= -11 \\
 -2x + y + 2z &= -3
 \end{aligned}$$

주어진 연립 방정식:

$$2.0 \cdot x_1 + 1.0 \cdot x_2 + -1.0 \cdot x_3 = 8.0$$

$$-3.0 \cdot x_1 + -1.0 \cdot x_2 + 2.0 \cdot x_3 = -11.0$$

$$-2.0 \cdot x_1 + 1.0 \cdot x_2 + 2.0 \cdot x_3 = -3.0$$

계수 행렬 (상삼각 형태):

$$\begin{bmatrix} 2. & 1. & -1. \\ 0. & 0.5 & 0.5 \\ 0. & 0. & -1. \end{bmatrix}$$

변환된 상수 벡터:

$$[8. \ 1. \ 1.]$$

미지수 벡터 (해):

$$\begin{bmatrix} 2. \\ 3. \\ -1. \end{bmatrix}$$

(3.2.6) 수치적 안정성을 위한 부분 피벗팅

① 가우스 소거중 피벗 검사

② 부분 피벗팅:

③ 가우스 소거:

피벗 행을 기준으로 아래 행들을 소거하여 계단형 행렬로 변환한다.

④ 후진 대입:

상삼각형 행렬에서 해를 차례로 계산한다.

⑤ 실행 예제:

주어진 행렬 A 는 첫 번째 피벗이 매우 작은 값 1×10^{-20} 을 가지며, 이로 인해 수치적 불안정성이 발생할 수 있다.

$$\begin{aligned} 1.0^{-20}x + y &= 1 \\ x + y &= 2 \end{aligned}$$

아래 문제는 피벗이 0이므로 여러번의 열교환이 이루어져야 한다.

$$\begin{aligned} 0x + 10y + 10z &= 20 \\ 0x + 0y + 5z &= 5 \\ x + y + z &= 3 \end{aligned}$$

부분 피벗팅을 사용하면 첫 번째 피벗을 절댓값이 큰 값으로 변경하여 안정성을 확보한다.

⑥ 소스코드

```

import numpy as np

def gauss_elimination_with_pivoting(A, b):
    """
    가우스 소거법 (부분 피벗팅 포함, 과정 출력)

    Parameters:
        A: 계수 행렬 (numpy array)
        b: 상수 벡터 (numpy array)

    Returns:
        x: 해 벡터
    """
    n = len(b) # 행렬 크기

    print("초기 상태:")
    print("A =\n", A)
    print("b =", b)
    print()

    # 전진 소거 (가우스 소거)
    for i in range(n):
        # 부분 피벗팅: 현재 열에서 절댓값이 가장 큰 요소를 피벗으로 설정
        max_row = np.argmax(abs(A[i:, i])) + i
        if max_row != i: # 행 교환이 필요한 경우
            print(f"부분 피벗팅: 행 {i}와 행 {max_row} 교환")
            A[[i, max_row]] = A[[max_row, i]] # 행 교환
            b[[i, max_row]] = b[[max_row, i]] # 상수 벡터도 동일하게 교환
            print("A =\n", A)
            print("b =", b)
            print()

        # 가우스 소거
        for j in range(i + 1, n):
            factor = A[j, i] / A[i, i]
            A[j, i:] -= factor * A[i, i:]
            b[j] -= factor * b[i]

```

```

# 현재 상태 출력
print(f"가우스 소거 후 (단계 {i + 1}):")
print("A =\n", A)
print("b =", b)
print()

# 후진 대입
x = np.zeros(n)
for i in range(n - 1, -1, -1):
    x[i] = (b[i] - np.dot(A[i, i + 1:], x[i + 1:])) / A[i, i]

return x

# 문제 정의
A = np.array([[0, 10, 10], # 계수 행렬
              [0, 0, 5],
              [1, 1, 1]], dtype=float)

b = np.array([20, 5, 3], dtype=float) # 상수 벡터

# 가우스 소거법 수행
x = gauss_elimination_with_pivoting(A, b)

# 결과 출력
print("최종 해 벡터 x:")
for i, val in enumerate(x):
    print(f"x{i+1} = {val}")

```

㉦ 결과 출력

$$\begin{array}{rcl}
 0x + 10y + 10z & = & 20 \\
 0x + 0y + 5z & = & 5 \\
 x + y + z & = & 3
 \end{array}$$

초기 상태:

```

A =
[[ 0. 10. 10.]
 [ 0.  0.  5.]
 [ 1.  1.  1.]]

```

3.1 선형연립방정식 수치해법 개요

$$b = [20. \quad 5. \quad 3.]$$

부분 피벗팅: 행 0와 행 2 교환

$$A =$$

$$[[1. \quad 1. \quad 1.]$$

$$[0. \quad 0. \quad 5.]$$

$$[0. \quad 10. \quad 10.]]$$

$$b = [3. \quad 5. \quad 20.]$$

가우스 소거 후 (단계 1):

$$A =$$

$$[[1. \quad 1. \quad 1.]$$

$$[0. \quad 0. \quad 5.]$$

$$[0. \quad 10. \quad 10.]]$$

$$b = [3. \quad 5. \quad 20.]$$

부분 피벗팅: 행 1와 행 2 교환

$$A =$$

$$[[1. \quad 1. \quad 1.]$$

$$[0. \quad 10. \quad 10.]$$

$$[0. \quad 0. \quad 5.]]$$

$$b = [3. \quad 20. \quad 5.]$$

가우스 소거 후 (단계 2):

$$A =$$

$$[[1. \quad 1. \quad 1.]$$

$$[0. \quad 10. \quad 10.]$$

$$[0. \quad 0. \quad 5.]]$$

$$b = [3. \quad 20. \quad 5.]$$

가우스 소거 후 (단계 3):

$$A =$$

$$[[1. \quad 1. \quad 1.]$$

$$[0. \quad 10. \quad 10.]$$

$$[0. \quad 0. \quad 5.]]$$

$$b = [3. \quad 20. \quad 5.]$$

최종 해 벡터 x:

$$x1 = 1.0$$

$$x2 = 1.0$$

 $x_3 = 1.0$

(3.3) LU Decomposition Method

(3.3.1) LU 분해법 개요(일반적으로 Doolittle LU 분해법)

(3.3.2) 알고리즘의 원리

(3.3.3) $A = LU$ 증명

(3.3.4) Doolittle LU 분해법 공식 유도 과정 이용해서 손으로 풀기

(3.3.5) Doolittle LU Python 코드 예제

① 문제 1

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$$

```
import numpy as np
```

```
def doolittle_lu_decomposition(A):
```

```
    n = len(A)
```

```
    L = np.zeros_like(A, dtype=float)
```

```
    U = np.zeros_like(A, dtype=float)
```

```
    for i in range(n):
```

```
        L[i][i] = 1 # L의 대각선은 1로 고정
```

```
        for j in range(i, n):
```

```
            U[i][j] = A[i][j] - sum(L[i][k] * U[k][j] for k in range(i))
```

```
        for j in range(i+1, n):
```

```
            L[j][i] = (A[j][i] - sum(L[j][k] * U[k][i] for k in range(i))) / U[i][i]
```

```
    return L, U
```

```
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
```

```
L, U = doolittle_lu_decomposition(A)
```

```
print("L:")
print(L)
print("U:")
print(U)
```

㉟ 실행 결과

```
L:
[[ 1.  0.  0. ]
 [-1.5  1.  0. ]
 [-1.  4.  1. ]]
U:
[[ 2.  1. -1. ]
 [ 0.  0.5  0.5]
 [ 0.  0. -1. ]]
```

㉠ 코드

```
import numpy as np

def lu_decomposition(A):
    """
    LU 분해 함수

    Parameters:
        A: 계수 행렬 (numpy array)

    Returns:
        L: 하삼각 행렬
        U: 상삼각 행렬
    """
    n = len(A)
    L = np.eye(n) # L을 단위 행렬로 초기화
    U = np.zeros_like(A, dtype=float)
```

```

for i in range(n):
    # U 계산
    for j in range(i, n):
        U[i, j] = A[i, j] - np.dot(L[i, :i], U[:i, j])

    # L 계산
    for j in range(i+1, n):
        L[j, i] = (A[j, i] - np.dot(L[j, :i], U[:i, i])) / U[i, i]

return L, U

def solve_lu(L, U, b):
    """
    LU 분해를 이용한 선형 방정식 풀이

    Parameters:
        L: 하삼각 행렬
        U: 상삼각 행렬
        b: 상수 벡터

    Returns:
        x: 해 벡터
    """
    # 전진 대입: L * y = b
    n = len(b)
    y = np.zeros_like(b, dtype=float)
    for i in range(n):
        y[i] = b[i] - np.dot(L[i, :i], y[:i])

    # 후진 대입: U * x = y
    x = np.zeros_like(b, dtype=float)
    for i in range(n-1, -1, -1):
        x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]

    return x

# 계수 행렬과 상수 벡터 정의
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

```

```
# LU 분해 및 해 풀이
L, U = lu_decomposition(A)
solution = solve_lu(L, U, b)

print("하삼각 행렬 L:\n", L)
print("상삼각 행렬 U:\n", U)
print("해:", solution)
```

③ 예제: Doolittle LU 분해법 실습

$$\begin{aligned} 2x + y - z &= 8 \\ -3x - y + 2z &= -11 \\ -2x + y + 2z &= -3 \end{aligned}$$

주석으로 설명한 소스 코드	출력 결과
<pre># 계수 행렬과 상수 벡터 정의 A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float) b = np.array([8, -11, -3], dtype=float) # LU 분해 및 해 풀이 L, U = lu_decomposition(A) solution = solve_lu(L, U, b) print("하삼각 행렬 L:\n", L) print("상삼각 행렬 U:\n", U) print("해:", solution)</pre>	<pre>하삼각 행렬 L: [[1. 0. 0.] [-1.5 1. 0.] [-1. 0.5 1.]] 상삼각 행렬 U: [[2. 1. -1.] [0. -0.5 0.5] [0. 0. 3.]] 해: [2. 3. -1.]</pre>

(3.3.6) Crout LU 분해법

- ① 하삼각 행렬 L의 원소 계산:
- ② 상삼각 행렬 U의 원소 계산:
- ③ Crout LU 분해법 알고리즘
- ④ Crout LU 분해법 손으로 풀기
- ⑤ Crout LU 분해법 Python 코드 예제

㉠ 코드

```
def crout_lu_decomposition(A):
    n = len(A)
    L = np.zeros_like(A, dtype=float)
    U = np.eye(n, dtype=float) # U는 대각선이 1인 행렬

    for j in range(n):
        for i in range(j, n):
            L[i][j] = A[i][j] - sum(L[i][k] * U[k][j] for k in range(j))
        for i in range(j+1, n):
            U[j][i] = (A[j][i] - sum(L[j][k] * U[k][i] for k in range(j))) / L[j][j]
    return L, U

A = np.array([[4, 12, -16], [12, 37, -43], [-16, -43, 98]], dtype=float)
L, U = crout_lu_decomposition(A)
print("L:")
print(L)
print("U:")
print(U)
```

㉠ 실행결과

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$$

```
L:
[[ 4.  0.  0.]
 [12.  1.  0.]
 [-16.  5.  9.]]
U:
[[ 1.  3. -4.]
 [ 0.  1.  5.]
 [ 0.  0.  1.]]
```

(3.3.7) $[A] = [L][D][L^T]$ 분해 및 Cholesky LU 분해법

- ① $[A] = [L][D][L^T]$ 분해
- ② $[A] = [L][L^T]$ Cholesky LU 분해
- ③ 임의의 행렬 A(3X3) 분해의 방법.

- ④ Cholesky LU 분해법 알고리즘
- ⑤ Cholesky LU 분해법 예제
- ⑥ Cholesky LU 분해법 Python 코드 예제

㉞ 코드

```
def cholesky_decomposition(A):
    n = len(A)
    L = np.zeros_like(A, dtype=float)

    for i in range(n):
        for j in range(i+1):
            if i == j: # 대각 원소 계산
                L[i][j] = np.sqrt(A[i][i] - sum(L[i][k] ** 2 for k in range(j)))
            else:
                L[i][j] = (A[i][j] - sum(L[i][k] * L[j][k] for k in range(j))) / L[j][j]
    return L

A = np.array([[4, 12, -16], [12, 37, -43], [-16, -43, 98]], dtype=float)
L = cholesky_decomposition(A)

print("L:")
print(L)
print("L^T:")
print(L.T)
```

㉞ 실행 결과

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}$$

```
L:
[[ 2.  0.  0.]
 [ 6.  1.  0.]
 [-8.  5.  3.]]
L^T:
[[ 2.  6. -8.]
```

```
[ 0.  1.  5.]
[ 0.  0.  3.]]
```

(3.3.8) SciPy에서의 LU 분해

SciPy는 `scipy.linalg.lu` 함수를 사용하여 LU 분해를 직접 수행할 수 있다. 이 함수는 행 교환 행렬 (피봇팅을 위한 행렬) P, 하삼각 행렬 (대각선 값은 1) L, 상삼각 행렬, U를 반환한다.

SciPy는 `scipy.linalg.lu` 함수를 사용하여 LU 분해를 직접 수행할 수 있다. 이 함수는 행 교환 행렬 (피봇팅을 위한 행렬) P, 하삼각 행렬 (대각선 값은 1) L, 상삼각 행렬, U를 반환한다.

주석으로 설명한 소스 코드	출력 결과
<pre>from scipy.linalg import lu # 계수 행렬 정의 A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float) # LU 분해 수행 P, L, U = lu(A) # 결과 출력 print("행 교환 행렬 P:\n", P) print("하삼각 행렬 L:\n", L) print("상삼각 행렬 U:\n", U)</pre>	<pre>행 교환 행렬 P: [[0. 1. 0.] [1. 0. 0.] [0. 0. 1.]] 하삼각 행렬 L: [[1. 0. 0.] [-0.66666667 1. 0.] [-0.66666667 0.5 1.]] 상삼각 행렬 U: [[-3. -1. 2.] [0. 0.33333333 0.66666667] [0. 0. 3.]]</pre>

(3.3.9) SciPy에서 LU 분해를 활용한 선형 방정식 풀이

SciPy의 `lu_solve`와 `lu_factor` 함수를 사용하면 LU 분해를 활용하여 선형 방정식을 효율적으로 해결할 수 있다.

주석으로 설명한 소스 코드	출력 결과
<pre>from scipy.linalg import lu_factor, lu_solve # 계수 행렬과 상수 벡터 정의 A = np.array([[2, 1, -1], [-3, -1, 2],</pre>	<pre>해 x: [2. 3. -1.]</pre>

```

        [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

# LU 분해 수행
lu_piv = lu_factor(A)

# LU 분해를 이용한 선형 방정식 풀이
x = lu_solve(lu_piv, b)

# 결과 출력
print("해 x:\n", x)

```

(3.3.10) LU 분해법과 NumPy의 numpy.linalg.solve 비교

numpy.linalg.solve는 내부적으로 LU 분해를 사용하여 선형 방정식을 풀이한다. 별도의 L과 U 행렬을 반환하지 않고, 해 x를 바로 계산하는 데 중점을 둔다.

주석으로 설명한 소스 코드	출력 결과
<pre> # 계수 행렬과 상수 벡터 정의 A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float) b = np.array([8, -11, -3], dtype=float) # 선형 방정식 풀이 x = np.linalg.solve(A, b) print("해 x:\n", x) </pre>	<pre> 해 x: [2. 3. -1.] </pre>

(3.4) Jacobi, Gauss-Seidal, SOR 반복법

(3.4.1) Jacobi, Gauss-Seidal, SOR 반복법 개요

(3.4.2) Jacobi 반복법

① 최종식:

② 수렴 조건은 무엇인가?

③ Condition Number $K(A)$

④ 코드

$$\begin{aligned} 4x_1 + x_2 - x_3 &= 3 \\ 2x_1 + 7x_2 + x_3 &= 19 \\ x_1 - 3x_2 + 12x_3 &= 31 \end{aligned}$$

초기값 $x_0 = [0, 0, 0]$ 로 시작하여 Jacobi 반복법을 적용한다.

```
import numpy as np

def jacobi(A, b, x0, tol=1e-10, max_iterations=100):
    """
    Jacobi 반복법으로 Ax = b를 풀이
    Args:
        A: 계수 행렬
        b: 상수 벡터
        x0: 초기 추정 값
        tol: 수렴 허용 오차
        max_iterations: 최대 반복 횟수
    Returns:
        x: 해 벡터
        iterations: 반복 횟수
    """
    n = len(A)
    x = x0.copy()
    x_new = np.zeros_like(x)

    for iteration in range(max_iterations):
        for i in range(n):
            sigma = sum(A[i][j] * x[j] for j in range(n) if j != i)
            x_new[i] = (b[i] - sigma) / A[i][i]

        # 수렴 확인
        if np.linalg.norm(x_new - x, ord=np.inf) < tol:
```

```

        return x_new, iteration + 1

    x = x_new.copy()

    raise ValueError("Jacobi method did not converge")

# 예제 문제
A = np.array([[4, 1, -1],
              [2, 7, 1],
              [1, -3, 12]], dtype=float)

b = np.array([3, 19, 31], dtype=float)
x0 = np.zeros_like(b)

# Jacobi method 실행
solution, iterations = jacobi(A, b, x0)

# 결과 출력
print(f"해: {solution}")
print(f"반복 횟수: {iterations}")

```

⑤ 코드 실행 결과:

```

해: [1.  2.  3.]
반복 횟수: 9

```

(3.4.3) Gauss-Seidal 반복법

① Python 코드

$$\begin{aligned}
 4x_1 + x_2 - x_3 &= 3 \\
 2x_1 + 7x_2 + x_3 &= 19 \\
 x_1 - 3x_2 + 12x_3 &= 31
 \end{aligned}$$

초기값 $x_0 = [0, 0, 0]$ 로 시작하여 Jacobi 반복법을 적용한다. 두 연속 반복의 값이 무한 노름 기준 10^{-6} 이하의 차이를 보이면 수렴한 것으로 간주.

```

import numpy as np

# 계수 행렬 A와 상수 벡터 b
A = np.array([[4, 1, -1],
              [2, 7, 1],
              [1, -3, 12]], dtype=float)
b = np.array([3, 19, 31], dtype=float)

# 초기값 설정
x = np.zeros_like(b)
tolerance = 1e-6 # 수렴 기준
max_iterations = 100

# Gauss-Seidel 반복법
def gauss_seidel(A, b, x, max_iterations, tolerance):
    n = len(b)
    for iteration in range(max_iterations):
        x_old = x.copy()

        for i in range(n):
            sum1 = sum(A[i, j] * x[j] for j in range(i))      # j < i의 항 계산
            sum2 = sum(A[i, j] * x_old[j] for j in range(i + 1, n)) # j > i의 항
계산
            x[i] = (b[i] - sum1 - sum2) / A[i, i]

        # 수렴 조건 확인
        if np.linalg.norm(x - x_old, ord=np.inf) < tolerance:
            print(f"Converged after {iteration + 1} iterations")
            return x
        print("Did not converge")
    return x

# 실행
solution = gauss_seidel(A, b, x, max_iterations, tolerance)
print("Solution:", solution)

```

② 코드 실행 결과:

Converged after 12 iterations
Solution: [1. 2. 3.]

(3.4.4) SOR(Successive Over-Relaxation) 반복법

① Python 코드

다음은 SOR 방법을 Python으로 구현한 코드이다:

```
import numpy as np

def sor_method(A, b, omega, tol=1e-10, max_iter=1000):
    """
    SOR(Successive Over-Relaxation) Method to solve Ax = b.

    Parameters:
    - A: Coefficient matrix
    - b: Constant vector
    - omega: Relaxation factor ( $\omega$ )
    - tol: Tolerance for convergence
    - max_iter: Maximum number of iterations

    Returns:
    - x: Solution vector
    - iter_count: Number of iterations
    """
    n = len(b)
    x = np.zeros(n) # Initial guess
    for iter_count in range(max_iter):
        x_old = x.copy()
        for i in range(n):
            sigma = sum(A[i][j] * x[j] for j in range(n) if j != i)
            x[i] = (1 - omega) * x_old[i] + omega * (b[i] - sigma) / A[i][i]

        # Convergence check
        if np.linalg.norm(x - x_old, ord=np.inf) < tol:
            return x, iter_count + 1
    raise ValueError("SOR method did not converge")
```

```

# Example comparison
A = np.array([[4, 1, 2], [1, 3, -1], [2, -1, 3]])
b = np.array([7, 4, 5])

# Parameters for comparison
omega_gs = 1.0 # Gauss-Seidel ( $\omega = 1$ )
omega_sor = 1.25 # SOR ( $\omega = 1.25$ )

# Gauss-Seidel
solution_gs, iterations_gs = sor_method(A, b, omega_gs)
# SOR
solution_sor, iterations_sor = sor_method(A, b, omega_sor)

# Result comparison
{
    "Gauss-Seidel": {"Solution": solution_gs, "Iterations": iterations_gs},
    "SOR ( $\omega=1.25$ )": {"Solution": solution_sor, "Iterations": iterations_sor}
}

```

② 코드 실행 및 결과 분석

위 코드를 사용해 Gauss-Seidel($\omega=1$)과 SOR($\omega=1.25$)을 비교하면, $\omega=1.25$ 일 때 더 적은 반복 횟수로 수렴함을 확인할 수 있다. 이는 SOR 방법이 수렴 속도를 효과적으로 향상시킨다는 것을 보여준다.

```

{'Gauss-Seidel': {'Solution': array([0.07692308, 2.07692308, 2.30769231]),
  'Iterations': 58},
 'SOR ( $\omega=1.25$ )': {'Solution': array([0.07692308, 2.07692308, 2.30769231]),
  'Iterations': 28}}

```

4 장 수치미분(Numerical Differentiation)

(4.1) 수치미분의 개요

(4.1.1) 테일러 급수(Taylor Series)

테일러 급수는 미분 가능한 함수 $f(t)$ 를 다항식 형태로 근사하여 복잡한 함수를 단순화하거나, 특정 점 근처에서의 함수 값을 계산하는 데 사용된다.

① 중심점 a 에서의 테일러 급수:

$$f(t) = f(a) + f'(a)(t-a) + f''(a)(t-a)^2/2! + f'''(a)(t-a)^3/3! + \dots + h.o.t \quad (4.1.2-2)$$

② 중심점 0에서의 테일러 급수(Maclaurin Series)

$$f(t) = f(0) + f'(0)t + f''(0)t^2/2! + f'''(0)t^3/3! + h.o.t \quad (4.1.2-3)$$

㉠ 함수 $f(t) = e^t$ 를 중심점 0에서 확장한 결과를 보여준다:

$$f(t) = e^t = 1 + t + t^2/2! + t^3/3! + h.o. \quad (4.1.2-4)$$

③ 소스 코드

```
import numpy as np
import matplotlib.pyplot as plt

# x 값 정의 및 원래 함수 y = exp(x) 계산
x = np.arange(0, 3.01, 0.01) # MATLAB의 -0:0.01:3과 동일
y = np.exp(x)

# 원래 함수 플롯
plt.plot(x, y, 'k', linewidth=3, label='y = exp(x)')

# 축 설정
```

```

plt.axis([0, 3, 0, 10])
plt.grid(True)
plt.title("Taylor Series Approximation of exp(x)")
plt.xlabel("x")
plt.ylabel("y")

# 첫 번째 다항식 (0차항)
Pol = [1]
y1 = np.polyval(Pol, x)
plt.plot(x, y1, 'b--', linewidth=2.2, label='y1 (0th order)')

# 두 번째 다항식 (1차항 추가)
Pol = [1, 0]
y2 = np.polyval(Pol, x)
plt.plot(x, y1 + y2, 'r--', linewidth=2.2, label='y1 + y2 (1st
order)')

# 세 번째 다항식 (2차, 3차항 추가)
Pol = [1/6, 1/2, 0, 0]
y3 = np.polyval(Pol, x)
plt.plot(x, y1 + y2 + y3, 'g--', linewidth=2.2, label='y1 + y2 +
y3 (3rd order)')

# 범례 추가
plt.legend()
plt.show()

```

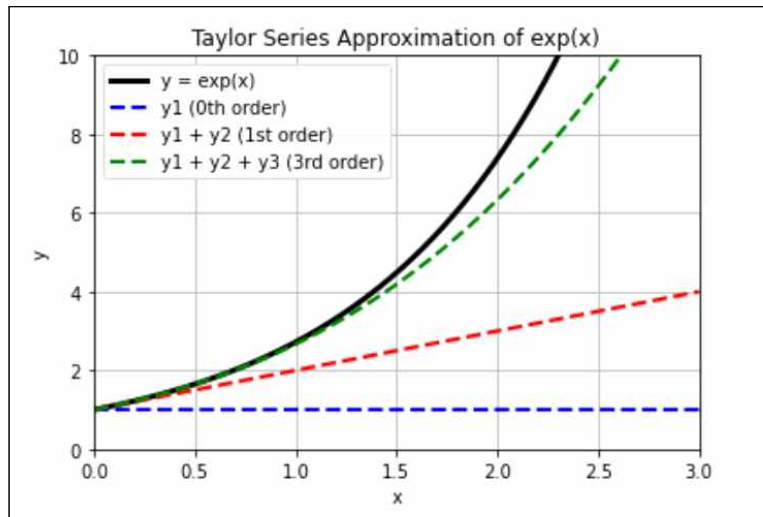
④ 실행 결과

주어진 파이썬 코드는 테일러 급수를 이용하여 함수 $y = e^x$ 를 다항식으로 근사한 후, 근사 결과를 그래프로 비교하는 프로그램이다. MATLAB의 `polyval()` 함수는 NumPy의 `numpy.polyval()` 함수로 대체 가능하다. 이를 사용하여 다항식 계수를 기반으로 y 값을 계산하고, 이를 원래 함수 $y = e^x$ 와 비교하며 플롯을 생성한다. 파이썬 실행하여 그림으로 확인해 보면 항의 수가 많아지면 원식에 가까워 지는 것을 알수 있다.

아래 그림은 테일러 급수의 이론적 정의와 실제 전개 과정, 그리고 특정 함수에 적용한 예제를 통해 수학적 원리를 명확히 보여준다. 이를 통해 수치 계산 및 함수 근사에서 테일러 급수가 얼마나 중요한 도구인지 확인할 수 있다.

함수 $f(t) = e^t$ 1번째 항까지, 2번째 항까지, 3번째 항까지 근사화시킨 그림을 나타낸다.

$$f(t) = e^t = 1 + t + \frac{t^2}{2!} + \frac{t^3}{3!} + h.o. \quad (4.1.2-5)$$



(4.1.2) 수치미분의 개요

(4.2) 전진, 후진, 중심차분, Richardson 외삽법

(4.2.1) 전진차분(forward difference):

(4.2.2) 후진차분(backward difference):

(4.2.3) 중심차분(Central difference):

(4.2.4) Richardson 외삽법(Richardson extrapolation):

(4.2.5) 오차 해석

(4.2.6) 수치미분 코드 구현

① 예제

3차 함수 및 도함수 정의하고 여러방법과 정확도를 비교한다.

```
f = lambda x: x**3 - 6*x**2 + 11*x - 6 # 3차 함수
f_prime = lambda x: 3*x**2 - 12*x + 11 # 실제 도함수
```

```
import numpy as np
import matplotlib.pyplot as plt

# 수치미분 함수
def numerical_derivative(f, x, h=1e-5, method='central'):
    if method == 'forward':
        return (f(x + h) - f(x)) / h
    elif method == 'backward':
        return (f(x) - f(x - h)) / h
    elif method == 'central':
        return (f(x + h) - f(x - h)) / (2 * h)
    elif method == 'richardson':
        # Richardson 외삽법
        central_h = (f(x + h) - f(x - h)) / (2 * h)
        central_h2 = (f(x + h / 2) - f(x - h / 2)) / h
        return (4 * central_h2 - central_h) / 3
    else:
        raise ValueError("Invalid method. Choose 'forward', 'backward', 'central', or 'richardson'.")

h=0.1

# 3차 함수 및 도함수 정의
f = lambda x: x**3 - 6*x**2 + 11*x - 6 # 3차 함수
f_prime = lambda x: 3*x**2 - 12*x + 11 # 실제 도함수

# 계산 지점
x_vals = np.linspace(0, 4, 20) # Discrete points in [0, 4]

# 수치 미분 값 계산
forward_values = np.array([numerical_derivative(f, x, h, 'forward') for x in x_vals])
backward_values = np.array([numerical_derivative(f, x, h, 'backward') for x in x_vals])
central_values = np.array([numerical_derivative(f, x, h, 'central') for x in x_vals])
richardson_values = np.array([numerical_derivative(f, x, h, 'richardson') for x in x_vals])
exact_values = np.array([f_prime(x) for x in x_vals])
```

```

# 에러 계산
forward_errors = np.abs(forward_values - exact_values)
backward_errors = np.abs(backward_values - exact_values)
central_errors = np.abs(central_values - exact_values)
richardson_errors = np.abs(richardson_values - exact_values)

# 그래프 시각화
plt.figure(figsize=(10, 6))
plt.scatter(x_vals, exact_values, label='Exact Derivative', color='black', marker='o',
s=50, zorder=5)
plt.step(x_vals, forward_values, label='Forward Difference', where='mid', linestyle='-',
alpha=0.7)
plt.step(x_vals, backward_values, label='Backward Difference', where='mid',
linestyle='-', alpha=0.7)
plt.step(x_vals, central_values, label='Central Difference', where='mid', linestyle='-',
alpha=0.7)
plt.step(x_vals, richardson_values, label='Richardson Extrapolation', where='mid',
linestyle='-', alpha=0.7)

# 그래프 설정
plt.title('Discrete Derivative Approximations for a Cubic Function', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('Derivative Value', fontsize=12)
plt.legend()
plt.grid(True)
plt.show()

# 에러 시각화
plt.figure(figsize=(10, 6))
plt.plot(x_vals, forward_errors, label='Forward Difference Error', linestyle='--',
marker='o')
plt.plot(x_vals, backward_errors, label='Backward Difference Error', linestyle='--',
marker='o')
plt.plot(x_vals, central_errors, label='Central Difference Error', linestyle='--',
marker='o')
plt.plot(x_vals, richardson_errors, label='Richardson Extrapolation Error',
linestyle='--', marker='o')

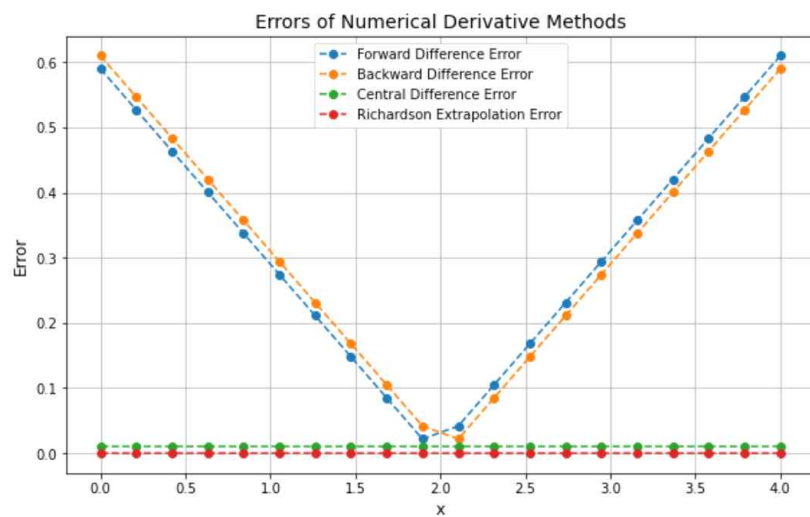
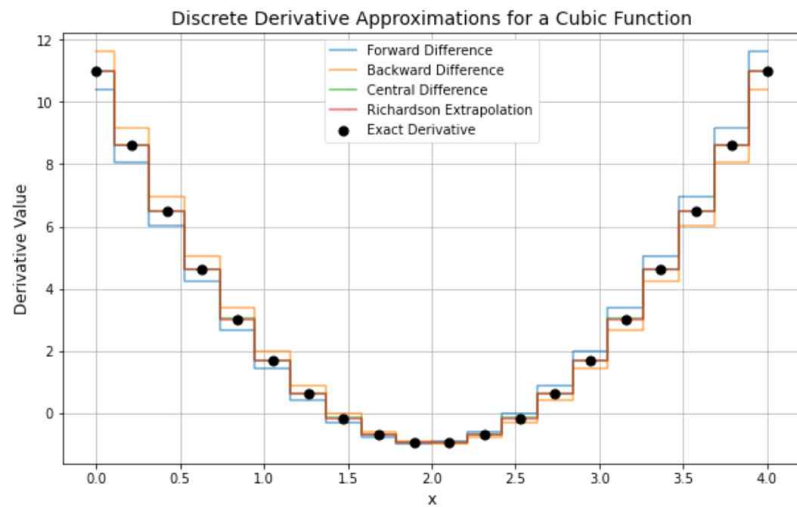
# 그래프 설정

```

```
plt.title('Errors of Numerical Derivative Methods', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('Error', fontsize=12)
plt.legend()
plt.grid(True)
plt.show()
```

② 코드 실행 시 화면에 출력될 결과

Enter the value of h (e.g., 1e-5): 0.1



5 장 수치적분(Numerical Integration)

(5.1) 리만 (Riemann) 적분

(5.1.1) 리만 적분의 정의

(5.1.2) 리만 (Riemann) 적분 가능성

(5.1.3) 리만 합 구현 및 가시화

아래는 리만 적분을 구현하고 가시화하는 Python 코드이다. 좌측, 우측, 중앙 리만 합을 각각 계산하고 비교하는 방식으로 작성하였다.

① 구현 방식:

- ㉠ 함수 $f(x) = x^2$ 를 정의하고, 구간 [0,2]에서 적분을 계산.
- ㉡ 리만 합의 세 가지 방법(좌측, 우측, 중앙)을 각각 구현.

② 결과:

- ㉠ 좌측 리만 합과 우측 리만 합은 구간의 시작과 끝에 따라 오차가 존재.
- ㉡ 중앙 리만 합은 오차가 줄어들며 더 정확한 결과 제공.

③ 가시화:

- ㉠ 리만 합 계산 과정에서 사용된 직사각형이 시각적으로 표현됨.
- ㉡ 좌측, 중앙, 우측 리만 합에 따라 직사각형 높이가 달라짐을 확인 가능.

④ 활용:

- ㉠ 리만 적분은 수치적분의 기초로, 사다리꼴 법칙이나 심프슨 법칙과 같은 고급 적분법의 기초가 된다.
- ㉡ 머신러닝에서는 데이터 분포를 근사하거나 손실 함수의 값을 계산할 때 유용하게 사용된다.

⑤ 코드

```
import numpy as np
```

```
import matplotlib.pyplot as plt

# 함수 정의
def f(x):
    return x**2 # 예제 함수:  $f(x) = x^2$ 

# 리만 합 계산 함수
def riemann_sum(f, a, b, n, method="left"):
    dx = (b - a) / n
    x = np.linspace(a, b, n + 1)

    if method == "left":
        x_points = x[:-1] # 좌측 리만 합
    elif method == "right":
        x_points = x[1:] # 우측 리만 합
    elif method == "midpoint":
        x_points = (x[:-1] + x[1:]) / 2 # 중앙 리만 합
    else:
        raise ValueError("method must be 'left', 'right', or 'midpoint'")

    return np.sum(f(x_points) * dx)

# 적분 구간 및 함수 정의
a, b = 0, 2 # 적분 구간
n = 10 # 구간 개수

# 리만 합 계산
left_sum = riemann_sum(f, a, b, n, method="left")
right_sum = riemann_sum(f, a, b, n, method="right")
midpoint_sum = riemann_sum(f, a, b, n, method="midpoint")

# 결과 출력
print(f"Left Riemann Sum: {left_sum}")
print(f"Right Riemann Sum: {right_sum}")
print(f"Midpoint Riemann Sum: {midpoint_sum}")

# 가시화
x = np.linspace(a, b, 100)
y = f(x)
```

```

x_rect = np.linspace(a, b, n + 1)
dx = (b - a) / n

# 리만 합 그림 그리기
plt.figure(figsize=(8, 6))
plt.plot(x, y, 'b-', label='f(x) = x^2')

# 좌측 리만 합 시각화
for i in range(n):
    x_rect_left = [x_rect[i], x_rect[i], x_rect[i+1], x_rect[i+1]]
    y_rect_left = [0, f(x_rect[i]), f(x_rect[i]), 0]
    plt.fill(x_rect_left, y_rect_left, 'r', edgecolor='k', alpha=0.3, label='Left Riemann'
            if i == 0 else '')

# 중앙 리만 합 시각화
for i in range(n):
    mid_x = (x_rect[i] + x_rect[i + 1]) / 2
    x_rect_mid = [x_rect[i], x_rect[i], x_rect[i+1], x_rect[i+1]]
    y_rect_mid = [0, f(mid_x), f(mid_x), 0]
    plt.fill(x_rect_mid, y_rect_mid, 'g', edgecolor='k', alpha=0.3, label='Midpoint
Riemann' if i == 0 else '')

# 우측 리만 합 시각화
for i in range(n):
    x_rect_right = [x_rect[i], x_rect[i], x_rect[i+1], x_rect[i+1]]
    y_rect_right = [0, f(x_rect[i+1]), f(x_rect[i+1]), 0]
    plt.fill(x_rect_right, y_rect_right, 'b', edgecolor='k', alpha=0.3, label='Right
Riemann' if i == 0 else '')

# 그래프 설정
plt.legend(loc='upper left', fontsize=10)
plt.title('Riemann Sums Visualization')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.grid()
plt.show()

```

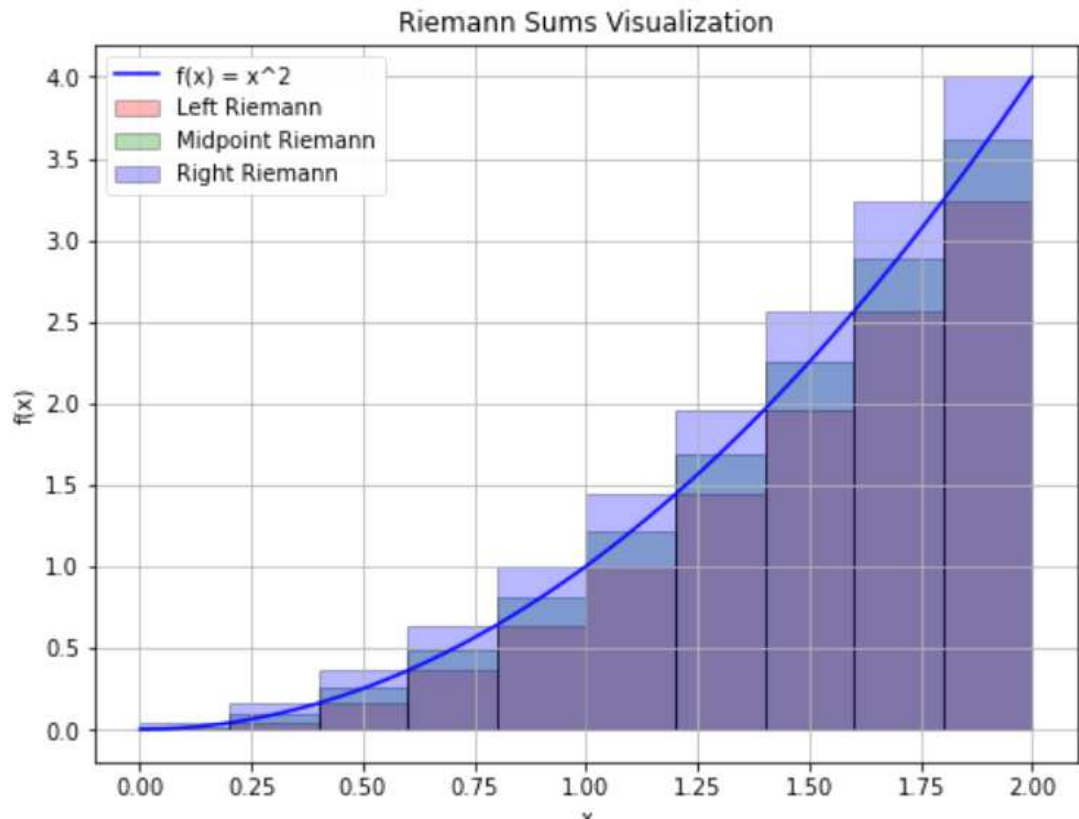
⑥ 코드 실행

5.2 사다리꼴 적분(Trapezoidal rule)

Left Riemann Sum: 2.2800000000000007

Right Riemann Sum: 3.08

Midpoint Riemann Sum: 2.6600000000000006



(5.2) 사다리꼴 적분(Trapezoidal rule)

(5.2.1) 사다리꼴 적분(Trapezoidal rule)

(5.2.2) 사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 1

① 이론값과 사다리꼴 공식을 통한 값의 비교

② 코드

사다리꼴 공식 구현: 예러 출력 및 주석 추가

def f(x):

5.2 사다리꼴 적분(Trapezoidal rule)

```
# 정의된 함수:  $f(x) = x + 3$ 
return x + 3

# 적분 구간 [a, b]
a = 0 # 시작 값
b = 10 # 끝 값

# 구간 분할 수
n = 100 # 사다리꼴 구간 수
sum1 = 0 # 중간점 합 초기화

# 사다리꼴 공식의 중간점 계산
for i in range(1, n): # 1부터 n-1까지 반복 (수정: 범위 오류)
    x = a + (b - a) / n * i # 현재 x값 계산
    sum1 += f(x) # 중간점 함수 값 누적

# 사다리꼴 적분 공식
trapezoid = (b - a) * (f(a) + 2 * sum1 + f(b)) / (2 * n)

# 출력
print("사다리꼴 공식의 적분값 =", trapezoid)

# 정확값 계산 (적분값  $f(x) = x + 3$ 의 해석적 적분 결과:  $F(x) = 0.5x^2 + 3x$ )
exact_value = (0.5 * b**2 + 3 * b) - (0.5 * a**2 + 3 * a)

# 절대 오차 계산
error = abs(exact_value - trapezoid)
print("정확한 값 =", exact_value)
print("절대 오차 =", error)
```

③ 수치 결과

```
사다리꼴 공식의 적분값 = 79.99999999999999
정확한 값 = 80.0
절대 오차 = 1.4210854715202004e-14
```

(5.2.3) 사다리꼴 적분(Trapezoidal rule) Python 프로그래밍 2

① 함수를 정의.

```
def y( x ):
    return (1 / (1 + x * x))
```

② 사다리꼴 함수를 정의. (구간1, 구간2, 나누는 횟수)

```
def trapezoidal (a, b, n):
    h = (b - a) / n
    s = (y(a) + y(b))
    i = 1
    while i < n:
        s += 2 * y(a + i * h)
        i += 1
    return ((h / 2) * s)
```

③ 구간과 나누는 횟수를 입력한 뒤 출력.

```
x0 = 0
xn = 1
n = 6
print ("Value of integral is ",
        "%.4f"%trapezoidal(x0, xn, n))
```

④ OUTPUT

Value of integral is 0.7842

⑤ 프로그램 종합

```
def y( x ):
    return (1 / (1 + x * x))
```

Value of integral is 0.7842

```
def trapezoidal (a, b, n):
    h = (b - a) / n
    s = (y(a) + y(b))
    i = 1
    while i < n:
        s += 2 * y(a + i * h)
        i += 1
    return ((h / 2) * s)

x0 = 0
xn = 1
n = 6
print ("Value of integral is ",
        "%.4f"%trapezoidal(x0, xn, n))
```

(5.3) Richardson 외삽법(Extrapolation)

(5.3.1) Richardson Extrapolation 개요

(5.3.2) 파이썬 코드

① 코드

```
import numpy as np

def trapezcomp(f, a, b, n):
    """
    복합 사다리꼴 공식으로 적분값 계산
    Parameters:
    - f: 적분할 함수
    - a: 적분 구간 시작점
    - b: 적분 구간 끝점
    - n: 분할 구간 수
    Returns:
    - 적분값 In
    """
    h = (b - a) / n # 구간 크기
    x = a
    In = f(a)
```

5.3 Richardson 외삽법(Extrapolation)

```
for k in range(1, n): # 중간 구간의 함수값 누적
    x = x + h
    ln += 2 * f(x)
return (ln + f(b)) * h * 0.5

def richardson_extrapolation(f, a, b, n1, n2):
    """
    Richardson Extrapolation을 사용하여 적분값 계산
    Parameters:
    - f: 적분할 함수
    - a: 적분 구간 시작점
    - b: 적분 구간 끝점
    - n1: 첫 번째 구간 분할 수
    - n2: 두 번째 구간 분할 수 (n2 > n1)
    Returns:
    - Richardson Extrapolation으로 계산된 적분값
    """
    # 두 개의 복합 사다리꼴 공식 값 계산
    I1 = trapezcomp(f, a, b, n1) # 적분값 (n1 구간)
    I2 = trapezcomp(f, a, b, n2) # 적분값 (n2 구간)

    # Richardson Extrapolation 공식
    I_richardson = (4 * I2 - I1) / (4 - 1)

    return I1, I2, I_richardson

if __name__ == '__main__':
    def func(x):
        return np.sin(x) # 적분할 함수 f(x) = sin(x)

    # 적분 구간 및 분할 수
    a = 0
    b = np.pi / 2
    n1 = 4 # 첫 번째 분할 수
    n2 = 8 # 두 번째 분할 수 (n2 > n1)

    # Richardson Extrapolation 수행
    I1, I2, I_richardson = richardson_extrapolation(func, a, b, n1, n2)

    # 출력
```

```

print("사다리꼴 적분값 (n1 구간) =", I1)
print("사다리꼴 적분값 (n2 구간) =", I2)
print("Richardson Extrapolation 적분값 =", I_richardson)

# 정확한 값 계산
exact_value = 1.0 #  $\int [0, \pi/2] \sin(x) dx = 1$ 
error = abs(exact_value - I_richardson)
print("정확한 값 =", exact_value)
print("Richardson Extrapolation 오차 =", error)

```

② 실행 결과

```

사다리꼴 적분값 (n1 구간) = 0.9871158009727754
사다리꼴 적분값 (n2 구간) = 0.9967851718861696
Richardson Extrapolation 적분값 = 1.0000082955239675
정확한 값 = 1.0
Richardson Extrapolation 오차 = 8.29552396752753e-06

```

(5.4) Romberg Integration

(5.4.1) Romberg integration Python 프로그래밍

① 예제 코드

```

import numpy as np

# 복합 사다리꼴 공식 함수 정의
def trapezcomp(f, a, b, n):
    """
    복합 사다리꼴 공식으로 적분값 계산

    Parameters:
    - f: 적분할 함수
    - a: 적분 구간 시작점
    - b: 적분 구간 끝점
    - n: 분할 구간 수
    """

```

Returns:

- 적분값 In

"""

h = (b - a) / n # 구간 크기

x = a

In = f(a)

for k in range(1, n): # 중간 구간의 함수값 누적

 x = x + h

 In += 2 * f(x)

return (In + f(b)) * h * 0.5

Romberg 적분 함수 정의

def romberg(f, a, b, p):

 """

 Romberg 적분법으로 적분값 계산

Parameters:

- f: 적분할 함수

- a: 적분 구간 시작점

- b: 적분 구간 끝점

- p: Romberg 행렬의 차원 (p x p)

Returns:

- Romberg 행렬 I

"""

I = np.zeros((p, p)) # Romberg 행렬 초기화

for k in range(0, p):

 # 복합 사다리꼴 공식으로 초기값 계산

 I[k, 0] = trapezcomp(f, a, b, 2**k)

 # Romberg 보간 공식 적용

 for j in range(0, k):

$$I[k, j+1] = (4^{j+1} * I[k, j] - I[k-1, j]) / (4^{j+1} - 1)$$

 # 각 단계의 결과 출력

 print(f"Step {k + 1}: {I[k, 0:k+1]}")

```

    return I

if __name__ == '__main__':
    # 적분할 함수 정의
    def func(x):
        return np.sin(x) # 함수 f(x) = sin(x)

    # Romberg 적분 설정
    a = 0 # 적분 구간 시작점
    b = np.pi / 2 # 적분 구간 끝점
    p_rows = 4 # Romberg 행렬 크기

    # Romberg 적분 수행
    print("Romberg 적분 결과:")
    I = romberg(func, a, b, p_rows)
    solution = I[p_rows-1, p_rows-1] # 최종 적분값

    # 정확한 적분값 계산 (f(x) = sin(x)의 해석적 적분)
    exact_value = 1.0 #  $\int [0, \pi/2] \sin(x) dx = 1$ 

    # 절대 오차 계산
    error = abs(exact_value - solution)

    # 결과 출력
    print("\n최종 Romberg 적분값 =", solution)
    print("정확한 적분값 =", exact_value)
    print("절대 오차 =", error)

```

② 예제 코드 실행 결과

```

Romberg 적분 결과:
Step 1: [0.78539816]
Step 2: [0.94805945  1.00227988]
Step 3: [0.9871158  1.00013458  0.99999157]
Step 4: [0.99678517  1.0000083  0.99999988  1.00000001]

최종 Romberg 적분값 = 1.0000000081440203
정확한 적분값 = 1.0
절대 오차 = 8.144020346989578e-09

```

(5.5) Simpson's 1/3rd Rule

(5.5.1) Simpson's Rule 개요

(5.5.2) Simpson's 1/3rd Rule 프로그래밍

$\int_0^{10} (x+3)dx$ 을 이론값, 사다리꼴 방법, 심슨법의 결과값 비교하고자 한다. 이론값은 $\int_0^{10} (x+3)dx = \left[\frac{1}{2}x^2 + 3x \right]_0^{10} = 80$ 으로 이론값은 80이다. 100회 나눈 사다리꼴 방식을

이용한 $I = (10-0) \frac{f(x_0) + 2 \sum_{i=1}^{99} f(x_i) + f(x_n)}{2*100}$ 값과 100회 나눈 심슨법을 이용한 $\frac{(10-0)}{3*100} \left(f(x_0) + 4 \sum_{i=1}^{49} f(x_{2i}) + 2 \sum_{i=1}^{50} f(x_{2i-1}) + f(x_n) \right)$ 값을 파이썬으로 구현하고 출력한다.

① 코드

```
# 함수와 적분 범위 설정
def f(x):
    """
    함수 정의: f(x) = x + 3
    """
    return x + 3

# 적분 구간과 분할 수
a = 0 # 적분 구간 시작점
b = 10 # 적분 구간 끝점
n = 100 # 구간 분할 수 (짝수여야 Simpson's Rule에 적합)

# 합 초기화
sum1 = 0 # 사다리꼴 공식을 위한 중간 합
```

```

sumeven = 0 # Simpson's Rule의 짝수 합
sumodd = 0 # Simpson's Rule의 홀수 합

# 구간별 합 계산
for i in range(1, n): # 범위 수정 (i=1부터 시작)
    x = a + (b - a) / n * i # 현재 x값 계산

    if i % 2 == 0: # 짝수 인덱스
        sumeven += f(x)
    else: # 홀수 인덱스
        sumodd += f(x)

# 모든 구간의 함수 값 합산 (사다리꼴 공식용)
sum1 += f(x)

# 사다리꼴 공식으로 적분값 계산
trapezoid = (b - a) * (f(a) + 2 * sum1 + f(b)) / (2 * n)

# Simpson's Rule로 적분값 계산
simpson = (b - a) * (f(a) + 4 * sumodd + 2 * sumeven + f(b)) / (3 * n)

# 정확한 적분값 계산 (해석적 적분 결과)
exact_value = (0.5 * b**2 + 3 * b) - (0.5 * a**2 + 3 * a)

# 절대 오차 계산
trapezoid_error = abs(exact_value - trapezoid)
simpson_error = abs(exact_value - simpson)

# 결과 출력
print("사다리꼴 공식의 값 =", trapezoid)
print("심슨 공식의 값 =", simpson)
print("정확한 적분값 =", exact_value)
print("사다리꼴 공식의 오차 =", trapezoid_error)
print("심슨 공식의 오차 =", simpson_error)

```

② 코드 결과 출력

사다리꼴 공식의 값 = 79.99999999999999

심슨 공식의 값 = 80.0
 정확한 적분값 = 80.0
 사다리꼴 공식의 오차 = 1.4210854715202004e-14
 심슨 공식의 오차 = 0.0

(5.6) Newton-Cotes Quadrature

(5.7) Gauss Quadrature Rule(가우스 구적법)

(5.7.1) Gauss Quadrature Rule (가우스 구적법)

(5.7.2) 가우스 구적법의 1중 적분

① 예제 1:

② 예제 2:

③ 코드:

$f(x) = e^{-x^2}$ 을 구간 $[0,1]$ 에서 적분한다. Gauss-Legendre 2점 법칙을 이용하여 구간 $[-1,1]$ 을 $[0,1]$ 로 변환하여 계산한다.

주석으로 설명한 소스 코드	출력 결과
<pre> import numpy as np # 1D 함수 정의 def f_1d(x): return np.exp(-x**2) # Gauss-Legendre 2점 법칙의 가중치와 노드 weights = np.array([1, 1]) nodes = np.array([-1/np.sqrt(3), 1/np.sqrt(3)]) # 적분 구간 a, b = 0, 1 </pre>	1중 적분 결과: 0.746594688282 8598

```
# 구간 변환: [-1, 1] -> [a, b]
transformed_nodes_1d = (b - a) / 2 * nodes + (b + a)
/ 2

# 적분 계산
integral_1d = (b - a) / 2 * np.sum(weights *
f_1d(transformed_nodes_1d))

# 출력
print(f"1중 적분 결과: {integral_1d}")
```

(5.7.3) 가우스 구적법의 2중 적분

(5.7.4) 가우스 구적법을 활용하여 사각형 면적 구하기

(5.7.5) 사각형 면적 구하기 코드

① 코드

```
# 사용자로부터 네 개의 점 좌표를 입력받는다.
x1, y1 = input("첫번째 좌표를 입력해주세요 : ").split() # 첫 번째 점의 좌표 입력
x1 = float(x1); y1 = float(y1) # 입력받은 좌표를 실수(float)로 변환
x2, y2 = input("두번째 좌표를 입력해주세요 : ").split() # 두 번째 점의 좌표 입력
x2 = float(x2); y2 = float(y2)
x3, y3 = input("세번째 좌표를 입력해주세요 : ").split() # 세 번째 점의 좌표 입력
x3 = float(x3); y3 = float(y3)
x4, y4 = input("네번째 좌표를 입력해주세요 : ").split() # 네 번째 점의 좌표 입력
x4 = float(x4); y4 = float(y4)

# 기본 자코비안(Jacobian) 계산
J1 = (1/8) * (((x1 - x3) * (y2 - y4)) - ((x2 - x4) * (y1 - y3)))
# 기본 자코비안은 네 점의 기하학적 위치를 기반으로 계산됨

# 2차 가우스 적분을 위한 자코비안 계산
# 적분점 1에서의 자코비안
J2_1 = (1/8) * (((x1 - x3) * (y2 - y4)) - ((x2 - x4) * (y1 - y3))) \
- (1/8) * (0.5774) * (((x1 - x2) * (y3 - y4)) - ((y1 - y2) * (x3 - x4))) \
- (1/8) * (0.5774) * (((x1 - x4) * (y2 - y3)) - ((y1 - y4) * (x2 - x3)))
```

```

# 적분점 2에서의 자코비안
J2_2 = (1/8) * (((x1 - x3) * (y2 - y4)) - ((x2 - x4) * (y1 - y3))) \
        - (1/8) * (0.5774) * (((x1 - x2) * (y3 - y4)) - ((y1 - y2) * (x3 - x4))) \
        - (1/8) * (-0.5774) * (((x1 - x4) * (y2 - y3)) - ((y1 - y4) * (x2 - x3)))

# 적분점 3에서의 자코비안
J2_3 = (1/8) * (((x1 - x3) * (y2 - y4)) - ((x2 - x4) * (y1 - y3))) \
        - (1/8) * (-0.5774) * (((x1 - x2) * (y3 - y4)) - ((y1 - y2) * (x3 - x4))) \
        - (1/8) * (0.5774) * (((x1 - x4) * (y2 - y3)) - ((y1 - y4) * (x2 - x3)))

# 적분점 4에서의 자코비안
J2_4 = (1/8) * (((x1 - x3) * (y2 - y4)) - ((x2 - x4) * (y1 - y3))) \
        - (1/8) * (-0.5774) * (((x1 - x2) * (y3 - y4)) - ((y1 - y2) * (x3 - x4))) \
        - (1/8) * (-0.5774) * (((x1 - x4) * (y2 - y3)) - ((y1 - y4) * (x2 - x3)))

# 가우스 적분 가중치 설정
g1w1 = 2 # 1차 가우스 적분의 가중치
g2w1 = 1 # 2차 가우스 적분의 첫 번째 가중치
g2w2 = 1 # 2차 가우스 적분의 두 번째 가중치

# 1차 가우스 적분 결과 출력
print('GP1-Gauss :', J1 * (g1w1 + g1w1))

# 2차 가우스 적분 결과 출력
print('GP2-Gauss :',
      J2_1 * (g2w1 * g2w1) + J2_2 * (g2w1 * g2w2) +
      J2_3 * (g2w2 * g2w1) + J2_4 * (g2w2 * g2w2))

```

② 코드 설명

㉠ $n = 1$ 일 때

$$\begin{aligned}
 |J| & \sum_{i=1}^1 \sum_{j=1}^1 w_i w_j \\
 &= \frac{1}{8} [(x_1 - x_3)(y_2 - y_4) - (x_2 - x_4)(y_1 - y_3)] (w_1 w_1)
 \end{aligned} \tag{5.7.5-1}$$

㉡ $n = 2$ 일 때

$$w_1 = w_2 = 1, \quad r_1 = -0.5774, r_2 = +0.5774, \quad t_1 = -0.5774,$$

$t_2 = +0.5774$ 을 사용하여 $\int_{-1}^1 \int_{-1}^1 1 \times |J| dr dt \approx \sum_{i=1}^2 \sum_{j=1}^2 \omega_i \omega_j |J|$ 을 계산

하고, 여기서 아래 J 는 r 과 t 의 함수이므로 두 번 summation 할 때 계산에 유의한다.

③ 결과비교

첫번째 좌표를 입력해주세요 : 0 5

두번째 좌표를 입력해주세요 : 1 1

세번째 좌표를 입력해주세요 : 4 1

네번째 좌표를 입력해주세요 : 8 5

GP1_Gauss : 22.0

GP2_Gauss : 22.0

6 장 수치 보간(interpolation)

(6.1) 뉴턴 분할 차분법

(6.1.1) Newton의 차분(divided-difference) 1차 선형 보간

(6.1.2) Newton의 차분(divided-difference) 2차 보간

(6.1.3) Newton의 차분(divided-difference) n차 보간

(6.1.4) Newton의 차분(divided-difference) 예제

이 코드는 뉴턴 나눗셈 차분법(Newton's Divided Difference Method)을 사용하여 주어진 데이터 포인트를 보간하는 과정과 그 결과를 시각화한다. 데이터 포인트 설정은 x -값: [0, 1, 2, 3] y -값: [1, 2, 0, 5] 으로 한다. 나눗셈 차분 계수 계산은 `divided_diff`를 호출하여 나눗셈 차분 계수 계산한다. 결과 계수는 [1, 1, -1.5, 1.1667]이다. 보간 다항식 계산은 새로운 x -값에 대해 `newton_poly`를 호출하여 y -값 계산한다. 원래 데이터 포인트와 보간된 다항식을 플롯으로 비교한다.

① 소스코드

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

# 데이터 포인트 (점 4개)
x_points = np.array([0, 1, 2, 3])
y_points = np.array([1, 2, 0, 5])

# 1. Newton's Divided Difference Interpolation
def divided_diff(x, y):
    n = len(y)
    coef = np.zeros([n, n])
    coef[:, 0] = y
```

```

    for j in range(1, n):
        for i in range(n - j):
            coef[i, j] = (coef[i + 1, j - 1] - coef[i, j - 1]) / (x[i + j] - x[i])
    return coef[0, :]

def newton_poly(coef, x_data, x):
    n = len(coef) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n - k] + (x - x_data[n - k]) * p
    return p

# Newton Interpolation coefficients
newton_coef = divided_diff(x_points, y_points)
x_new = np.linspace(0, 3, 100)
y_newton = newton_poly(newton_coef, x_points, x_new)

# Plotting the results
plt.figure(figsize=(10, 6))
# Data points
plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5)
# Newton's method
plt.plot(x_new, y_newton, label="Newton Divided Difference", color='blue', linewidth=2)

# Configure plot
plt.title('Newton Divided Difference Interpolation')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, color='black', linewidth=0.5, linestyle='--')
plt.axvline(0, color='black', linewidth=0.5, linestyle='--')
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

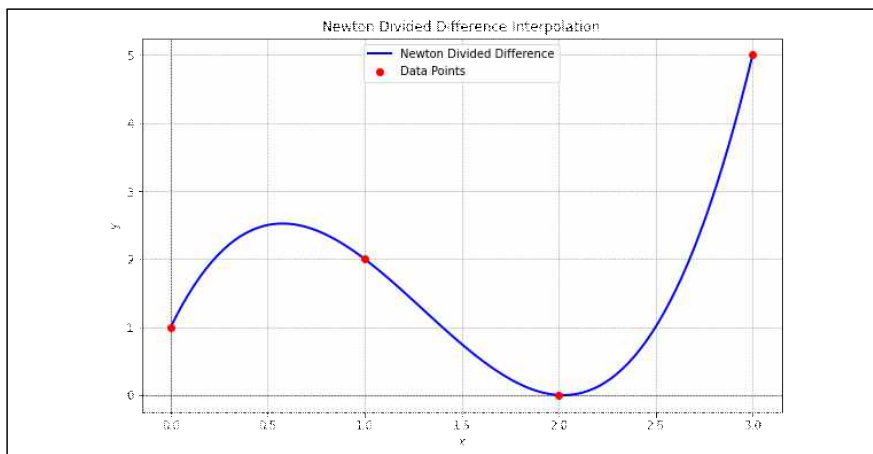
```

② 결과분석

보간된 다항식은 데이터 포인트를 정확히 통과하며, 다음과 같은 형태로 계산된다:

$$\begin{aligned}
 P_3(x) = & 1 + 1(x - 0) - 1.5(x - 0)(x - 1) \\
 & + 1.1667(x - 0)(x - 1)(x - 1)
 \end{aligned}
 \tag{6.2.4-1}$$

그래프에서 빨간 점은 원래 데이터 포인트, 파란 선은 Newton's Divided Difference로 계산된 보간 다항식이며, 그래프는 x 축과 y 축에서 보간 다항식이 데이터 점을 정확히 통과함을 보여준다.



(6.2) Lagrange Interpolation 수치 보간

S

(6.2.1) Lagrange Interpolation 정의

(6.2.2) Lagrange Interpolation 예제

이 코드는 `x_points = np.array([0, 1, 2, 3])` `y_points = np.array([1, 2, 0, 5])`로 데이터 포인트 정의한다.

① 코드

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

# 데이터 포인트 (점 4개)
x_points = np.array([0, 1, 2, 3])
y_points = np.array([1, 2, 0, 5])

# Lagrange Interpolation
lagrange_poly = lagrange(x_points, y_points)
y_lagrange = lagrange_poly(x_new)

# Plotting the results
plt.figure(figsize=(10, 6))
```

```

# Data points
plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5)

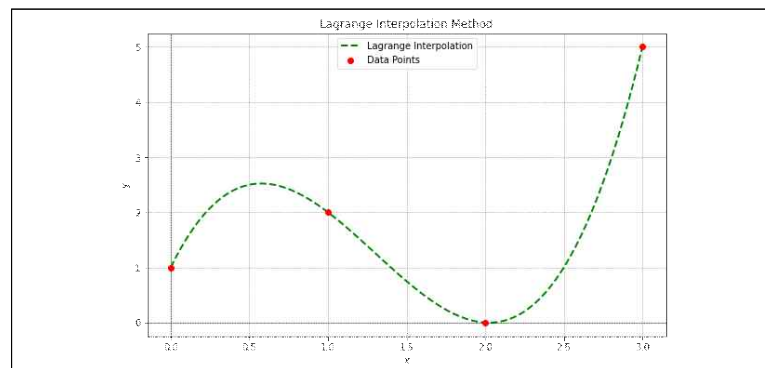
# Lagrange
plt.plot(x_new, y_lagrange, label="Lagrange Interpolation", color='green', linestyle='--',
linewidth=2)

# Configure plot
plt.title('Lagrange Interpolation Method')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, color='black', linewidth=0.5, linestyle='--')
plt.axvline(0, color='black', linewidth=0.5, linestyle='--')
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

```

② 코드 실행 및 결과분석

lagrange_poly = lagrange(x_points, y_points)을 사용하여 주어진 데이터 포인트를 사용해 라그랑주 다항식을 생성한다. 결과로 나온 lagrange_poly는 다항식 객체로, 특정 x, y 값을 계산할 수 있다. 새로운 x-값에 대한 y-값 계산은 y_lagrange = lagrange_poly(x_new)을 사용한다. x_new는 x-값의 새로운 범위(코드에서는 미리 정의된 변수로 가정), y_lagrange: 라그랑주 다항식을 사용해 계산된 y-값이다. 빨간 점: 원래 데이터 포인트, 녹색 점선: 라그랑주 보간법으로 계산된 다항식이다. 다항식은 주어진 데이터 포인트를 정확히 통과한다. 각 구간 사이에서 다항식은 곡선 형태를 유지하며 데이터를 보간한다. 라그랑주 보간법은 단순히 데이터 점을 통과하는 하나의 다항식을 생성한다. 데이터 점이 많아지면, 고차 다항식이 생성되며 진동 현상(Runge 현상)이 발생할 가능성이 있다. 따라서 데이터가 복잡한 경우, Cubic Spline 등의 방법이 더 적합할 수 있다.



(6.3) 스플라인 보간

(6.3.1) Quadratic Spline

(6.3.2) Quadratic Spline 예제

이 코드는 Quadratic Spline Interpolation(2차 스플라인 보간)을 사용하여 데이터를 매끄럽게 연결한 후, 결과를 시각화하는 과정을 보여준다. Quadratic Spline은 각 구간을 2차 다항식으로 모델링하여 자연스러운 곡선을 생성한다.

① 코드

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

# 데이터 포인트 (점 4개)
x_points = np.array([0, 1, 2, 3])
y_points = np.array([1, 2, 0, 5])

# Quadratic Spline Interpolation (using CubicSpline with k=2)
quadratic_spline = CubicSpline(x_points, y_points, bc_type='natural', extrapolate=False)
y_quadratic_spline = quadratic_spline(x_new)

# Plotting the results
plt.figure(figsize=(10, 6))
# Data points
plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5)

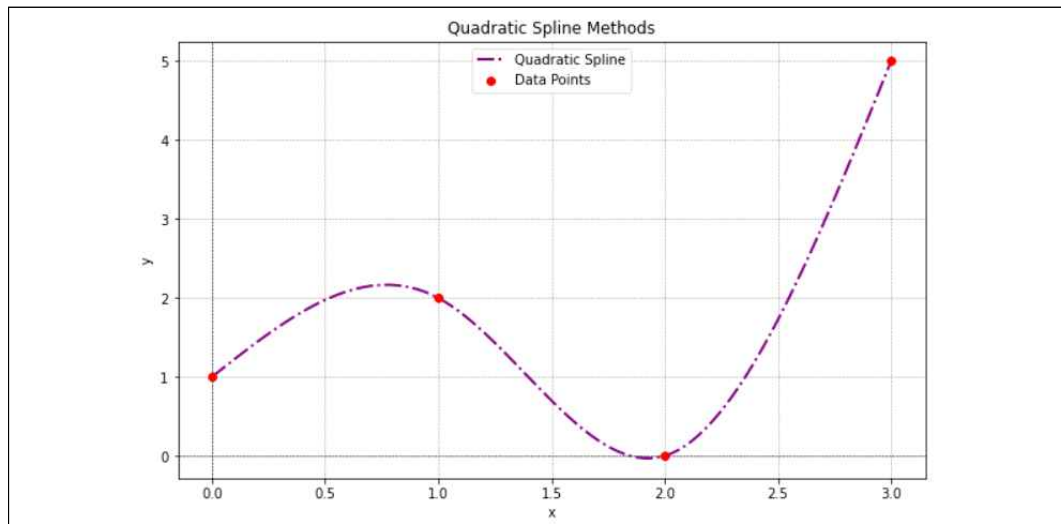
# Quadratic Spline
plt.plot(x_new, y_quadratic_spline, label="Quadratic Spline", color='purple',
linestyle='-.', linewidth=2)

# Configure plot
plt.title('Quadratic Spline Methods')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, color='black', linewidth=0.5, linestyle='--')
plt.axvline(0, color='black', linewidth=0.5, linestyle='--')
```

```
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()
```

② 코드 실행 및 결과분석

Quadratic Spline 함수는 주어진 데이터 포인트에 대해 3차 스플라인을 생성한다. 기본적으로 자연 경계 조건이 적용된다. `quadratic_spline = CubicSpline(x_points, y_points, bc_type='natural', extrapolate=False)`에서 `CubicSpline` 함수가 사용되었지만, 실제로는 2차 스플라인을 구성하도록 조건이 조정되고, `bc_type='natural'`: 자연 경계 조건을 적용, `extrapolate=False`: 주어진 데이터 포인트 범위를 초과한 외삽을 허용하지 않는다. `y_quadratic_spline = quadratic_spline(x_new)`에서 `x_new`는 `x`-값의 새로운 범위이고 (코드에서 `x_new`는 미리 정의되었다고 가정함.) `y_quadratic_spline`은 2차 스플라인을 사용해 계산된 `y`-값이다. 은 계산된 스플라인을 통해 구한다. 빨간 점은 원래 데이터 포인트이고 보라색 점선: Quadratic Spline으로 계산된 보간 곡선이다. 각 구간을 2차 다항식으로 연결하므로 곡선이 부드럽지만, 2차 도함수는 불연속일 수 있다. 자연 경계 조건으로 인해 양 끝점에서 곡선이 부드럽게 정리된다. 2차 스플라인은 Cubic Spline보다 단순하지만, 매끄러움이 덜하다.



(6.3.3) Cubic Spline

(6.3.4) Cubic Spline 예제

이 코드는 Cubic Spline Interpolation(3차 스플라인 보간)을 사용하여 데이터를 매끄럽게 연결한 후, 결과를 시각화하는 과정을 보여준다. Cubic Spline은 각 구간을 3차 다항식으로 모델링하여 매끄럽고 자연스러운 곡선을 생성한다.

① 소스코드

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline

# 데이터 포인트 (점 4개)
x_points = np.array([0, 1, 2, 3])
y_points = np.array([1, 2, 0, 5])

# 새 x 좌표 생성 (보간용)
x_new = np.linspace(0, 3, 100) # 0에서 3 사이를 100개의 점으로 샘플링

# Cubic Spline Interpolation
cubic_spline = CubicSpline(x_points, y_points) # CubicSpline 객체 생성
y_spline = cubic_spline(x_new) # x_new에 해당하는 y 값 계산

# 결과 그래프 시각화
plt.figure(figsize=(10, 6))

# 데이터 포인트 표시
plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5)

# Cubic Spline 곡선
plt.plot(x_new, y_spline, label="Cubic Spline", color='orange', linewidth=2)

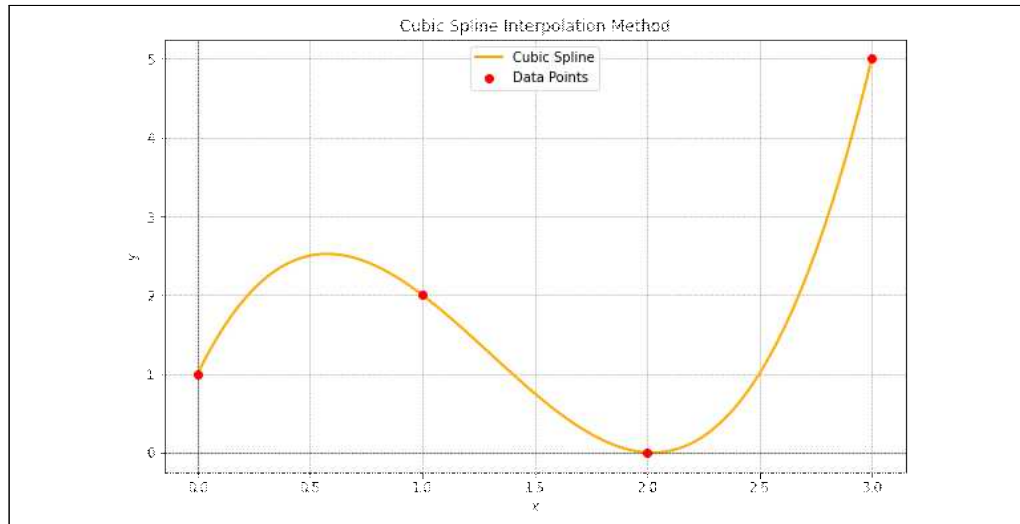
# 그래프 설정
plt.title('Cubic Spline Interpolation Method')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, color='black', linewidth=0.5, linestyle='--')
plt.axvline(0, color='black', linewidth=0.5, linestyle='--')
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

```

② 소스 실행 및 결과분석

CubicSpline 함수는 주어진 데이터 포인트에 대해 3차 스플라인을 생성한다. 기본적으로 자연 경계 조건이 적용된다. `y_spline = cubic_spline(x_new)`에서 `x_new`는 `x`-값의 새로운 범위이고 (코드에서 `x_new`는 미리 정의되었다고 가정함.) `y`-값은 계산된 스플라인을 통해 구한다. 빨간 점은 원

래 데이터 포인트이고 주황색 선은 Cubic Spline으로 계산된 보간 곡선이다.



(6.4) 수치보간법들과의 차이점

(6.4.1) 뉴턴의 분할 차분법(Newton's Divided Difference Method)과 라그랑주 보간법(Lagrange Interpolation)이 동일함

(6.4.2) Cubic Spline이 다른 보간법과 다른 이유

(6.4.3) 코드 구현

① 코드

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

# 데이터 포인트 (점 4개)
x_points = np.array([0, 1, 2, 3])
y_points = np.array([1, 2, 0, 5])

# 1. Newton's Divided Difference Interpolation
def divided_diff(x, y):
```

```

    n = len(y)
    coef = np.zeros([n, n])
    coef[:, 0] = y
    for j in range(1, n):
        for i in range(n - j):
            coef[i, j] = (coef[i + 1, j - 1] - coef[i, j - 1]) / (x[i + j] - x[i])
    return coef[0, :]

def newton_poly(coef, x_data, x):
    n = len(coef) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n - k] + (x - x_data[n - k]) * p
    return p

# Newton Interpolation coefficients
newton_coef = divided_diff(x_points, y_points)
x_new = np.linspace(0, 3, 100)
y_newton = newton_poly(newton_coef, x_points, x_new)

# 2. Cubic Spline Interpolation
cubic_spline = CubicSpline(x_points, y_points)
y_spline = cubic_spline(x_new)

# 3. Lagrange Interpolation
lagrange_poly = lagrange(x_points, y_points)
y_lagrange = lagrange_poly(x_new)

# 4. Quadratic Spline Interpolation (using CubicSpline with k=2)
quadratic_spline = CubicSpline(x_points, y_points, bc_type='natural',
                               extrapolate=False)
y_quadratic_spline = quadratic_spline(x_new)

# Plotting the results
plt.figure(figsize=(10, 6))
# Data points
plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5)
# Newton's method
plt.plot(x_new, y_newton, label="Newton's Divided Difference", color='blue',
         linewidth=2)

```

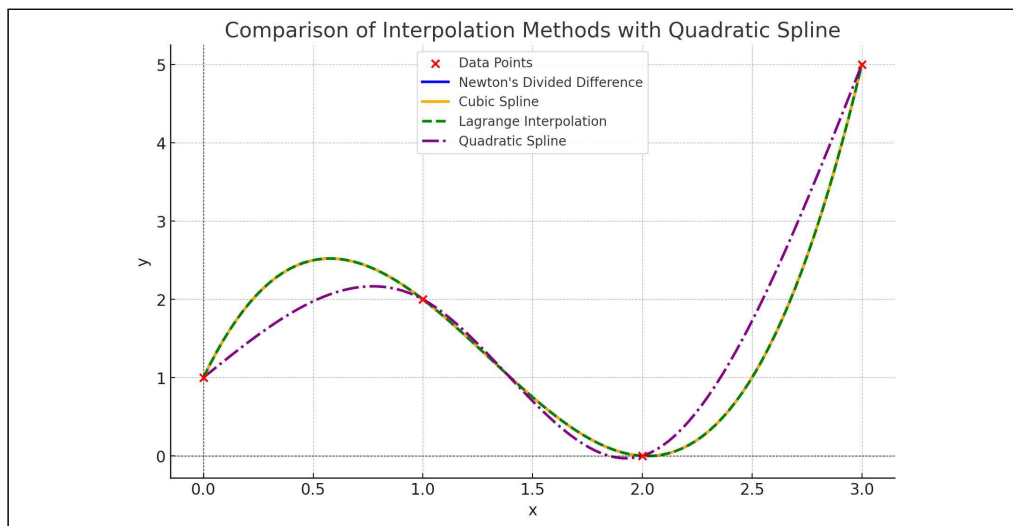
```

# Cubic Spline
plt.plot(x_new, y_spline, label="Cubic Spline", color='orange', linewidth=2)
# Lagrange
plt.plot(x_new, y_lagrange, label="Lagrange Interpolation", color='green',
linestyle='--', linewidth=2)
# Quadratic Spline
plt.plot(x_new, y_quadratic_spline, label="Quadratic Spline", color='purple',
linestyle='-.', linewidth=2)

# Configure plot
plt.title('Comparison of Interpolation Methods')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, color='black', linewidth=0.5, linestyle='--')
plt.axvline(0, color='black', linewidth=0.5, linestyle='--')
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.show()

```

② 코드 실행 결과



③ 코드 실행 결과 분석

위 그래프는 점 4개에 대해 네 가지 보간법(뉴턴의 차분법, 스플라인 보간법, 라그랑주 보간법, 2차 스플라인 보간법)을 비교한 결과이다. Quadratic Spline은 각 구간에 대해 독립적인 2차 다항식을 사용하여 보간하며, 곡선이 부드럽게 연결되도록 보장한다. 다른 방법들과 비교했을 때 Quadratic Spline은 국소적 보간 성능이 강점으로, 곡선의 연속성과 데이터의 지역적 특성

을 잘 반영한다. Lagrange 보간법은 Newton과 달리 계수 계산 방식이 다르며, 데이터가 많아질수록 계산 복잡도가 증가한다. Lagrange와 Newton 보간법은 점의 수가 많아질수록 고차 다항식이 되고, 이는 진동 문제를 유발할 가능성이 있다. Cubic Spline은 각 구간에서 독립적인 3차 다항식으로 보간한다. 함수 값, 1차 도함수, 2차 도함수가 모든 구간에서 연속되도록 설계된다. 데이터 포인트의 증가에 따라 국소적으로 적합한 곡선을 생성하므로 Runge 현상이 발생하지 않는다. 경계 조건(natural, clamped 등)을 통해 곡선의 형태를 제어할 수 있어 매우 유연하다. 이 세 가지 방법은 데이터의 분포와 목표에 따라 선택적으로 사용할 수 있다.

7 장 수치 회귀(Numerical Regression)

(7.1) 최소자승법에 의한 1차 다항식 근사

(7.2) 최소자승법에 의한 m 차 다항식 근사

(7.2.1) 최소자승법에 의한 m 차 다항식 근사

(7.2.2) np.polyfit 이용한 m 차 다항식 근사 소스코드

① 코드

```
import numpy as np
import matplotlib.pyplot as plt

# 주어진 데이터 포인트
x = np.array([1, 2, 3, 4, 5]) # x 좌표
y = np.array([1, 4, 9, 16, 25]) # y 좌표

# 1차, 2차, 3차 다항식 계수를 계산
coeff1 = np.polyfit(x, y, 1) # 1차 다항식
coeff2 = np.polyfit(x, y, 2) # 2차 다항식
coeff3 = np.polyfit(x, y, 3) # 3차 다항식

# 다항식을 이용해 y 값을 예측
poly1 = np.polyval(coeff1, x) # 1차 다항식으로 예측
poly2 = np.polyval(coeff2, x) # 2차 다항식으로 예측
poly3 = np.polyval(coeff3, x) # 3차 다항식으로 예측

# 데이터와 회귀 결과를 플롯
plt.figure(figsize=(8, 6))
plt.scatter(x, y, color='red', label='Data') # 데이터 포인트
```

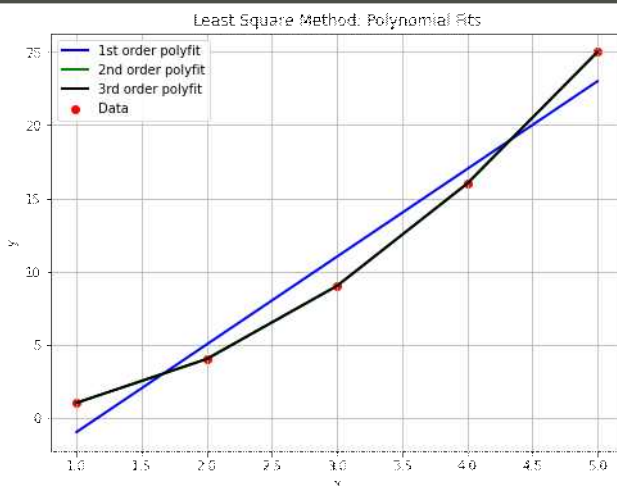
```
plt.plot(x, poly1, color='blue', label='1st order polyfit', linewidth=2) # 1차 근사
plt.plot(x, poly2, color='green', label='2nd order polyfit', linewidth=2) # 2차 근사
plt.plot(x, poly3, color='black', label='3rd order polyfit', linewidth=2) # 3차 근사

# 그래프 꾸미기
plt.xlabel('x')
plt.ylabel('y')
plt.title('Least Square Method: Polynomial Fits')
plt.legend()
plt.grid(True)

# 결과 출력
plt.show()
```

② 코드 실행 결과:

Least Square Method에 대한 결과로, 이 코드를 실행하면, 데이터 포인트와 함께 1차, 2차, 3차 다항식 회귀 모델을 비교할 수 있는 그래프가 생성된다.



(7.2.3) 행렬식을 이용한 m 차 다항식 근사 소스코드

이 코드는 2차 회귀분석(Nonlinear Regression)을 수행하여 데이터 포인트를 2차 다항식 형태로 근사하는 과정을 보여준다. 100개의 데이터 점을 $[-5, 10]$ 균등하게 생성하고, noise는 각 데이터 점에 추가될 잡음으로 랜덤하게 분포된 값으로 데이터를 더 현실적으로 만든다.

$y = -(10 + x + 2x^2)$ 와 잡음을 포함한 데이터 값이다. 회귀 계수 계산 공식 $\theta = (\Phi^T \Phi)^{-1} \Phi^T y$ 을 이용한다. 잡음이 섞인 데이터 포인트가 파란색 점으로 표시되고, 이들을 근사하는 부드러운 빨간색 곡선이 그려진다.

① 코드

```
# y = -(theta0 + theta1*x + theta2*x^2 + noise)
n = 100
x = -5 + 15*np.random.rand(n, 1)
noise = 10*np.random.randn(n, 1)

y = -( 10 + 1*x + 2*x**2 + noise)

A = np.hstack([x**0, x, x**2])
A = np.asmatrix(A)

theta = (A.T*A).I*A.T*y
print('theta:\n', theta)

xp = np.linspace(np.min(x), np.max(x))
yp = theta[0,0] + theta[1,0]*xp + theta[2,0]*xp**2

plt.figure(figsize = (10, 8))
plt.plot(x, y, 'o', markersize = 4, label = 'actual data')
plt.plot(xp, yp, 'r', linewidth = 2, label = 'regression: 2nd order')

plt.title('Nonlinear regression', fontsize = 15)
plt.xlabel('X', fontsize = 15)
plt.ylabel('Y', fontsize = 15)
plt.xlim([np.min(x), np.max(x)])
plt.grid(alpha = 0.3)
plt.legend(fontsize = 15)
plt.show()
```

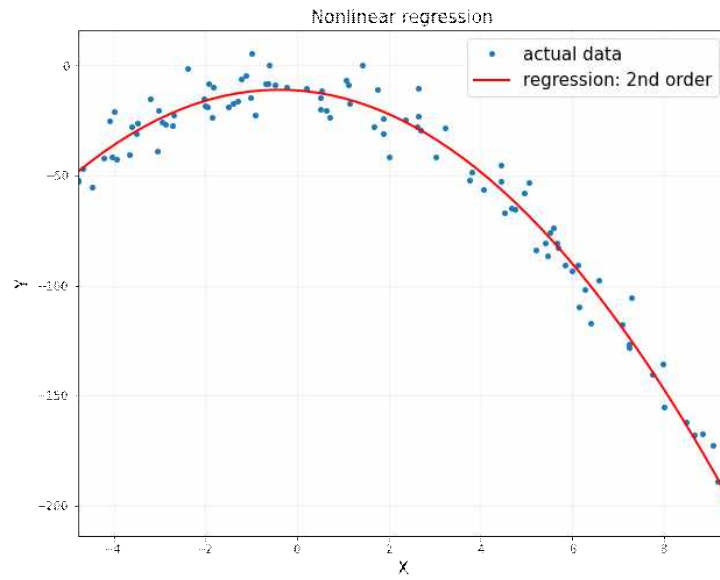
② 코드 실행 결과

```
theta:
[[-11.38941552]
```

7.3 연습문제

[-1.50205688]

[-1.93332106]]



8 장 비선형연립방정식 수치해법

(8.1) Newton-Raphson Method

(8.1.1) 비선형 연립방정식의 뉴턴-랩슨법(Newton-Raphson Method)

아래 다변수 함수를 근을 구하고자 한다. 근의 존재를 알기 위해 3차원으로 가시화 하여 보자.

$$\begin{aligned} (x)^2 + (y)^2 &= 4, \\ y &= 2x + 1 \\ f(x, y) &= (x)^2 + (y)^2 - 4, \\ g(x, y) &= y - 2x - 1 \end{aligned} \quad (8.1.1-7)$$

① 코드

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 첫 번째 함수 정의: 원 (circle)  $f(x, y) = x^2 + y^2 - 4$ 
def myCirc(x, y):
    return x**2 + y**2 - 4

# 두 번째 함수 정의: 쌍곡선 (hyperbolic)  $g(x, y) = y - 2x - 1$ 
def myHyp(x, y):
    return y - 2*x - 1

# 3D 그래프 생성
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)

# 계산된 Z 값
```

```

Z1 = myCirc(X, Y)
Z2 = myHyp(X, Y)

fig = plt.figure(figsize=(12, 6))

# 3D 서피스 플롯
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z1, cmap='viridis', alpha=0.7, edgecolor='none',
label='Circle')
ax1.plot_surface(X, Y, Z2, cmap='plasma', alpha=0.7, edgecolor='none',
label='Hyperbolic')
ax1.set_title("3D Plot of Circle and Hyperbolic")
ax1.set_xlabel("X-axis")
ax1.set_ylabel("Y-axis")
ax1.set_zlabel("Z-axis")

# 2D 암묵적 함수 플롯
ax2 = fig.add_subplot(122)
CS1 = ax2.contour(X, Y, Z1, levels=[0], colors='blue', linewidths=2,
linestyles='dashed', label="Circle")
CS2 = ax2.contour(X, Y, Z2, levels=[0], colors='red', linewidths=2,
linestyles='solid', label="Hyperbolic")
ax2.set_title("2D Implicit Plot of Circle and Hyperbolic")
ax2.set_xlabel("X-axis")
ax2.set_ylabel("Y-axis")
ax2.grid(True)

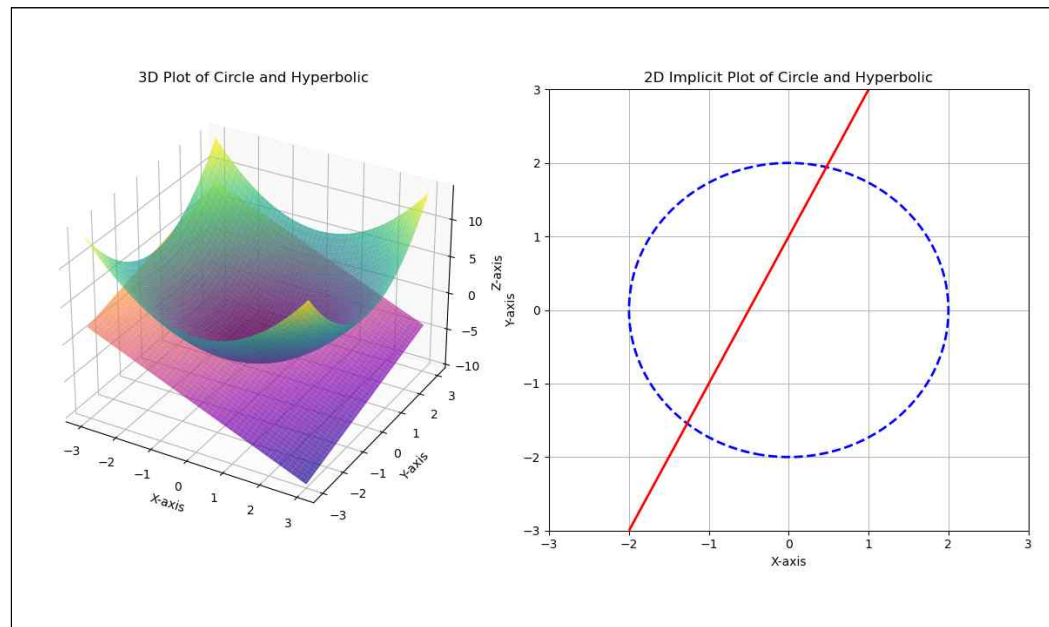
plt.tight_layout()
plt.show()

```

② 코드 실행 결과

아래 식은 간단히 식 2개, 미지수 2개의 벡터 함수 $\vec{x}$ 와 벡터 $\vec{f}$ 를 $\vec{f}(\vec{x}) = \vec{0}$ 로 나타낼 수 있는 수식, $(x)^2 + (y)^2 = 4$, $y = 2x + 1$ 의 결과를 그래프로 가시화 결과이다.

(8.2) 비선형연립방정식 수치해법



(8.2.1) 원과 직선이 만나는 점을 찾는 연립 비선형방정식

(8.2.2) 원과 직선이 만나는 점을 찾는 연립 비선형방정식

연립 비선형방정식과 최소값 문제를 풀기 위한 뉴턴법의 일반화에 대해 설명하고 다음 원과 직선이 만나는 점을 찾는 연립 비선형방정식이다. 한 쌍의 해를 초기치에따라 Newton-Raphson 방법을 사용해 비선형 방정식 시스템의 해를 찾고, 해당 방정식(원과 직선)의 시각화와 함께 수렴 경로 및 최종 해를 플롯하는 코드다.

$$\begin{aligned}(x)^2 + (y)^2 &= 1, \\ y &= 2x + 1\end{aligned}$$

$J = \begin{bmatrix} \partial f_1 / \partial x & \partial f_1 / \partial y \\ \partial f_2 / \partial x & \partial f_2 / \partial y \end{bmatrix} = \begin{bmatrix} 2(x) & 2(y) \\ -2 & 1 \end{bmatrix}$ 계산 과정을 코드로 나타내면 다음과 같다:

```
def j(x1, x2):
    j11 = 2.0 * x1 # ∂f1/∂x1
    j12 = 2.0 * x2 # ∂f1/∂x2
    j21 = -2.0     # ∂f2/∂x1
    j22 = 1.0      # ∂f2/∂x2
    return array([[j11, j12], [j21, j22]])
```

newton_nonlinear 함수는 Newton-Raphson 방법을 사용하여 비선형 방정식 시스템의 해를 반복

적으로 계산하는 기능을 한다. `newton_nonlinear` 함수는 Newton-Raphson 방법을 사용하여 두 개의 비선형 방정식 시스템을 해결하기 위한 알고리즘이다. 이 함수는 초기 추정값을 바탕으로 반복적으로 계산하여 방정식의 해를 점진적으로 구하며, 최종적으로 수렴된 해를 반환한다.

① 소스 코드

```
import matplotlib.pyplot as plt
from numpy import array, dot, zeros, linspace, meshgrid, set_printoptions
from numpy.linalg import inv

# 비선형 방정식을 풀기 위한 Newton-Raphson 방법
def newton_nonlinear(f1, f2, x10, x20, n, omega):
    x1 = zeros(n)
    x2 = zeros(n)
    x1[0] = x10
    x2[0] = x20
    for k in range(n - 1):
        f = array([f1(x1[k], x2[k]), f2(x1[k], x2[k])])
        dx1, dx2 = -omega * dot(inv(j(x1[k], x2[k])), f)
        x1[k + 1] = x1[k] + dx1
        x2[k + 1] = x2[k] + dx2
    return x1, x2

# 야코비안 행렬 (Jacobian matrix)
def j(x1, x2):
    j11 = 2.0 * x1
    j12 = 2.0 * x2
    j21 = -2.0
    j22 = 1.0
    return array([[j11, j12], [j21, j22]])

# 비선형 방정식
def f1(x1, x2):
    return x1**2 + x2**2 - 1.0 # 원 방정식

def f2(x1, x2):
    return -2 * x1 + x2 - 1.0 # 직선 방정식

# 매개변수 설정
```

```

try:
    x10 = float(input("x1의 초기 추정값을 입력하세요: "))
    x20 = float(input("x2의 초기 추정값을 입력하세요: "))
except ValueError:
    print("잘못된 입력! 기본값 x1 = 0.0, x2 = 2.0을 사용합니다.")
    x10 = 0.0
    x20 = 2.0

omega = 1.0 # 과보정 계수
n = 10      # 반복 횟수

# Newton-Raphson 방법을 사용하여 문제 해결
x1, x2 = newton_nonlinear(f1, f2, x10, x20, n, omega)

# 출력 시 소수점 자리 설정
set_printoptions(precision=4)

# 결과 출력
print("x1 = ", x1)
print("x2 = ", x2)
print(f"Converged Solution: x1 = {x1[-1]:.4f}, x2 = {x2[-1]:.4f}")

# 시각화를 위한 그리드 생성
x_vals = linspace(-1.5, 1.5, 300)
y_vals = linspace(-1.5, 1.5, 300)
X, Y = meshgrid(x_vals, y_vals)

# 그리드에서 f1과 f2 값 계산
F1 = f1(X, Y) # 원 방정식
F2 = f2(X, Y) # 직선 방정식

# Plot the solution and the equations
plt.figure(figsize=(6, 6))

# Contour plot for f1 (Circle)
plt.contour(X, Y, F1, levels=[0], colors='blue', linewidths=2, label='f1: Circle')

# Contour plot for f2 (Line)
plt.contour(X, Y, F2, levels=[0], colors='green', linewidths=2, label='f2: Line')

```

```

# Plot convergence path
plt.plot(x1, x2, label="Convergence Path", marker="o", linestyle="-", color="red")

# Highlight the converged solution
plt.scatter(x1[-1], x2[-1], color='orange', s=100, label=f"Solution: ({x1[-1]:.4f}, {x2[-1]:.4f})")

# Configure the plot
plt.title("Newton-Raphson Convergence with f1 and f2 Visualization", fontsize=15)
plt.xlabel("x1", fontsize=12)
plt.ylabel("x2", fontsize=12)
plt.axhline(0, color="black", linestyle="--", linewidth=0.5)
plt.axvline(0, color="black", linestyle="--", linewidth=0.5)
plt.grid(alpha=0.5)
plt.legend(fontsize=12)
plt.show()

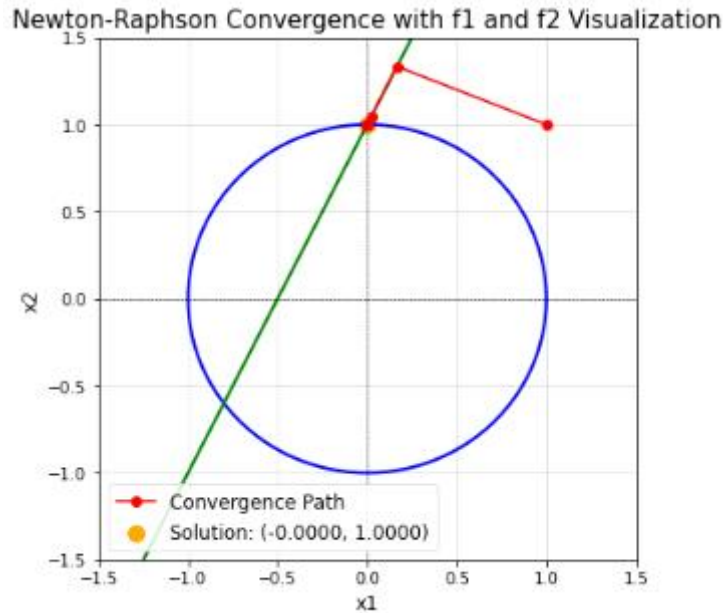
```

② 소스 코드 실행 결과 1:

Newton-Raphson 방법에서 초기치는 알고리즘의 성공과 수렴 속도에 중요한 영향을 미친다. 적절한 초기값을 설정하기 위해 다음 사항을 고려한다. 예를 들어 $f_1(x_1, x_2) = x_1^2 + x_2^2 - 1.0$ (원의 방정식)의 경우, 초기치는 반지름 1의 원 근처 값으로 설정하는 것이 적절하다. 그래프를 시각화하여 시각화하여 두 방정식의 교점 근처에서 초기값을 선택한다.

8.2 비선형연립방정식 수치해법

```
Enter initial guess for x1: 1
Enter initial guess for x2: 1
x1 = [ 1.0000e+00  1.6667e-01  2.4510e-02  7.0756e-04  6.2469e-07  4.8780e-13
      -3.6876e-17 -3.6876e-17 -3.6876e-17 -3.6876e-17]
x2 = [1.      1.3333 1.049  1.0014 1.      1.      1.      1.      1.      1.    ]
Converged solution: x1 = -0.0000, x2 = 1.0000
```



③ 소스 코드 실행 결과 2:

코드에서 사용자 입력을 통해 초기값을 설정한다.

```
try:
    x10 = float(input("x1의 초기 추정값을 입력하세요: "))
    x20 = float(input("x2의 초기 추정값을 입력하세요: "))
except ValueError:
    print("잘못된 입력! 기본값 x1 = 0.0, x2 = 2.0을 사용합니다.")
    x10 = 0.0
    x20 = 2.0
```

코드에서 키보드에 (-1,1)을 입력하면 3번 만에 수렴함을 알수 있다.

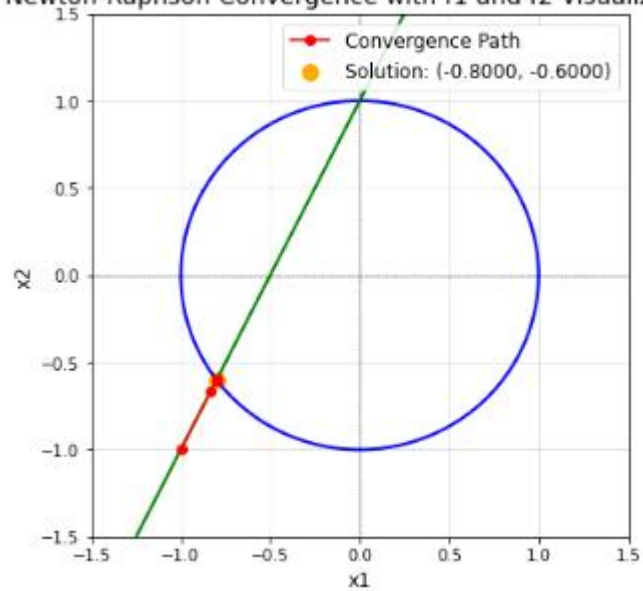
```
Enter initial guess for x1: -1
Enter initial guess for x2: -1
x1 = [-1.      -0.8333 -0.8013 -0.8      -0.8      -0.8      -0.8      -0.8      -0.8
      -0.8    ]
x2 = [-1.      -0.6667 -0.6026 -0.6      -0.6      -0.6      -0.6      -0.6      -0.6
```

8.2 비선형연립방정식 수치해법

-0.6]

Converged solution: $x_1 = -0.8000$, $x_2 = -0.6000$

Newton-Raphson Convergence with f1 and f2 Visualization



9 장 상미분 방정식 [초기치문제]

(9.1) 초기치문제 상미분 방정식 수치해석 개요

(9.2) Euler Method

(9.3) 2차 Runge-Kutta Method(RK2)

(9.4) 4차 Runge-Kutta Method(RK4)

(9.5) 고계 상미분방정식을 1계 연립 ODE 변환 방법

(9.5.1) mass-spring-damper계 1계 연립 ODE로 변환

(9.5.2) 파이썬을 이용한 RK4 프로그래밍

① 모듈을 정의하고 기능 불러오기

- ㉠ Numpy 모듈을 np 라는 함수로 명명.
- ㉡ 선형대수에 관련된 함수 사용을 위하여 Numpy 안에 numpy.linalg 안의 inv 모듈을 불러옴.
- ㉢ 그래프 출력을 위하여, matplotlib 안의 파이플롯을 불러오고, plt로 명명.

```
import numpy as np
from numpy.linalg import inv
from matplotlib import pyplot as plt
```

② $\vec{f}$ 를 정의

- ㉠ np. 라는 것은 도입부에서 넘파이 모듈의 사용을 정의할 때 'np'로 정의하였기 때문.
- ㉡ array([0.0,0.0])는 1x2 0행렬을 만들어 초기 벡터 공간을 정의.
- ㉢ if-else 조건문을 통해 t가 15 이하라면 행렬의 0번째 항을 $F_0 * \cos(\omega t)$ 로 정의고 return을 통해 벡터 F를 반환.

㉔ t가 15 이상이 되면 F행렬의 0번째 항을 0으로 초기화 시키고 벡터를 반환.

```
def F(t):
    F = np.array([0.0,0.0])
    if t <= 15:
        F[0] = F0 * np.cos(omega*t)
    else:
        F[0] = 0.0
    return F
```

③ 룽게-쿠타 방법의 Python 프로그래밍 구현

㉕ $G(y,t)$ 함수는 $A^{-1}(f(t) - By)$ 를 반환하도록 정의됨.

㉖ $RK4_step(y, t, dt)$ 는 룽게-쿠타 방법의 핵심인 증분값에 대한 부분임.

㉗ $(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4)$ 을 계산하기 위한 함수 $RK4_step(y, t, dt)$ 가 정의.

$$\begin{aligned}
 \vec{k}_1 &= f(t_n, \vec{y}_n) \\
 \vec{k}_2 &= f\left(t_n + \frac{1}{2}h, \vec{y}_n + \frac{1}{2}h\vec{k}_1\right) \\
 \vec{k}_3 &= f\left(t_n + \frac{1}{2}h, \vec{y}_n + \frac{1}{2}h\vec{k}_2\right) \\
 \vec{k}_4 &= f(t_n + h, \vec{y}_n + h\vec{k}_3)
 \end{aligned}
 \tag{9.5.3-1}$$

이론적인 아래의 식들과 계산하고 return 함.

$$\begin{aligned}
 \vec{y}_{n+1} &= \vec{y}_n + \frac{1}{6}h(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4) \\
 t_{n+1} &= t_n + h
 \end{aligned}
 \tag{9.5.3-2}$$

```
def G(y,t):
    return A_inv.dot( F(t) - B.dot(y) )

def RK4_step(y, t, dt):
    k1 = G(y,t)
    k2 = G(y+0.5*k1*dt, t+0.5*dt)
    k3 = G(y+0.5*k2*dt, t+0.5*dt)
    k4 = G(y+k3*dt, t+dt)

    return dt * (k1 + 2*k2 + 2*k3 + k4) /6
```

④ 주요 변수 정의

- ㉠ m, k, c는 질량, 스프링, 댐퍼 값을 의미함.
- ㉡ F0는 초기 가진, delta_t는 시간간격, omega는 가진 주파수, time은 np.arrange를 활용하여 0부터 특정시간(위의 경우 40)까지 delta_t 단위로 행렬 배열시킨 것임.
- ㉢ y = np.array([0,1]) 의 [] 속 값들은 초기 속도와 위치를 의미.

```
m = 2.0
k = 1.0
c = 0.5

# critical damping = 2 * SQRT(m*k) = 4.0 , c가 0이면 계
속 진동

F0 = 0.0
delta_t = 0.1
omega = 1.0
time = np.arange(0.0, 40.0, delta_t)

y = np.array([0,1]) # [velocity, displacement]
```

⑤ 행렬 A, B 정의, 역행렬과 Y, force 벡터 정의

- ㉠ A와 B는 상태방정식의 A행렬, B행렬을 의미하며 kovw.net 자료와는 살짝 다르게 정의한다. 이는 변수의 위치를 아래위로 바꾸어 놓은 것으로 결과에는 차이가 없다.
- ㉡ A_inv = inv(A) 는 단순히 역행렬을 정의하기 위한 함수를 만든 것임.

$$\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} y_2' \\ y_1' \end{pmatrix} + \begin{pmatrix} c & k \\ -1 & 0 \end{pmatrix} \begin{pmatrix} y_2 \\ y_1 \end{pmatrix} = \begin{pmatrix} f(t) \\ 0 \end{pmatrix} \quad (9.5.3-3)$$

$[A]y' + [B]y = f(t) \rightarrow \begin{pmatrix} y_2' \\ y_1' \end{pmatrix} = A^{-1}(f(t) - By)$ 를 통해 구한다.

- ㉔ $Y = []$, $\text{force} = []$ 의 []는 Y 와 force 값을 받을 비어있는 벡터를 만들어 놓은 것으로, 코딩 편의를 위해 초기 행렬 공간을 정의하는 개념이라고 보면 됨.

```
A = np.array([[m,0],[0,1]])
```

```
B = np.array([[c,k],[-1,0]])
```

```
A_inv = inv(A)
```

```
Y = []
```

```
force = []
```

- ㉔ $G(y,t)$: $\text{return } A_inv.\text{dot}(F(t) - B.\text{dot}(y))$ 로 정의된 함수를 반복 출력하여 Standard Form으로 변형된 외부 가진력에 해당하는 많은양의 연산을 시도하기 위함임.

```
def G(y,t):
```

```
    return A_inv.dot( F(t) - B.dot(y) )
```

⑥ 에너지 총합(기계적 에너지와 포텐셜 에너지)과 진동 해 출력하기

- ㉔ for t in time 의 경우 변수 t 를 위에서 정의한 행렬 time의 원소로 정의하면서 반복하는 것이다. 예를 들어 time 행렬이 (1, 2, 3, 7, 5)라면, t 를 순서대로 1, 2, 3, 7, 5로 정의하며 특정 함수를 반복 출력하게끔 활용 할 수 있다.
- ㉔ for 반복문 속에는 $y = y + \text{RK4_step}(y, t, \text{delta_t})$ 를 통해 y 값을 새로 정의해주는 부분과 벡터 Y 와 force 에 각각 y , $F(t)$ 값을 0번째와 1번째 항에 넣어주는 부분이 정의 되어있습니다. 또한 기계적 에너지와 포텐셜 에너지를 계산해주는 KE, PE 변수를 정의한 부분이 있습니다. 단순 계산식이니 설명은 하지 않겠습니다.
- ㉔ if $t \% 1 \leq 0.01$: print ('Total Energy:', KE+PE) 의 경우는 if 조건문으로써 1을 t 로 나눈 나머지 값이 0.01 이하가 되면 총 에너지 량인 KE + PE 값을 출력해주는 조건문입니다.

```

for t in time:
    y = y + RK4_step(y, t, delta_t) # y값 뽑고 다시

    Y.append(y[1])
    force.append(F(t)[0])

    KE = 0.5 * m * y[0]**2 #기계적 에너지
    PE = 0.5 * k * y[1]**2 #포텐셜 에너지

    if t % 1 <= 0.01:
        print ('Total Energy:', KE+PE)

```

⑦ 그래프 출력

⑦ # matplotlib 라이브러리 모듈의 그래프 출력기능인 plot을 통해 시간-Y(변위), 시간-force(힘) 그래프를 출력한다. 모든 선을 같이 출력하는 grid, 선의 이름을 설정하는 legend, 그리고 이 모든 것을 화면에 출력하는 show를 통해 그래프를 화면에 띄우고 끝난다.

⑧ 코드

```

import numpy as np
import matplotlib.pyplot as plt

# 상수와 초기 조건
m = float(input("mass 값을 입력하세요(0.01): "))
c = float(input("damping 값을 입력하세요(0.25): "))
k = float(input("spring 값을 입력하세요(0.069): "))

print("f(t)=Asin(wt) 입니다")
Amp = float(input("A 값을 입력하세요(25.0): "))
omega = float(input("omega 값을 입력하세요(25.0): "))

dt = float(input("시간 간격 dt 값을 입력하세요(기본값 0.001): ") or 0.001)
time_max = 30.0

# Runge-Kutta 4th Order 메소드 정의

```

```

def rk4(dt, y1, y2, time):
    def ft1(y1, y2, time):
        return y2

    def ft2(y1, y2, time):
        return (-c * y2 - k * y1 + Amp * np.sin(omega * time)) / m

    k1 = dt * np.array([ft1(y1, y2, time), ft2(y1, y2, time)])
    k2 = dt * np.array([ft1(y1 + 0.5 * k1[0], y2 + 0.5 * k1[1], time), ft2(y1 + 0.5 *
k1[0], y2 + 0.5 * k1[1], time)])
    k3 = dt * np.array([ft1(y1 + 0.5 * k2[0], y2 + 0.5 * k2[1], time), ft2(y1 + 0.5 *
k2[0], y2 + 0.5 * k2[1], time)])
    k4 = dt * np.array([ft1(y1 + k3[0], y2 + k3[1], time), ft2(y1 + k3[0], y2 + k3[1],
time)])
    return np.array([y1, y2]) + (k1 + 2 * k2 + 2 * k3 + k4) / 6

# Explicit Euler 메소드 정의
def euler(dt, y1, y2, time):
    def ft1(y1, y2, time):
        return y2

    def ft2(y1, y2, time):
        return (-c * y2 - k * y1 + Amp * np.sin(omega * time)) / m

    dy1 = dt * ft1(y1, y2, time)
    dy2 = dt * ft2(y1, y2, time)
    return y1 + dy1, y2 + dy2

# 데이터 저장을 위한 배열
times = []
y1_euler = []
y1_rk2 = []
y1_rk4 = []

# 초기 조건 설정
time = 0.0
y_0 = float(input("초기변위 y(0)=?: "))
y1_e = y1_r2 = y1_r4 = y_0
y_dot = float(input("초기속도 y_dot(0)=?: "))
y2_e = y2_r2 = y2_r4 = y_dot

```

```

# 사용자 선택에 따라 계산 방법 설정
param = int(input("Enter 1 for Euler, 2 for RK2, 3 for RK4, or 4 for all: "))

while time <= time_max:
    if param == 1 or param == 4:
        y1_e, y2_e = euler(dtime, y1_e, y2_e, time)
        if param == 1:
            y1_euler.append(y1_e)

    if param == 2 or param == 4:
        k1_rk2 = dtime * np.array([y2_r2, (-c * y2_r2 - k * y1_r2 + Amp *
np.sin(omega * time)) / m])
        k2_rk2 = dtime * np.array([y2_r2 + 0.5 * k1_rk2[1], (-c * (y2_r2 + 0.5 *
k1_rk2[1]) - k * (y1_r2 + 0.5 * k1_rk2[0]) + Amp * np.sin(omega * (time + dtime /
2))) / m])
        y1_r2 += (k1_rk2[0] + k2_rk2[0]) / 2
        y2_r2 += (k1_rk2[1] + k2_rk2[1]) / 2
        if param == 2:
            y1_rk2.append(y1_r2)

    if param == 3 or param == 4:
        y1_r4, y2_r4 = rk4(dtime, y1_r4, y2_r4, time)
        if param == 3:
            y1_rk4.append(y1_r4)

    if param == 4:
        y1_euler.append(y1_e)
        y1_rk2.append(y1_r2)
        y1_rk4.append(y1_r4)

    times.append(time)
    time += dtime

# 결과 시각화
plt.figure(figsize=(10, 6))
if param == 1:
    plt.plot(times, y1_euler, label='Euler')
elif param == 2:
    plt.plot(times, y1_rk2, label='RK2')

```

```

elif param == 3:
    plt.plot(times, y1_rk4, label='RK4')
elif param == 4:
    plt.plot(times, y1_euler, label='Euler')
    plt.plot(times, y1_rk2, label='RK2')
    plt.plot(times, y1_rk4, label='RK4')

plt.xlabel('Time(sec)')
plt.ylabel('Displacement(m)')

# 동적 제목 설정
title = f'mass-spring-damper: m={m}, c={c}, k={k}, omega={omega}'
plt.title(title)

plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

```

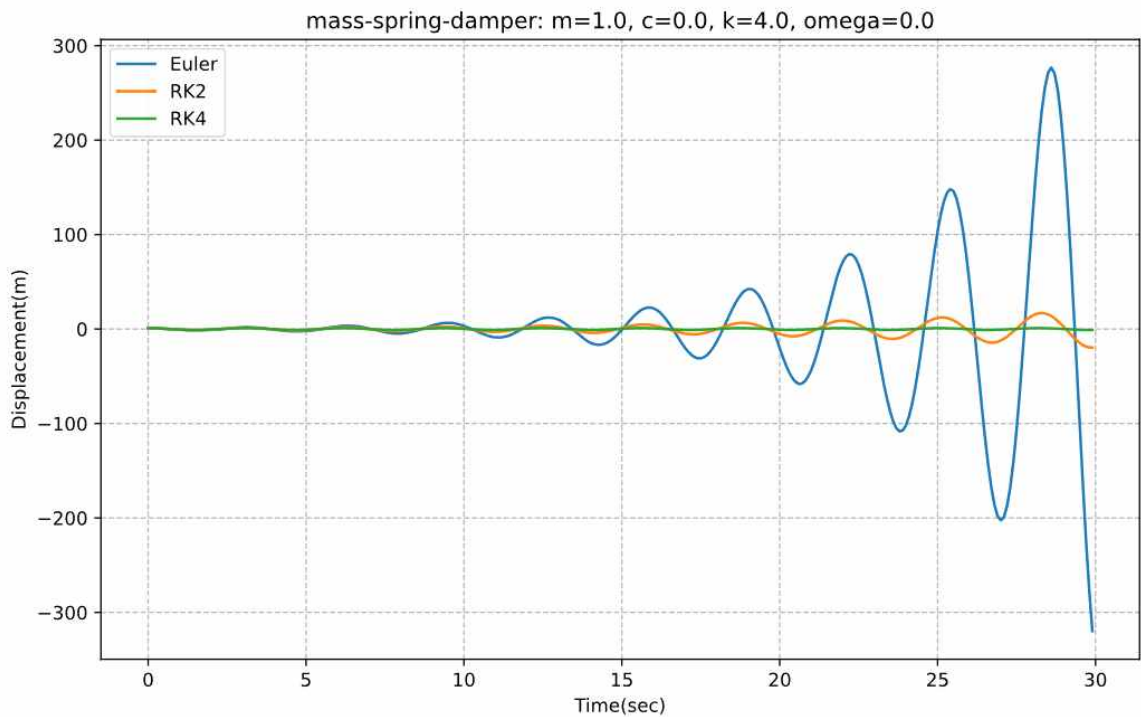
⑨ Euler, RK2, RK4 프로그래밍 테스트 및 결과 비교

외력이 영이고, c 값이 0인 경우, 즉 소스코드의 변수 설정 부분 중 질량, 스프링, 댐퍼값을 $m = 1.0$, $k = 4.0$, $c = 0.0$ 로 했을 때 결과이다. 댐퍼값이 0이기에 변위가 0으로 감쇄 지동하지 않고 진동하는 것을 볼 수 있다. 이론해는 진동 감쇄나 증폭없이 진폭이 일정하게 유지 되어야 하나 RK4 이외 다른 방법은 오차가 누적되고 있음을 알수 있다.

```

mass 값을 입력하세요(0.01): 1
damping 값을 입력하세요(0.25): 0
spring 값을 입력하세요(0.069): 4
f(t)=Asin(wt) 입니다
A 값을 입력하세요(25.0): 0
omega 값을 입력하세요(25.0): 0
시간 간격 dtime 값을 입력하세요(기본값 0.001): 0.1
초기변위 y(0)=?: 1
초기속도 y_dot(0)=?: 0
Enter 1 for Euler, 2 for RK2, 3 for RK4, or 4 for all: 4

```



그 외 본 프로그램 소스를 사용하여 변수값에 변화를 주면서 실행한 결과 질량, 스프링, 댐퍼 시스템의 진동 해석을 적절히 해주는 것을 볼 수 있다. spring-mass-damper 시스템의 초기값 문제에 적용해보고 직접 소스코드를 한 결과 RK4 결과가 오일러 법에 비해 매우 우수함을 알 수 있다. 그 외 공진도 외력, $f(t)=A\sin(\omega t)$ 을 공진 주파수로 맞추어 테스트 해 보면 공진 현상을 이해 할 수 있다.

(9.6) 미분-적분 방정식 수치기법

(9.6.1) Differential-Integral Equation Solution (Euler, RK2, RK4)

① 코드

```
import numpy as np
import matplotlib.pyplot as plt

# 시간 설정
t0, tf = 0, 5 # 초기 및 종료 시간
dt = 0.5 # 시간 간격
t = np.arange(t0, tf + dt, dt) # 시간 배열
```

```

# 초기 조건
y_euler = np.zeros(len(t))
y_rk2 = np.zeros(len(t))
y_rk4 = np.zeros(len(t))
y_euler[0] = 1 # Euler 초기 조건
y_rk2[0] = 1 # RK2 초기 조건
y_rk4[0] = 1 # RK4 초기 조건

# 적분항 초기화
integral = np.zeros(len(t))

# 방정식 정의
def f(t, y, integral):
    return -y + integral

# Euler, RK2 및 RK4 선택적으로 사용
def solve_differential_integral(param):
    global y_euler, y_rk2, y_rk4

    for i in range(1, len(t)):
        # 적분항 계산 (누적합)
        integral[i] = integral[i-1] + np.exp(-t[i-1]) * dt

        if param == 4 or param == 1: # Euler 방법
            y_euler[i] = y_euler[i-1] + dt * f(t[i-1], y_euler[i-1], integral[i-1])

        if param == 4 or param == 2: # RK2 사용
            k1_rk2 = f(t[i-1], y_rk2[i-1], integral[i-1])
            k2_rk2 = f(t[i-1] + dt, y_rk2[i-1] + dt * k1_rk2, integral[i-1])
            y_rk2[i] = y_rk2[i-1] + (dt / 2) * (k1_rk2 + k2_rk2)

        if param == 4 or param == 3: # RK4 사용
            k1_rk4 = f(t[i-1], y_rk4[i-1], integral[i-1])
            k2_rk4 = f(t[i-1] + dt / 2, y_rk4[i-1] + dt / 2 * k1_rk4, integral[i-1])
            k3_rk4 = f(t[i-1] + dt / 2, y_rk4[i-1] + dt / 2 * k2_rk4, integral[i-1])
            k4_rk4 = f(t[i-1] + dt, y_rk4[i-1] + dt * k3_rk4, integral[i-1])
            y_rk4[i] = y_rk4[i-1] + (dt / 6) * (k1_rk4 + 2 * k2_rk4 + 2 * k3_rk4 +
k4_rk4)

```

```

    if param not in [1, 2, 3, 4]:
        raise ValueError("Invalid parameter: Use param=1 for Euler, param=2 for RK2,
param=3 for RK4, param=4 for all methods")

# 사용자 입력에 따라 Euler, RK2, RK4, 또는 모두 실행
param = int(input("Enter 1 for Euler, 2 for RK2, 3 for RK4, or 4 for all: "))
solve_differential_integral(param)

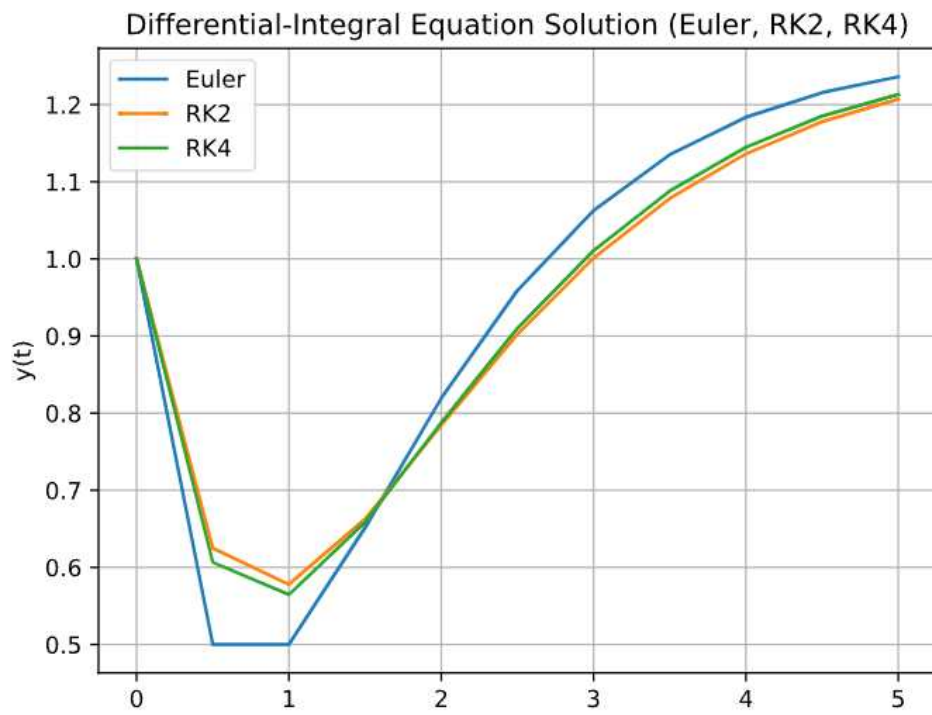
# 결과 시각화
if param == 1:
    plt.plot(t, y_euler, label="Euler")
elif param == 2:
    plt.plot(t, y_rk2, label="RK2")
elif param == 3:
    plt.plot(t, y_rk4, label="RK4")
elif param == 4:
    plt.plot(t, y_euler, label="Euler")
    plt.plot(t, y_rk2, label="RK2")
    plt.plot(t, y_rk4, label="RK4")

plt.xlabel("Time (t)")
plt.ylabel("y(t)")
plt.title("Differential-Integral Equation Solution (Euler, RK2, RK4)")
plt.legend()
plt.grid()
plt.show()

```

② 실행 결과

Enter 1 for Euler, 2 for RK2, 3 for RK4, or 4 for all: 4



10 장 Shooting Method

[경계치 문제]

(10.1) 선형 Shooting Method

(10.1.1) Explicit Euler Method

(10.1.2) Explicit Euler Method 예제

이 파이썬 코드는 2차 경계값 문제(Boundary Value Problem, BVP)를 해결하기 위해 Explicit Forward Euler 방법을 사용한다. 방정식은 아래와 같다. BC, $BC: y(0) = 3, y(3) = 1$ 을 IC, $IC: y(0) = 3, \frac{dy}{dx}(0) = 0$ 로 가정하여 푼다. $BC: y(0) = 3, y(3) = 1$ 로 출발하여 초기치 문제로 풀어 끝단 경계 조건이 만족하는지 확인해 본다.

$$\frac{d^2y}{dx^2} + 3\frac{dy}{dx} + 1y = 0, \quad (10.1.2-1)$$

$$BC: y(0) = 3, y(3) = 1$$

Explicit Forward Euler 방법:

$$\begin{pmatrix} y_1^{n+1} \\ y_2^{n+1} \end{pmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \Delta x \begin{bmatrix} 0 & 1 \\ -A & -B \end{bmatrix} \begin{pmatrix} y_1^n \\ y_2^n \end{pmatrix} \quad (10.1.2-2)$$

로 수치적 해를 계산한다. 결과적으로 $y(x)$ 와 $\frac{dy}{dx}$ 의 그래프를 통해 경계값 문제의 해를 시각화한다.

① Explicit Euler Method 코드

```
import numpy as np
import matplotlib.pyplot as plt

# 계수 A와 B 정의
A = 3
```

```
B = 1

# 행렬 A_ 정의
A_ = np.array([[0, 1],
               [-A, -B]])

# 스텝 크기와 x의 범위 정의
dx = 0.0005
x_end = 3

# 초기 조건  $y(0) = 3$ ,  $dy/dx(0) = 0$ 
y0 = np.array([3, 0])

# Explicit Forward Euler
n_steps = int(x_end / dx) + 1 # 스텝 수 계산
x = np.zeros(n_steps)         # x 배열 초기화
y = np.zeros((2, n_steps))    # y 배열 초기화
y[:, 0] = y0                  # 초기값 설정

# Forward Euler 반복
for n in range(n_steps - 1):
    x[n + 1] = x[n] + dx
    y[:, n + 1] = (np.eye(2) + dx * A_) @ y[:, n]

# y(x) 그래프 그리기
plt.figure()
plt.plot(x, y[0, :], label='y(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('y(x) vs x')
plt.grid()
plt.legend()

# dy/dx 그래프 그리기
plt.figure()
plt.plot(x, y[1, :], label="dy/dx", color='orange')
plt.xlabel('x')
plt.ylabel('dy/dx')
plt.title('dy/dx vs x')
plt.grid()
```

```
plt.legend()
```

```
plt.show()
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 계수 A와 B 정의
```

```
A = 3
```

```
B = 1
```

```
# 행렬 A_ 정의
```

```
A_ = np.array([[0, 1],  
               [-A, -B]])
```

```
# 스텝 크기와 x의 범위 정의
```

```
dx = 0.0005
```

```
x_end = 3
```

```
# 초기 조건  $y(0) = 3$ ,  $dy/dx(0) = 0$ 
```

```
y0 = np.array([3, 0])
```

```
# Explicit Forward Euler
```

```
n_steps = int(x_end / dx) + 1 # 스텝 수 계산
```

```
x = np.zeros(n_steps)         # x 배열 초기화
```

```
y = np.zeros((2, n_steps))    # y 배열 초기화
```

```
y[:, 0] = y0                  # 초기값 설정
```

```
# Forward Euler 반복
```

```
for n in range(n_steps - 1):
```

```
    x[n + 1] = x[n] + dx
```

```
    y[:, n + 1] = (np.eye(2) + dx * A_) @ y[:, n]
```

```
#  $y(x)$  그래프 그리기
```

```
plt.figure()
```

```
plt.plot(x, y[0, :], label='y(x)')
```

```
plt.xlabel('x')
```

```
plt.ylabel('y')
```

```

plt.title('y(x) vs x')
plt.grid()
plt.legend()

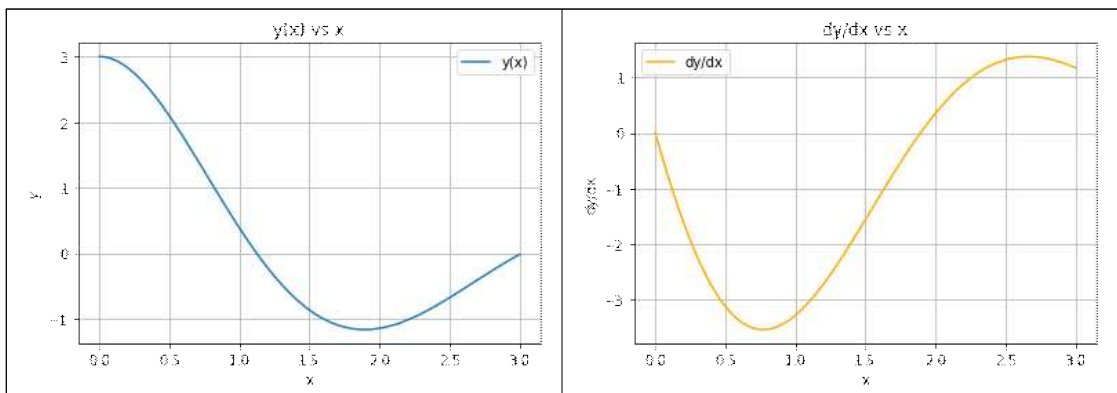
# dy/dx 그래프 그리기
plt.figure()
plt.plot(x, y[1, :], label="dy/dx", color='orange')
plt.xlabel('x')
plt.ylabel('dy/dx')
plt.title('dy/dx vs x')
plt.grid()
plt.legend()

plt.show()

```

② 1번째 초기 조건:

Explicit Euler Method : 프로그램 결과 끝단에서 경계조건과 차이가 난다.

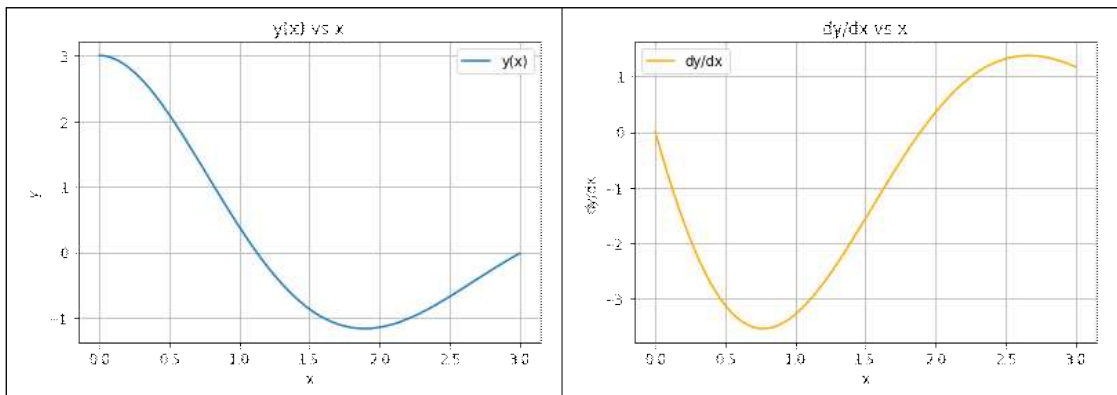


③ 2번째 초기 조건:

$IC: y(0) = 3, \quad \frac{dy}{dx}(0) = -0.5$ 로 출발하여 초기치 문제로 풀어 끝단 경계 조건이 어떻게 되는지 확인해 본다.

④ 3번째 초기 조건 :

10.1 선형 Shooting Method



3번째 초기 조건을 1번째, 2번째 결과로부터 보간하여 결정한다. 주어진 경계조건은

$BC: y(0) = 3, y(3) = 1$ 이다. 첫 번째 초기 추정은 $\dot{y}(0) = 0$ 에서

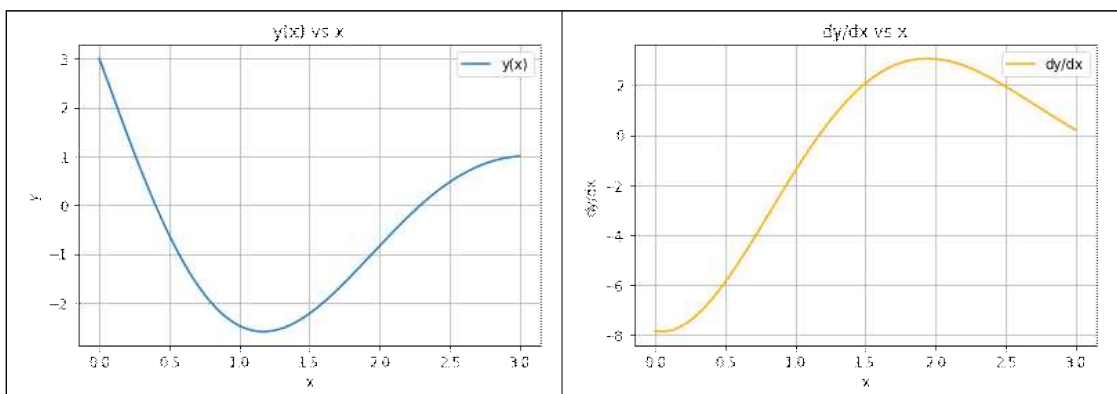
$y(3) = -0.0209$ 이고, $\dot{y}(0) = -0.5$ 에서 $y(3) = 0.0442$ 이다. 보간법을 이용해 $\dot{y}(0)$ 의 값을 업데이트:

$$f(x) = f(a) + \frac{f(b) - f(a)}{b - a}(x - a)$$

$$1 = -0.0209 + \frac{0.0442 - (-0.0209)}{-0.5 - 0}(x - 0)$$

보간법 적용 결과: $\dot{y}(0) = -7.841$

⑤ Explicit Euler Method : 올바른 결과가 나왔다.



(10.2) 비선형 Shooting Method

$$f''' + ff'' = 0 \quad (10.2-1)$$

Blasius 방정식의 경계 조건은 다음과 같다.

$$f(0) = 0, \quad f'(0) = 0, \quad f'(\infty) = 1 \quad (10.2-2)$$

이 방정식은 3개의 1차 미분 방정식으로 변환된다:

$$y_1' = y_2, \quad y_2' = y_3, \quad y_3' = -y_1 y_3, \quad (10.2-3)$$

위 식을 초기값 문제(IVP)로 변환하여 다음과 같은 초기 조건을 정의한다:

$$y_1(0) = 0, \quad y_2(0) = 0, \quad y_3(0) = guess \quad (10.2-4)$$

$y_3(0) = guess$ 로 출발하여 $y_2(\infty) = 1$ 이 되도록 조정한다.

① 코드

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

# Blasius 방정식 미분 방정식 정의
def blasius(eta, y):
    return [y[1], y[2], -y[0] * y[2]]

# 목표 경계 조건
target = 1.0

# 초기 추정값 및 결과 저장 배열
guesses = [1, 0.1] # 초기 추정값 두 개
solutions = []

# 첫 번째 추정값 계산
sol1 = solve_ivp(blasius, [0, 10], [0, 0, guesses[0]], t_eval=np.linspace(0, 10, 200))
solutions.append(sol1.y[1, -1])

# 두 번째 추정값 계산
```

```

sol2 = solve_ivp(blasius, [0, 10], [0, 0, guesses[1]], t_eval=np.linspace(0, 10, 200))
solutions.append(sol2.y[1, -1])

# 선형 보간으로 새로운 추정값 계산
m = (guesses[0] - guesses[1]) / (solutions[0] - solutions[1])
guess3 = guesses[1] + m * (target - solutions[1])

# 새로운 추정값으로 계산
sol3 = solve_ivp(blasius, [0, 10], [0, 0, guess3], t_eval=np.linspace(0, 10, 200))
solution3 = sol3.y[1, -1]

# 결과 출력
print("추정값과 결과")
print(f"추정값 1: {guesses[0]:.4f}, 결과: {solutions[0]:.4f}")
print(f"추정값 2: {guesses[1]:.4f}, 결과: {solutions[1]:.4f}")
print(f"추정값 3: {guess3:.4f}, 결과: {solution3:.4f}")
print(f"목표값: {target:.3f}")

# 결과 시각화
plt.figure(figsize=(8, 5))

# f(η) 플롯
plt.plot(sol1.t, sol1.y[0], label=f"f(η) guess1={guesses[0]:.4f}")
plt.plot(sol2.t, sol2.y[0], label=f"f(η) guess2={guesses[1]:.4f}")
plt.plot(sol3.t, sol3.y[0], label=f"f(η) guess3={guess3:.4f}")

# f'(η) 플롯
plt.plot(sol1.t, sol1.y[1], linestyle="--", label=f"f'(η) guess1={guesses[0]:.4f}")
plt.plot(sol2.t, sol2.y[1], linestyle="--", label=f"f'(η) guess2={guesses[1]:.4f}")
plt.plot(sol3.t, sol3.y[1], linestyle="--", label=f"f'(η) guess3={guess3:.4f}")

plt.xlabel("η")
plt.ylabel("f(η) , f'(η)")
plt.title("Blasius solutions")
plt.legend()
plt.grid()
plt.show()

```

② 코드 설명

Blasius 방정식은 $f''' + ff'' = 0$ 으로 정의되며, 이를 1차 미분 방정식 시스템으로 변환하여 이 방정식을 3개의 1차 미분 방정식, $y_1' = y_2$, $y_2' = y_3$, $y_3' = -y_1y_3$, 으로 변환된다

```
def blasius(eta, y):
    return [y[1], y[2], -y[0] * y[2]]
```

초기 추정값 설정: $f''(0)$ 에 대한 초기 추정값을 설정한다. 첫 번째와 두 번째 추정값으로 각각 1과 0.1을 사용한다.

```
guesses = [1, 0.1]
```

첫 번째와 두 번째 추정값 풀이: 각 초기 추정값에 대해 IVP를 풀이하여 $f'(\infty) \simeq f'(10)$ 를 계산. $f'(\infty) \simeq f'(10)$ 값을 solutions 리스트에 저장.

```
sol1 = solve_ivp(blasius, [0, 10], [0, 0, guesses[0]], t_eval=np.linspace(0, 10, 200))
solutions.append(sol1.y[1, -1])
```

선형 보간법: 두 추정값과 결과를 기반으로 선형 보간을 수행하여 $f''(0)$ 의 새로운 추정값 guess3를 계산.

```
m = (guesses[0] - guesses[1]) / (solutions[0] - solutions[1])
guess3 = guesses[1] + m * (target - solutions[1])
```

결과 출력: 각 추정값과 $f'(10)$ 값을 출력하여 경계 조건을 만족하는지 확인.

```
print(f"추정값 1: {guesses[0]:.4f}, 결과: {solutions[0]:.4f}")
```

결과 시각화: 각 추정값에 대해 계산된 $f(\eta), f'(\eta)$ 를 플롯한다.

```
plt.plot(sol1.t, sol1.y[0], label=f"f(η) guess1={guesses[0]:.4f}")
plt.plot(sol1.t, sol1.y[1], linestyle="--", label=f"f'(η) guess1={guesses[0]:.4f}")
```

③ 코드실행 및 출력 내용:

각 추정값에 대해 $f'(0), f'(10)$ 값이 출력된다.

그래프에서: 주어진 초기 추정값에서의 속도 프로파일, 경계층 내 속도의 변화율 $f(\eta), f'(\eta)$ 플롯.

④ 코드 실행 결과

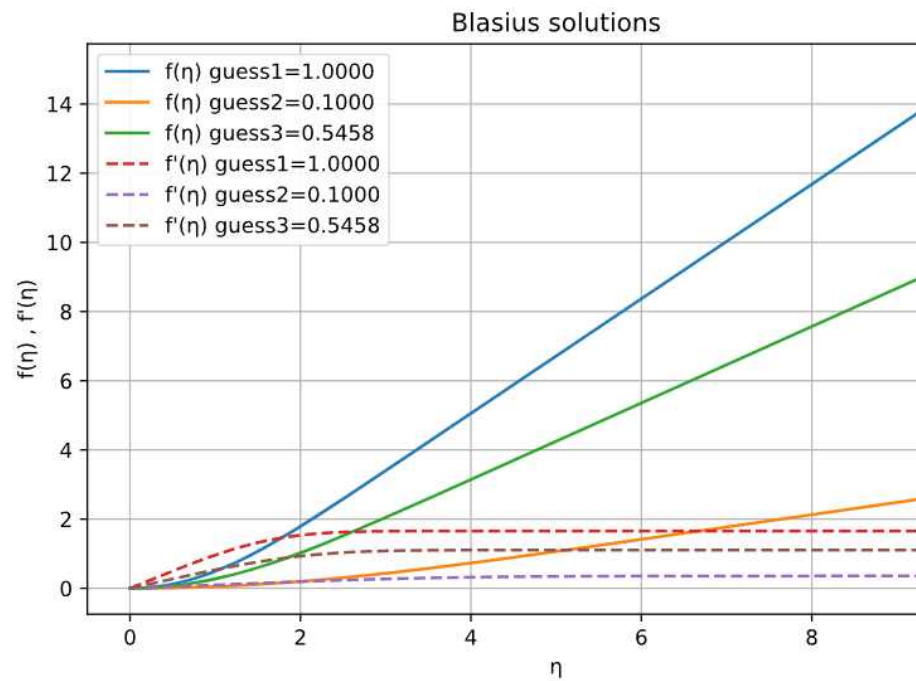
추정값과 결과

추정값 1: 1.0000, 결과: 1.6554

추정값 2: 0.1000, 결과: 0.3567

추정값 3: 0.5458, 결과: 1.1056

목표값: 1.000



11 장 유한차분법

(11.1) 편미분 방정식 개요

(11.2) 유한차분법의 수치해석

(11.2.1) 유한차분법(Finite Difference Method, FDM) 개요

(11.2.2) 1차원 비정상 열전달(확산) 방정식의 유한차분법

(11.2.3) Explicit Euler Method (명시적 오일러법)

(11.2.4) Implicit Euler Method (암시적 오일러법)

(11.2.5) Crank-Nicholson 법

(11.2.6) 기타 중요 사항

(11.2.7) 1차원 정상 열전달 FDM 수치해석

식(11.2.1-2)에서 2차원 포아송 방정식을 1차원으로 가정하여 프로그램 한다. 비정상 항이 없으므로 1차원 비정상 열전달(확산) 방정식의 유한차분법에서 비정상 항을 제거하면 된다.

① 소스 코드

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# 문제 정의
def function_f(x):
    return np.exp(x)

def exact_solution(x):
    return np.exp(x)

# FDM 계산 함수
def fdm_solver(n, a, b):
    h = (b - a) / (n - 1) # 격자 간격
    x = np.linspace(a, b, n) # x 좌표 생성

    # 초기화
    A = np.zeros((n, n)) # 계수 행렬
    F = np.zeros(n)      # 외부 힘 벡터

    # 계수 행렬 A 설정 (2차 중앙 차분법)
    for i in range(1, n - 1):
        A[i, i - 1] = 1 / h**2 # 왼쪽 대각선 요소
        A[i, i] = -2 / h**2    # 중앙 대각선 요소
        A[i, i + 1] = 1 / h**2 # 오른쪽 대각선 요소

    # 외부 힘 벡터 F 설정
    for i in range(1, n - 1):
        F[i] = function_f(x[i])

    # 경계 조건 적용
    A[0, 0] = 1
    F[0] = 1 # u(0) = 1

    A[-1, -1] = 1
    F[-1] = np.exp(1) # u(1) = e^1

    # 연립 방정식 Ax = F 해결
    U = np.linalg.solve(A, F)

    return x, U

# 입력 데이터
n = 50 # 격자 수

```

```

a, b = 0, 1 # 문제 정의 구간

# FDM 해 계산
x_fdm, u_fdm = fdm_solver(n, a, b)

# 정확한 해 계산
x_exact = np.linspace(a, b, 1000) # 매끄러운 x 좌표
u_exact = exact_solution(x_exact)

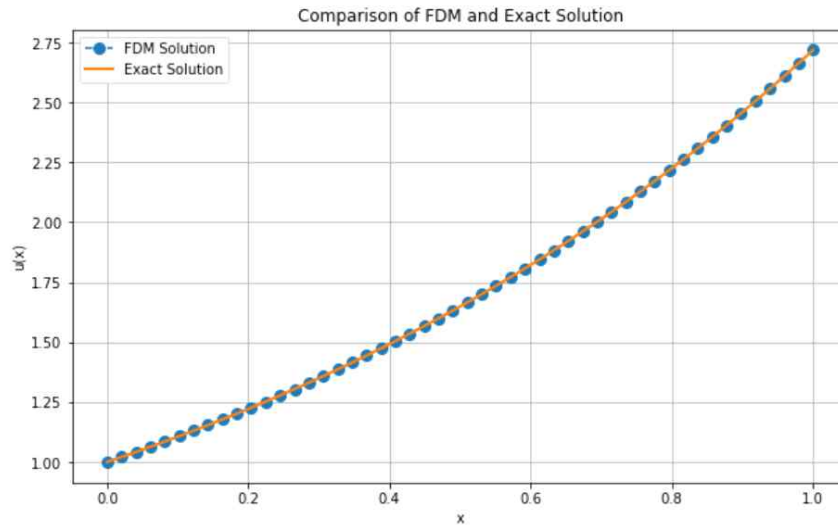
# 결과 시각화
plt.figure(figsize=(10, 6))
plt.plot(x_fdm, u_fdm, 'o--', label='FDM Solution', markersize=8)
plt.plot(x_exact, u_exact, '-', label='Exact Solution', linewidth=2)
plt.title('Comparison of FDM and Exact Solution')
plt.xlabel('x')
plt.ylabel('u(x)')
plt.grid(True)
plt.legend()
plt.show()

```

② 소스 코드 설명

코드에서는 $\frac{d^2u(x)}{dx^2} = e^x$, $x \in [0, 1]$, 로 주어진다. 경계 조건은 강성 행렬 $[A], [F]$ 에 직접 반영된다. $u(0) = 1$, $u(1) = e^1$ 의 처리는 $A[0, :] = A[-1, :] = 0$ 으로 초기화하고, 경계에서의 외부 힘은 즉 $F[0] = 1$, $F[-1] = e^1$ 로 한다.

③ 소스 코드 실행결과



(11.2.8) 1차원 비정상 열전달 FDM 수치해석

① 소스 코드

```
import numpy as np
import matplotlib.pyplot as plt

def Heat(param=3, n=4, m=24, plot_steps=10):
    # 초기화
    xO = 0; xL = 1 # 공간의 시작점 # 공간의 끝점
    tO = 0; tL = 1 # 시간의 시작점 # 시간의 끝점

    # 공간 및 시간 간격 계산
    dx = (xL - xO) / (n + 1) # 공간 간격
    dt = (tL - tO) / (m + 1) # 시간 간격

    a2 = 1; r = (a2 * dt) / dx**2 # 열전도 계수 # 안정성 계수

    # 행렬 A와 B 정의
    A = np.zeros((n, n)); B = np.zeros((n, n))

    for i in range(n):
        A[i, i] = 1 + 2 * r * (1 if param == 1 else 0.5) # 대각선 요소
        B[i, i] = 1 - 2 * r * (0.5 if param == 3 else 0)
        if i > 0:
            A[i, i-1] = -r * (1 if param in [1, 3] else 0) # 아래 대각선 요소
            B[i, i-1] = r * (0.5 if param == 3 else 0)
        if i < n-1:
```

```

A[i, i+1] = -r * (1 if param in [1, 3] else 0) # 위 대각선 요소
B[i, i+1] = r * (0.5 if param == 3 else 0)

# 공간 및 시간 좌표 설정
x = np.linspace(xO, xL, n + 2) # 공간 좌표
t = np.linspace(tO, tL, m + 2) # 시간 좌표

# 초기 조건 및 경계 조건
BC_L = lambda x, t: 0 # 왼쪽 경계 조건
BC_R = lambda x, t: 0 # 오른쪽 경계 조건
IC = lambda x, t: np.sin(np.pi * x) # 초기 조건

# 초기 벡터 b 설정
b = np.zeros((n, 1))
for j in range(n):
    b[j, 0] = IC(x[j + 1], t[0])

# 반복 계산 수행
bnew = b.copy(); results = []
results.append(np.concatenate((([BC_L(x[0], t[0])], b.flatten(), \
                                [BC_R(x[-1], t[0])])), axis=0))

for j in range(1, m + 2):
    if param == 0: # 명시적 방법
        bnew = B @ bnew
    elif param == 1: # 암시적 방법
        bnew = np.linalg.inv(A) @ bnew
    elif param == 3: # 크랭크-니콜슨 방법
        bnew = np.linalg.inv(A) @ (B @ bnew)
    else:
        raise ValueError("Invalid param value. Use 0 (Explicit), 1 (Implicit), or
3 (C-N).")

    col = bnew.flatten()
    results.append(np.concatenate((([BC_L(x[0], t[j])], col, [BC_R(x[-1], t[j])]),
axis=0))

# 결과를 2D 배열로 변환
results = np.array(results)

```

```

# 선택한 알고리즘 이름 결정
method_name = {0: "Explicit", 1: "Implicit", 3: "Crank-Nicholson"}.get(param,
"Unknown")

# 선 스타일 및 색상 설정
line_styles = ['-', '--', '-.', ':']
colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k']

# 시각화
plt.figure(figsize=(8, 4))
step_size = max(1, m // plot_steps)
for i, idx in enumerate(range(1, results.shape[0], step_size)):
    style = line_styles[i % len(line_styles)] # 선 스타일 순환
    color = colors[i % len(colors)] # 색상 순환
    plt.plot(x, results[idx, :], linestyle=style, color=color,
             marker='o', label=f"t={t[idx]:.2f}")

plt.xlabel("Space (x)")
plt.ylabel("Temperature (u)")
plt.title(f"1D Heat Transfer Solution ({method_name} Method)")

# legend를 오른쪽에 배치
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)
plt.grid(True)
plt.tight_layout() # 레이아웃 자동 조정
plt.show()

# 사용자 입력
param = int(input("Choose algorithm (0: Explicit, 1: Implicit, 3: Crank-Nicholson)
[default=3]: ") or 3)
n = int(input("Number of spatial divisions (n) [default=4]: ") or 4)
m = int(input("Number of time divisions (m) [default=24]: ") or 24)
plot_steps = int(input("Number of time steps to plot [default=10]: ") or 10)

Heat(param=param, n=n, m=m, plot_steps=plot_steps)

```

② 소스 코드 설명

이 코드는 사용자가 선택한 방법(명시적, 암시적, 크랭크-니콜슨)에 따라 열전도 방정식을 풀며, 각 방법은 위에서 설명한 수학적식에 따라 계산된다. 안정성과 정확성은 τ 값 및 방법에 따라 달라진다.

$r \leq 0.5$ 경우 명시적 방법이 안정하며, 크랭크-니콜슨 방법은 높은 정확성을 제공한다. 벡터 b 는 계산의 핵심 요소로, 각 시간 단계에서 공간 격자의 온도를 나타낸다.

```
b = np.zeros((n, 1))
for j in range(n):
    b[j, 0] = tso(x[j + 1], t[0])
```

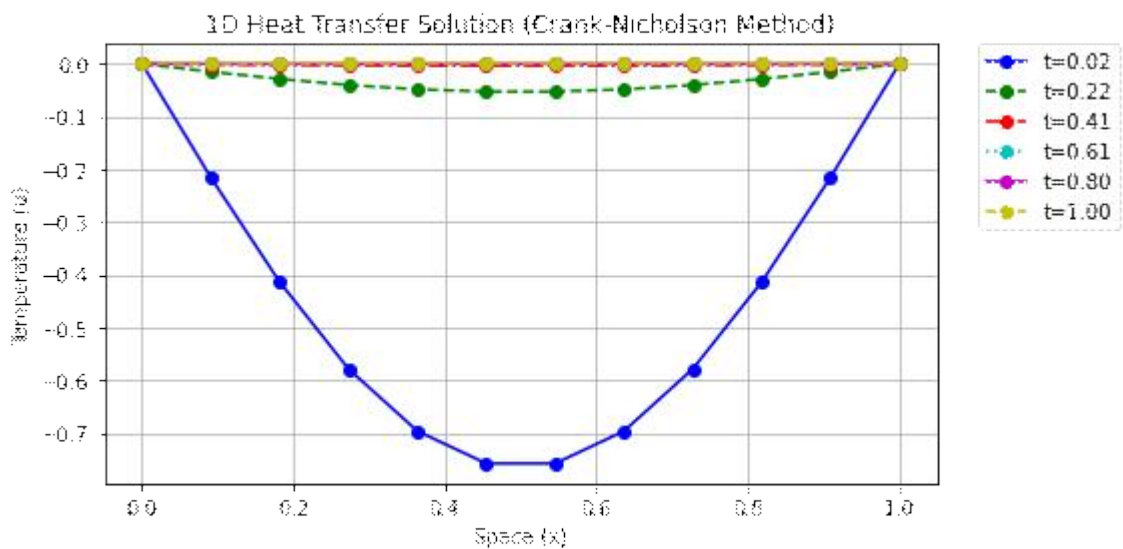
③ 소스 코드 실행결과

알고리즘 선택 (0: 명시적, 1: 암시적, 3: 크랭크-니콜슨) [기본값=3]: 3

공간 분할 수 (n) [기본값=4]: 10

시간 분할 수 (m) [기본값=24]: 50

그래프 출력 시간 단계 수 [기본값=10]: 5



12 장 유한요소법 (Finite Element Method)

(12.1) 유한요소법 개요

(12.2) 형상함수(Shape Function)

(12.3) 실제좌표계에서의 유한요소법(FEM)

(12.3.1) 실제좌표계에서의 유한요소법(FEM) 예제

이 코드는 식(12.3-10), 식(12.3-11)을 FEM을 구현한 것이다. 코드에서는 $-1 \frac{d^2 u(x)}{dx^2} + 0 \frac{du(x)}{dx} + 0u(x) = f(x), x \in [0, 1], f(x) = -e^x$ 로 주어진다. 경계 조건은 강성 행렬 $[K], [F]$ 에 직접 반영된다. $u(0) = 1, u(1) = e^1$ 의 처리는 $K[0,:][K[-1,:]] = 0$ 으로 초기화하고, $K[0,0][K[-1,-1]] = e^1$ 로 설정하고, 경계에서의 외부 힘은 0으로, 즉 $F[0] = 0, F[-1] = e^1$ 로 한다.

① 소스코드

```
import numpy as np
import matplotlib.pyplot as plt

class ode:
    def function_f(x):
        return -np.exp(x)
    def function_g(x):
        if x == 0:
            return 1.0 # u(0) = 1
        elif x == 1:
            return np.exp(1) # u(1) = e^1

# FEM 강성 행렬 계산 함수
```

```

def stiffness_matrix(n, h, k):
    K = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i == j:
                K[i, j] = 2 * k / h # 대각선 요소
            elif abs(i - j) == 1:
                K[i, j] = -k / h # 인접 요소
    return K

# FEM 외부 힘 벡터 계산 함수
def force_vector(n, h):
    F = np.zeros(n)
    for i in range(n):
        x_i = i * h # 노드 위치
        F[i] = ode.function_f(x_i) * h # 외부 힘 계산
        #F[i] = f(x_i) * h # 외부 힘 계산
    return F

# 이론해 함수
def corrected_analytical_solution(x):
    return np.exp(x)

# 입력 데이터
n = 5 # 노드 수; h = 1.0 / (n - 1) # 요소 간격; k = 1.0 # 계수

# FEM 계산
K = stiffness_matrix(n, h, k); F = force_vector(n, h)

# 디리클레 경계 조건 적용
K[0, :] = 0; K[0, 0] = 1; F[0] = ode.function_g(0)

K[-1, :] = 0; K[-1, -1] = 1
F[-1] = ode.function_g(1)

# FEM 해 구하기
U_fem = np.linalg.solve(K, F)

# x 좌표 생성 (FEM 노드)
x_fem = np.linspace(0, 1, n)

```

```

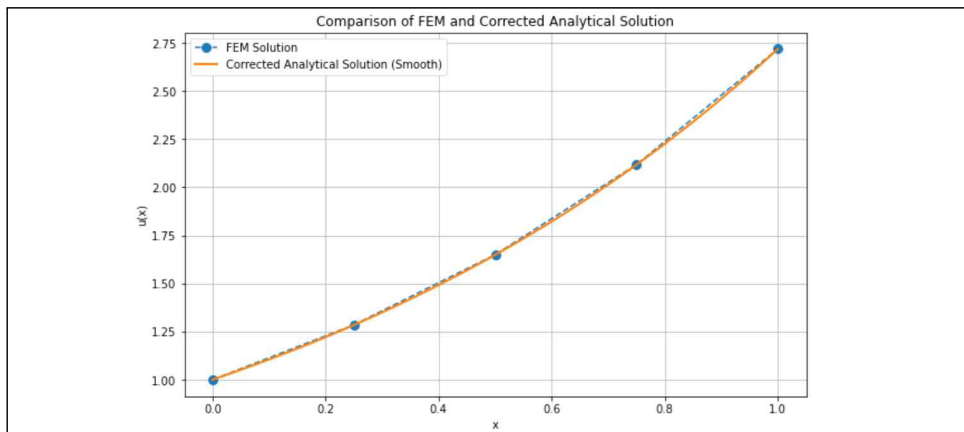
# x 좌표 생성 (Smooth 이론해)
x_smooth = np.linspace(0, 1, 1000) # 이론해를 위한 더 촘촘한 x 좌표
U_corrected = corrected_analytical_solution(x_smooth)

# 가시화
plt.figure(figsize=(10, 6))
plt.plot(x_fem, U_fem, 'o--', label='FEM Solution', markersize=8)
plt.plot(x_smooth, U_corrected, '-', label='Corrected Analytical Solution (Smooth)', linewidth=2)
plt.title('Comparison of FEM and Corrected Analytical Solution')
plt.xlabel('x'); plt.ylabel('u(x)')
plt.grid(True); plt.legend(); plt.show()

```

② 실행 결과

그래프에는 $u(x)$ 수치해석값이 표시된다. 수치해는 $u(x) = e^x$ 에 의해 지수 증가하고 양 끝에서 경계조건을 만족함을 관찰할 수 있다. 수치 실험 데이터를 비교하기 위해 추가 해석적 해(analytical solution), $u(x) = e^x$ 를 포함하고 있다.



(12.4) 기준좌표계에서의 유한요소법(FEM)

(12.4.1) 기준좌표계 변환의 필요성 및 중요성

(12.4.2) 기준좌표계에서의 1차원 FEM 해석

(12.4.3) 기준좌표계에서의 유한요소법(FEM) 코드

① 1차원 해석 코드

주어진 문제는

$$-\frac{d}{dx}\left(a(x)\frac{du(x)}{dx}\right) = f(x), \quad x \in [0,1] \quad (12.4.4-1)$$

$$u(0) = 0 \quad u(1) = \cos(1)$$

이다. 계수 및 소스 함수는 아래와 같다.

$$a(x) = e^x, \quad (12.4.4-2)$$

$$f(x) = e^{(x)}(\cos(x) - 2\sin(x) - x\cos(x) - x\sin(x))$$

정확한 해(Exact Solution)는 실제 해를 ODE에 대입하여 확인해 보자.

$$u(x) = x\cos(x) \quad (12.4.4-3)$$

계수 및 소스 함수 유한요소 분할은 `num_elements = int(input("Enter the number of elements:"))`을 이용해서 도메인을 `num_elements` 개의 요소로 나누고 각 요소의 길이를 계산, 요소의 국부 행렬(stiffness matrix)와 국부 벡터(load vector)를 정의한다. 어셈블 과정은 요소별 stiffness matrix와 load vector를 전역 행렬 및 벡터에 추가한다. $u(0) = 0$ 는 첫 번째 행렬과 벡터를 수정하여 경계값 적용, $u(1) = \cos(1)$ 은 간단히 디리클레 조건으로 변경하여 마지막 노드에서 값 적용한다. 솔버는 전역 행렬 `K`와 외력 벡터 `f`를 용해 선형 방정식을 풀어 수치 근사값을 계산하고, 계산된 $u(x)$ 값을 그래프로 시각화 한다. 이 코드를 실행 가능한 환경에서 실행하면 정확한 수치 결과와 그래프를 확인할 수 있다.

이 문제를 통해 1차원 FEM의 기본 원리를 이해하고, 수치 해석 결과를 정확한 해와 비교하여 FEM의 효율성과 정확성을 확인할 수 있다.

```
# -*- coding: UTF-8 -*-
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import linalg
```

```
# ODE 클래스 정의 - 방정식 정보 포함
class ode:
```

```
    def exact_solution(x):
```

```

    return np.exp(x)

def function_a(x):
    return 1.0

def function_f(x):
    return -np.exp(x)

def function_g(x):
    if x == 0:
        return 1.0
    elif x == 1:
        return np.exp(1)

# 기준 구간 [-1,1]에서 가우스 적분점과 가중치 생성
def generate_Gauss_reference_1D(nGQP):
    if nGQP == 4:
        J_1D = [0.3478548451, 0.3478548451, 0.6521451549, 0.6521451549]
        Gauss_point_reference_1D = [0.8611363116, -0.8611363116, 0.3399810436, -0.3399810436]
    elif nGQP == 2:
        J_1D = [1, 1]
        Gauss_point_reference_1D = [-1 / np.sqrt(3), 1 / np.sqrt(3)]
    return J_1D, Gauss_point_reference_1D

# 로컬 구간 [a, b]로 가우스 적분점과 가중치 변환
def generate_Gauss_local_1D(J_1D, Gauss_point_reference_1D, lower_bound, upper_bound):
    J_1D = (upper_bound - lower_bound) * np.array(J_1D) / 2
    x_i = (upper_bound - lower_bound) * np.array(Gauss_point_reference_1D) / 2 +
    (upper_bound + lower_bound) / 2
    return J_1D, x_i

# 로컬 형상 함수 정의
def SFN_1D(x, vertices, basis_index):
    if basis_index == 1:
        return (vertices[1] - x) / (vertices[1] - vertices[0])
    elif basis_index == 2:
        return (x - vertices[0]) / (vertices[1] - vertices[0])

def SFN_1D_der1(x, vertices, basis_index):
    if basis_index == 1:

```

```

        return -1 / (vertices[1] - vertices[0])
    elif basis_index == 2:
        return 1 / (vertices[1] - vertices[0])

# 요소 내에서 유한 요소 해 계산
def FE_solution_1D(x, uh_local, vertices):
    return uh_local[0] * SFN_1D(x, vertices, 1) + uh_local[1] * SFN_1D(x, vertices, 2)

# 강성 행렬과 하중 벡터 조립
def assemble_1D(left, right, num_elements, nGQP):
    h = (right - left) / num_elements
    nodes = np.linspace(left, right, num_elements + 1)
    A = np.zeros((num_elements + 1, num_elements + 1))
    b = np.zeros(num_elements + 1)

    J_1D, Gauss_point_reference_1D = generate_Gauss_reference_1D(nGQP)

    for i in range(num_elements):
        vertices = [nodes[i], nodes[i + 1]]
        J_1D_local, x_i = generate_Gauss_local_1D(
            J_1D, Gauss_point_reference_1D, vertices[0], vertices[1]
        )

        # 강성 행렬 계산
        for alpha in range(2):
            for beta in range(2):
                for q in range(len(J_1D_local)):
                    A[i + alpha, i + beta] += (
                        J_1D_local[q]
                        * ode.function_a(x_i[q])
                        * SFN_1D_der(x_i[q], vertices, alpha + 1)
                        * SFN_1D_der(x_i[q], vertices, beta + 1)
                    )

        # 하중 벡터 계산
        for q in range(len(J_1D_local)):
            b[i + alpha] += (
                J_1D_local[q]
                * ode.function_f(x_i[q])
                * SFN_1D(x_i[q], vertices, alpha + 1)

```

```

    )

    # 디리클레 경계 조건 적용
    A[0, :] = 0
    A[0, 0] = 1
    b[0] = ode.function_g(left)

    A[-1, :] = 0
    A[-1, -1] = 1
    b[-1] = ode.function_g(right)

    return A, b, nodes

def main():
    left, right = 0, 1
    nGQP = 4

    # 사용자 입력으로 요소의 개수 설정
    num_elements = int(input("Enter the number of elements: "))

    A, b, nodes = assemble_1D(left, right, num_elements, nGQP)
    uh = linalg.solve(A, b)

    # 정확한 해와 수치 해 시각화
    fine_x = np.linspace(left, right, 100)
    exact_y = ode.exact_solution(fine_x)

    numerical_y = []
    for x in fine_x:
        for i in range(len(nodes) - 1):
            if nodes[i] <= x <= nodes[i + 1]:
                numerical_y.append(FE_solution_1D(x, uh[i:i+2], [nodes[i], nodes[i+1]]))
                break
    numerical_y = np.array(numerical_y)

    plt.plot(fine_x, exact_y, label="Exact Solution", color="blue")
    plt.plot(fine_x[len(numerical_y):], numerical_y, label="FEM Solution", linestyle="--",
    color="red")

    # ODE 수식과 경계 조건 추가 (그래프와 겹침 방지)

```

```

plt.figtext(0.15, 0.65, r"$\frac{d}{dx}\left(a(x)\frac{du}{dx}\right) = f(x)$", fontsize=12,
color="green")
plt.figtext(0.15, 0.5, r"$u(0) = 1, \ u(1) = \exp(1)$", fontsize=12, color="purple")

plt.xlabel("x")
plt.ylabel("ODE Solution")
plt.legend()
plt.title("Comparison of exact and FEM solutions with ODE and Boundary Conditions")
plt.grid()
plt.show()

if __name__ == "__main__":
    main()

```

